Product Requirements Document (PRD): Distributed Smart Meeting Room Occupancy System

1. System Architecture Overview

Sistem terdistribusi menggunakan pola N-Tier untuk memisahkan akuisisi data fisik dengan logika bisnis reservasi. Sistem mendeteksi tingkat okupansi ruang meeting secara real-time dan mensinkronisasikannya dengan Google Calendar untuk mengotomatisasi manajemen ruangan dan mencegah *ghost meetings*.

Arsitektur terdiri dari tiga layer utama:

1. **Hardware & Edge Layer:** Sensor dan lokal prosesor.
2. **IoT Gateway Backend (.NET):** Layanan terpusat untuk *telemetry ingestion* dan *state management*.
3. **Meeting Room Booking Backend (.NET):** Layanan *core business* untuk reservasi dan integrasi pihak ketiga.

2. Hardware & Edge Layer (Raspberry Pi)

Fokus pada akuisisi data fisik dan transmisi telemetri ke *backend* terpusat.

- **Sensor:** HLK-LD2410B (24GHz FMCW Radar). Sensitivitas tinggi terhadap pergerakan mikro dan statis.
- **Konektivitas Sensor:** Komunikasi Serial (UART) RX/TX dengan *baud rate* 256000 ke Raspberry Pi.
- **Edge Processing Logic:**
    - Membaca *byte frame* dari pin UART secara terus-menerus.
    - **Debouncing Algorithm:** Menggunakan logika *time-to-empty*. Jika sensor mengubah status menjadi Empty, sistem melakukan *delay* 30 detik sebelum memvalidasi status untuk mencegah *false-negative* saat target diam sepenuhnya.
- **Konektivitas Jaringan:** Isolasi jaringan *edge* menggunakan Local Network ke server IoT Gateway. HTTP Connection.
- **Protokol Pengiriman:** HTTP over CORS pada masing-masing Raspberry Pi untuk autentikasi *device identity*.
- **Telemetry Payload (JSON):**

- JSON

None

```
{
  "roomId": "string",
  "timestamp": "ISO8601",
  "isOccupied": boolean,
  "stationaryTargetDistance": number,
  "movingTargetDistance": number
}
```

3. IoT Gateway Backend Layer

Layanan untuk menangani koneksi perangkat, memproses telemetri, dan menjaga *state* perangkat.

- **Framework:** ASP.NET Core Worker Service.
- **Arsitektur:** Clean Architecture dengan pemisahan *layer* (Domain, Application, Infrastructure).
- **Event Payload (RabbitMQ):**
  - **Exchange:** `room.events`
  - **Routing Key:** `room.state.changed`
- JSON

None

```
{
  "eventId": "UUID",
  "roomId": "string",
  "previousState": boolean,
  "newState": boolean,
  "changedAt": "ISO8601",
  "triggerSource": "IoT_Gateway"
}
```

-

4. Meeting Room Booking Backend Layer

Layanan untuk mengatur logika reservasi dan menyediakan API integrasi.

- **Framework:** ASP.NET Core Web API.
- **Arsitektur:** Clean Architecture dengan implementasi CQRS (Command Query Responsibility Segregation).
- **Integrasi IoT:** Mengonsumsi *event* dari RabbitMQ (`room.events`).
- **Integrasi Google Calendar API:**
    - **Autentikasi:** Implementasi wajib menggunakan OAuth 2.0. Penggunaan Service Account tidak diizinkan karena keterbatasan dalam menambahkan *attendees* dan meng-generate Google Meet links.
    - **Sinkronisasi:** Menggunakan Google Calendar Push Notifications (Webhooks) untuk menangkap perubahan langsung dari sisi Google Calendar.
    - **Resiliency:** Menggunakan library `Polly` untuk implementasi pola *Retry* dan *Circuit Breaker* terhadap *rate limit* atau *downtime* Google API.

5. Core Features & Business Logic

5.1. Real-time Status Monitoring

Status ketersediaan ruangan ditentukan berdasarkan kombinasi jadwal Google Calendar dan data IoT Gateway:

- **Available:** Tidak ada jadwal kalender, ruangan terdeteksi kosong.
- **Occupied:** Ada jadwal kalender, ruangan terdeteksi ada orang.
- **Unscheduled Occupancy:** Tidak ada jadwal kalender, ruangan terdeteksi ada orang.

5.2. Auto-Release Logic (Anti-Ghost Meeting)

Mencegah ruangan berstatus dipesan tetapi secara fisik kosong.

- Parameter: 15 menit dari waktu mulai jadwal.
- Logika: Jika *event* dimulai pada 10:00 dan hingga 10:15 *state* IoT Gateway (`isOccupied`) tetap bernilai `false`, sistem otomatis membatalkan reservasi tersebut via Google Calendar API.
- Ruangan otomatis kembali ke status *Available*.

5.3. Ad-Hoc Booking

Menangani *Unscheduled Occupancy*.

- Parameter: 5 menit.
- Logika: Jika sensor mendeteksi kehadiran berkelanjutan selama 5 menit pada ruangan yang *Available*, sistem otomatis membuat *event* baru di Google Calendar berdurasi 30 menit ke depan untuk mengamankan slot ruangan.

6. Development, CI/CD & Quality Standards

- **Design Patterns & Libraries:** Penerapan DTO, AutoMapper, dan FluentValidation untuk seluruh proses *request handling*.
- **CI/CD Pipeline:** Konfigurasi *multi-branch pipelines* menggunakan Jenkins.
- **Static Analysis:** Semua *Pull Request* harus dianalisis menggunakan JetBrains Qodana. Eksekusi *pipeline* di Jenkins dikonfigurasi untuk gagal (*fail build*) jika Resharper Inspection mendeteksi *warning* terkait kualitas kode atau pelanggaran arsitektur.

Product Requirements Document (PRD): Distributed Smart Meeting Room Occupancy System

1. System Architecture Overview

Sistem terdistribusi menggunakan pola N-Tier untuk memisahkan akuisisi data fisik dengan logika bisnis reservasi. Sistem mendeteksi tingkat okupansi ruang meeting secara real-time dan mensinkronisasikannya dengan Google Calendar untuk mengotomatisasi manajemen ruangan dan mencegah *ghost meetings*.

Arsitektur terdiri dari tiga layer utama:

- **Hardware & Edge Layer:** Sensor dan lokal prosesor.
- **IoT Gateway Backend (.NET):** Layanan terpusat untuk *telemetry ingestion*, *state management*, serta manajemen pemetaan perangkat ke ruangan menggunakan In-Memory Cache.
- **Meeting Room Booking Backend (.NET):** Layanan *core business* sebagai *Single Source of Truth* (SSOT) untuk entitas ruangan, reservasi, dan integrasi pihak ketiga.

2. Hardware & Edge Layer (Raspberry Pi)

Fokus pada akuisisi data fisik dan transmisi telemetri ke backend terpusat. Perangkat keras di lapangan beroperasi sebagai entitas *stateless* terhadap domain bisnis (tidak mengetahui konteks logis ruangan).

- **Sensor:** HLK-LD2410B (24GHz FMCW Radar). Sensitivitas tinggi terhadap pergerakan mikro dan statis.
- **Konektivitas Sensor:** Komunikasi Serial (UART) RX/TX dengan *baud rate* 256000 ke Raspberry Pi.
- **Edge Processing Logic:**
    - Membaca *byte frame* dari pin UART secara terus-menerus.
    - **Debouncing Algorithm:** Menggunakan logika *time-to-empty*. Jika sensor mengubah status menjadi `Empty`, sistem melakukan *delay* 30 detik sebelum memvalidasi status untuk mencegah *false-negative* saat target diam sepenuhnya.
- **Konektivitas Jaringan:** Isolasi jaringan *edge* menggunakan Local Network ke server IoT Gateway.
- **Protokol Pengiriman:** HTTP POST pada masing-masing Raspberry Pi menggunakan Bearer/Auth Token untuk autentikasi *device identity*.
- **Telemetry Payload (JSON):**

- *(Catatan Arsitektur: Payload dikoreksi untuk menggunakan `deviceId`, bukan `roomId`, guna mematuhi prinsip Bounded Context. Sensor fisik tidak boleh memiliki dependensi ID terhadap sistem kalender).*
- JSON

None

-

```
{
  "deviceId": "string (MAC Address / GUID)",
  "timestamp": "ISO8601",
  "isOccupied": boolean,
  "stationaryTargetDistance": number,
  "movingTargetDistance": number
}
```

-

-

3. IoT Gateway Backend Layer

Layanan untuk menangani koneksi perangkat, memproses telemetri *hot path*, dan menjaga *state* pemetaan.

- **Framework:** ASP.NET Core Web API (untuk Device Management REST API) & ASP.NET Core Worker Service (untuk *telemetry ingestion* dan integrasi RabbitMQ).
- **Arsitektur:** Clean Architecture dengan pemisahan layer (Domain, Application, Infrastructure).
- **Replikasi Data Terdistribusi:** Mengonsumsi *event* dari Booking Backend untuk menyimpan *Read-Model* lokal dari data ruangan guna mencegah dependensi API sinkron yang rawan *failure*.
- **In-Memory Caching (Redis):** Menyimpan *mapping* antara `DeviceId` dan `RoomId`. Saat telemetri masuk, *gateway* melakukan *lookup* O(1) ke Redis sebelum meneruskan *event* perubahan status ke sistem *booking*.
- **Event Payload Outgoing (RabbitMQ):**
  - **Exchange:** `room.events`
  - **Routing Key:** `room.state.changed`
- JSON

None

-

```
{
  "eventId": "UUID",
  "roomId": "string",
  "previousState": boolean,
  "newState": boolean,
  "changedAt": "ISO8601",
  "triggerSource": "IoT_Gateway"
}
```

4. Meeting Room Booking Backend Layer

Layanan untuk mengatur logika reservasi dan menyediakan API integrasi. Bertindak sebagai penguasa domain (SSOT) untuk entitas ruangan.

- **Framework:** ASP.NET Core Web API.
- **Arsitektur:** Clean Architecture dengan implementasi CQRS (Command Query Responsibility Segregation).
- **Integrasi IoT:**
  - **Inbound:** Mengonsumsi *event* perubahan status dari RabbitMQ (`room.events`).
  - **Outbound:** Mempublikasikan *Integration Event* ke RabbitMQ (Exchange: `booking.events`, Routing Key: `room.created`/`room.updated`) setiap kali ada penambahan atau perubahan data ruangan oleh Administrator.
- **Integrasi Google Calendar API:**
  - **Autentikasi:** Implementasi wajib menggunakan OAuth 2.0. Penggunaan Service Account tidak diizinkan karena keterbatasan dalam menambahkan *attendees* dan meng-generate Google Meet links.
  - **Sinkronisasi:** Menggunakan Google Calendar Push Notifications (Webhooks) untuk menangkap perubahan langsung dari sisi Google Calendar.
  - **Resiliency:** Menggunakan library Polly untuk implementasi pola *Retry* dan *Circuit Breaker* terhadap *rate limit* atau *downtime* Google API.

5. Core Features & Business Logic

5.1. Real-time Status Monitoring

Status ketersediaan ruangan ditentukan berdasarkan kombinasi jadwal Google Calendar dan data IoT Gateway:

- **Available:** Tidak ada jadwal kalender, ruangan terdeteksi kosong.
- **Occupied:** Ada jadwal kalender, ruangan terdeteksi ada orang.
- **Unscheduled Occupancy:** Tidak ada jadwal kalender, ruangan terdeteksi ada orang.

5.2. Auto-Release Logic (Anti-Ghost Meeting)

Mencegah ruangan berstatus dipesan tetapi secara fisik kosong.

- **Parameter:** 15 menit dari waktu mulai jadwal.
- **Logika:** Jika event dimulai pada 10:00 dan hingga 10:15 *state* IoT Gateway (`isOccupied`) tetap bernilai `false`, sistem otomatis membatalkan reservasi tersebut via Google Calendar API.
- Ruangan otomatis kembali ke status *Available*.

5.3. Ad-Hoc Booking

Menangani *Unscheduled Occupancy*.

- **Parameter:** 5 menit.
- **Logika:** Jika sensor mendeteksi kehadiran berkelanjutan selama 5 menit pada ruangan yang *Available*, sistem otomatis membuat *event* baru di Google Calendar berdurasi 30 menit ke depan untuk mengamankan slot ruangan.

5.4. Device Management & Dynamic Binding

Pengelolaan inventaris perangkat keras dan pemetaannya ke ruangan logis di dalam Booking Backend.

- **Domain Entities (IoT Gateway):**
    1. `Device`: Identitas fisik perangkat (`Id`, `MacAddress`, `AuthToken`, `Status`, `LastPingAt`).
    2. `Room`: *Read-model* dari Booking Backend (`Id`, `Name`).
    3. `RoomDeviceMapping`: Tabel relasional *binding* perangkat ke ruangan.
- **Alur Binding:**
    1. Teknisi mendaftarkan perangkat baru via REST API IoT Gateway untuk mendapatkan *AuthToken*.
    2. Melalui *dashboard*, perangkat ditautkan (*bind*) ke `RoomId` yang tersedia di *Read-Model*.
    3. Saat transaksi *binding* berhasil disimpan ke database SQL/SQLite, IoT Gateway secara paksa memperbarui *cache* pemetaan di Redis (`device:{DeviceId}:mapping -> RoomId`) untuk mendukung pemrosesan telemetri latensi rendah pada *hot path*.

6. Development, CI/CD & Quality Standards

- **Design Patterns & Libraries:** Penerapan DTO, AutoMapper, dan FluentValidation untuk seluruh proses *request handling*.
- **CI/CD Pipeline:** Konfigurasi *multi-branch pipelines* menggunakan Jenkins.
- **Static Analysis:** Semua *Pull Request* harus dianalisis menggunakan JetBrains Qodana. Eksekusi *pipeline* di Jenkins dikonfigurasi untuk gagal (*fail build*) jika Resharper Inspection mendeteksi *warning* terkait kualitas kode atau pelanggaran arsitektur.

Product Requirements Document (PRD): Distributed Smart Meeting Room Occupancy System

1. System Architecture Overview

Sistem terdistribusi menggunakan pola N-Tier untuk memisahkan akuisisi data fisik dengan logika bisnis reservasi. Sistem mendeteksi tingkat okupansi ruang meeting secara real-time dan mensinkronisasikannya dengan Google Calendar untuk mengotomatisasi manajemen ruangan dan mencegah *ghost meetings*.

Arsitektur terdiri dari tiga layer utama:

- **Hardware & Edge Layer:** Sensor dan lokal prosesor.
- **IoT Gateway Backend (.NET):** Layanan terpusat untuk *telemetry ingestion*, *state management*, serta manajemen pemetaan perangkat ke ruangan menggunakan In-Memory Cache.
- **Meeting Room Booking Backend (.NET):** Layanan *core business* sebagai *Single Source of Truth* (SSOT) untuk entitas ruangan, reservasi, dan integrasi pihak ketiga.

2. Hardware & Edge Layer (Raspberry Pi)

Fokus pada akuisisi data fisik dan transmisi telemetri ke backend terpusat. Perangkat keras di lapangan beroperasi sebagai entitas *stateless* terhadap domain bisnis (tidak mengetahui konteks logis ruangan).

- **Sensor:** HLK-LD2410B (24GHz FMCW Radar). Sensitivitas tinggi terhadap pergerakan mikro dan statis.
- **Konektivitas Sensor:** Komunikasi Serial (UART) RX/TX dengan *baud rate* 256000 ke Raspberry Pi.
- **Edge Processing Logic:**
    - Membaca *byte frame* dari pin UART secara terus-menerus.
    - **Debouncing Algorithm:** Menggunakan logika *time-to-empty*. Jika sensor mengubah status menjadi `Empty`, sistem melakukan *delay* 30 detik sebelum memvalidasi status untuk mencegah *false-negative* saat target diam sepenuhnya.
- **Konektivitas Jaringan:** Isolasi jaringan *edge* menggunakan Local Network ke server IoT Gateway.
- **Protokol Pengiriman:** HTTP POST pada masing-masing Raspberry Pi menggunakan Bearer/Auth Token untuk autentikasi *device identity*.
- **Telemetry Payload (JSON):**

- *(Catatan Arsitektur: Payload dikoreksi untuk menggunakan deviceId, bukan roomId, guna mematuhi prinsip Bounded Context. Sensor fisik tidak boleh memiliki dependensi ID terhadap sistem kalender).*

JSON

None

```
{
  "deviceId": "string (MAC Address / GUID)",
  "timestamp": "ISO8601",
  "isOccupied": boolean,
  "stationaryTargetDistance": number,
  "movingTargetDistance": number
}
```

3. IoT Gateway Backend Layer

Layanan untuk menangani koneksi perangkat, memproses telemetri *hot path*, dan menjaga *state* pemetaan.

- **Framework:** ASP.NET Core Web API (untuk Device Management REST API) & ASP.NET Core Worker Service (untuk *telemetry ingestion* dan integrasi RabbitMQ).
- **Arsitektur:** Clean Architecture dengan pemisahan layer (Domain, Application, Infrastructure).
- **Database & Storage Strategy (Polyglot Persistence):**
    - **Hot Path (Telemetry Ingestion & State):** Menggunakan **Redis**. Diaplikasikan untuk menyimpan *state* okupansi sementara dan *lookup cache* pemetaan `DeviceId` ke `RoomId` untuk mencapai performa *routing* O(1). Data *raw telemetry* berfrekuensi tinggi dari HTTP POST dilarang masuk ke RDBMS.
    - **Cold Path (Metadata & Device Management):** Menggunakan **RDBMS** yang dikelola melalui **Entity Framework Core (EF Core)** pada layer *Infrastructure*. Menggunakan **MySQL** (untuk arsitektur terpusat) atau **SQLite** (untuk *deployment on-premise/edge gateway* yang lebih mandiri). Digunakan khusus untuk operasi CRUD entitas logis `Device`, *Read-Model* `Room`, dan tabel relasional `RoomDeviceMapping` dengan jaminan integritas ACID.
- **Replikasi Data Terdistribusi:** Mengonsumsi *event* dari Booking Backend untuk menyimpan *Read-Model* lokal dari data ruangan guna mencegah dependensi API sinkron yang rawan *failure*.
- **Event Payload Outgoing (RabbitMQ):**

    - **Exchange:** room.events
    - **Routing Key:** room.state.changed

JSON

None

```
{
  "eventId": "UUID",
  "roomId": "string",
  "previousState": boolean,
  "newState": boolean,
  "changedAt": "ISO8601",
  "triggerSource": "IoT_Gateway"
}
```

4. Meeting Room Booking Backend Layer

Layanan untuk mengatur logika reservasi dan menyediakan API integrasi. Bertindak sebagai penguasa domain (SSOT) untuk entitas ruangan.

- **Framework:** ASP.NET Core Web API.
- **Arsitektur:** Clean Architecture dengan implementasi CQRS (Command Query Responsibility Segregation).
- **Integrasi IoT:**
    - **Inbound:** Mengonsumsi *event* perubahan status dari RabbitMQ (room.events).
    - **Outbound:** Mempublikasikan *Integration Event* ke RabbitMQ (Exchange: booking.events, Routing Key: room.created/room.updated) setiap kali ada penambahan atau perubahan data ruangan oleh Administrator.
- **Integrasi Google Calendar API:**
    - **Autentikasi:** Implementasi wajib menggunakan OAuth 2.0. Penggunaan Service Account tidak diizinkan karena keterbatasan dalam menambahkan *attendees* dan meng-generate Google Meet links.
    - **Sinkronisasi:** Menggunakan Google Calendar Push Notifications (Webhooks) untuk menangkap perubahan langsung dari sisi Google Calendar.
    - **Resiliency:** Menggunakan library Polly untuk implementasi pola *Retry* dan *Circuit Breaker* terhadap *rate limit* atau *downtime* Google API.

5. Core Features & Business Logic

5.1. Real-time Status Monitoring

Status ketersediaan ruangan ditentukan berdasarkan kombinasi jadwal Google Calendar dan data IoT Gateway:

- **Available:** Tidak ada jadwal kalender, ruangan terdeteksi kosong.
- **Occupied:** Ada jadwal kalender, ruangan terdeteksi ada orang.
- **Unscheduled Occupancy:** Tidak ada jadwal kalender, ruangan terdeteksi ada orang.

5.2. Auto-Release Logic (Anti-Ghost Meeting)

Mencegah ruangan berstatus dipesan tetapi secara fisik kosong.

- **Parameter:** 15 menit dari waktu mulai jadwal.
- **Logika:** Jika event dimulai pada 10:00 dan hingga 10:15 *state* IoT Gateway (`isOccupied`) tetap bernilai `false`, sistem otomatis membatalkan reservasi tersebut via Google Calendar API.
- Ruangan otomatis kembali ke status *Available*.

5.3. Ad-Hoc Booking

Menangani *Unscheduled Occupancy*.

- **Parameter:** 5 menit.
- **Logika:** Jika sensor mendeteksi kehadiran berkelanjutan selama 5 menit pada ruangan yang *Available*, sistem otomatis membuat *event* baru di Google Calendar berdurasi 30 menit ke depan untuk mengamankan slot ruangan.

5.4. Device Management & Dynamic Binding

Pengelolaan inventaris perangkat keras dan pemetaannya ke ruangan logis di dalam Booking Backend.

- **Domain Entities (IoT Gateway):**
    1. `Device`: Identitas fisik perangkat (`Id`, `MacAddress`, `AuthToken`, `Status`, `LastPingAt`).
    2. `Room`: *Read-model* dari Booking Backend (`Id`, `Name`).
    3. `RoomDeviceMapping`: Tabel relasional *binding* perangkat ke ruangan.
- **Alur Binding:**
    1. Teknisi mendaftarkan perangkat baru via REST API IoT Gateway untuk mendapatkan `AuthToken`.
    2. Melalui *dashboard*, perangkat ditautkan (*bind*) ke `RoomId` yang tersedia di *Read-Model*.

3. Saat transaksi *binding* berhasil disimpan ke database (via EF Core), IoT Gateway secara paksa memperbarui *cache* pemetaan di Redis (`device:{DeviceId}:mapping` -> `RoomId`) untuk mendukung pemrosesan telemetri latensi rendah pada *hot path*.

6. Development, CI/CD & Quality Standards

- **Design Patterns & Libraries:** Penerapan DTO, AutoMapper, dan FluentValidation untuk seluruh proses *request handling*.
- **CI/CD Pipeline:** Konfigurasi *multi-branch pipelines* menggunakan Jenkins.
- **Static Analysis:** Semua *Pull Request* harus dianalisis menggunakan JetBrains Qodana. Eksekusi *pipeline* di Jenkins dikonfigurasi untuk gagal (*fail build*) jika Resharper Inspection mendeteksi *warning* terkait kualitas kode atau pelanggaran arsitektur.

Product Requirements Document (PRD): Simplified HTTP-Based Smart Meeting Room Occupancy System

1. System Architecture Overview

Arsitektur sistem terdistribusi menggunakan pola N-Tier yang disederhanakan murni menggunakan komunikasi HTTP/REST. Ketergantungan terhadap *message broker* (RabbitMQ) dan *in-memory data store* eksternal (Redis) dihilangkan untuk meminimalkan kompleksitas infrastruktur.

Sistem terdiri dari tiga layer utama:

- **Hardware & Edge Layer:** Sensor dan prosesor lokal (Raspberry Pi) yang mengirimkan telemetri via HTTP POST.
- **IoT Gateway Backend (.NET):** Layanan terpusat pengelola *state* perangkat dengan *In-Process Caching* murni. Bertindak sebagai pengirim Webhook ke sistem *booking*.
- **Meeting Room Booking Backend (.NET):** Layanan *core business* (SSOT) untuk logika reservasi, penerima Webhook okupansi, dan integrasi Google Calendar tanpa *boilerplate* CQRS.

2. Hardware & Edge Layer (Raspberry Pi)

Fokus pada akuisisi data fisik dan transmisi telemetri secara *stateless*.

- **Sensor Utama:** HLK-LD2410B (24GHz FMCW Radar).
- **Konektivitas Sensor:** Serial (UART) RX/TX, *baud rate* 256000.
- **Edge Processing Logic:**
    - Pembacaan *byte frame* UART.
    - **Debouncing Algorithm:** *Delay* 30 detik sebelum validasi transisi status dari `Occupied` menjadi `Empty` untuk mencegah *false-negative*.
- **Protokol Pengiriman:** HTTP POST dengan *header* `Authorization: Bearer {AuthToken}`.
- **Optimasi Jaringan:** Implementasi wajib **HTTP Keep-Alive** pada level OS/Library Edge untuk menjaga soket TCP tetap terbuka guna pengiriman frekuensi tinggi.

3. IoT Gateway Backend Layer

Layanan *stateless* untuk *routing* dan komputasi *hot path*.

- **Framework:** ASP.NET Core Web API.
- **Arsitektur:** Clean Architecture.
- **Database & Storage Strategy:**

    - **Hot Path (Telemetry Ingestion):** `IMemoryCache` (Singleton). Menyimpan *state* okupansi sementara dan *lookup mapping* `DeviceId` ke `RoomId`. Akses memori O(1).
    - **Cold Path (Device Management):** SQLite dikelola via Entity Framework Core (EF Core). Menangani operasi CRUD untuk entitas `Device` dan `RoomDeviceMapping`.
- **HTTP Webhook Dispatcher:** Menggunakan `HttpClientFactory` dengan implementasi `Polly` (Retry, Circuit Breaker) untuk menembak *endpoint* Webhook Booking Backend saat terjadi perubahan status ruangan.

4. Meeting Room Booking Backend Layer

Layanan *core business* dan penguasa domain ruangan.

- **Framework:** ASP.NET Core Web API.
- **Arsitektur:** Clean Architecture (N-Tier) berbasis **Service-Repository Pattern**. Mengeliminasi penggunaan CQRS dan MediatR untuk mereduksi *boilerplate code*. Lapisan *Controller* langsung menginjeksi *Application Services* (misal: `IOccupancyService`, `IRoomService`) yang memuat seluruh *business logic* dan berinteraksi langsung dengan *Repository*.
- **Integrasi Google Calendar API:**
    - **Autentikasi:** OAuth 2.0 (Delegated Access). *Service Account* dilarang secara arsitektural.
    - **Sinkronisasi:** Google Calendar Push Notifications (Webhooks).
- **Real-time Client Updates:** Implementasi ASP.NET Core SignalR untuk mendorong perubahan status ruangan secara instan ke antarmuka klien (Web).

5. Alur Data & Komunikasi (Pure HTTP)

1. **Fase 1: Transmisi Edge -> IoT Gateway**
    - Raspberry Pi mengirim HTTP POST secara kontinu ke Gateway.
    - Gateway mengevaluasi *payload* di RAM (`IMemoryCache`). Jika status tidak berubah, respons HTTP 200 OK dikembalikan, *payload* dibuang. Tidak ada operasi *database* relasional.
2. **Fase 2: Sinkronisasi IoT Gateway -> Booking Backend (Webhook)**
    - Jika terdeteksi transisi status (misal: masuk ruangan), IoT Gateway memanggil API Webhook milik Booking Backend secara sinkron/asinkron berbasis HTTP.
3. **Fase 3: Sinkronisasi Metadata Ruangan (Direct HTTP GET)**
    - Replikasi data asinkron dihapus. Saat teknisi membuka *dashboard* IoT Gateway untuk melakukan *binding* perangkat keras, IoT Gateway melakukan HTTP GET secara langsung ke Booking Backend untuk mengambil daftar ruangan aktif.

6. Core Features & Business Logic

- **Auto-Release Logic (Anti-Ghost Meeting):** Jika sesi dimulai pada 10:00 dan hingga 10:15 status dari IoT Gateway adalah `IsPhysicallyOccupied == false`, sistem membatalkan reservasi di Google API.
- **Ad-Hoc Booking:** Deteksi kehadiran fisik selama 5 menit berturut-turut pada ruangan berstatus *Available* akan memicu pembuatan reservasi Google Calendar instan selama 30 menit ke depan.

7. Development, CI/CD & Quality Standards

- **Standar Kode:** N-Tier, pemisahan DTO yang ketat, AutoMapper, dan FluentValidation.
- **CI/CD Pipeline:** Implementasi *multi-branch pipeline* pada Jenkins.
- **Static Analysis:** Pemindaian wajib menggunakan JetBrains Qodana. Peringatan arsitektural dari Resharper *inspection* akan menggagalkan proses *build* (*fail fast*).

---

8. API Contracts (Interface Agreements)

Kontrak API ini mendefinisikan komunikasi antar-komponen dalam arsitektur yang telah disederhanakan.

8.1. Edge Device -> IoT Gateway

Fungsi: Mengirimkan telemetri status fisik ruangan secara kontinu.

- **Endpoint:** `POST /api/telemetry`
- **Host:** IoT Gateway
- **Headers:** `Authorization: Bearer {DeviceToken}`
- **Request Body (JSON):**
- `JSON`

None

-

```
{
  "deviceId": "string",
  "timestamp": "2026-05-04T18:50:41Z",
  "isOccupied": true,
  "stationaryTargetDistance": 1.2,
  "movingTargetDistance": 0.0
}
```

-
-
- **Response:**
    - 200 OK (Diterima dan status tidak berubah atau berhasil dievaluasi)
    - 401 Unauthorized (Token tidak valid)

8.2. IoT Gateway -> Booking Backend (Occupancy Webhook)

Fungsi: Memberitahu sistem reservasi bahwa telah terjadi perubahan state fisik secara definitif.

- **Endpoint:** POST /api/webhooks/occupancy
- **Host:** Booking Backend
- **Headers:** X-Gateway-Secret: {SharedSecret}
- **Request Body (JSON):**
- JSON

None

-

```
{
  "eventId": "guid",
  "roomId": "guid",
  "isOccupied": true,
  "changedAt": "2026-05-04T18:50:41Z"
}
```

-
- **Response:**
    - 200 OK (Webhook diterima dan dieksekusi langsung oleh *Service Layer*)
    - 400 Bad Request (Struktur *payload* salah)

8.3. IoT Gateway -> Booking Backend (Room Synchronization)

Fungsi: Memuat daftar ruangan aktif dari sistem reservasi untuk ditampilkan pada antarmuka manajemen perangkat IoT Gateway.

- **Endpoint:** GET /api/rooms/active
- **Host:** Booking Backend

- **Headers:** X-Gateway-Secret: {SharedSecret}
- **Response Body (JSON):**
- JSON

None

```
[
  {
    "roomId": "guid",
    "roomName": "Meeting Room A",
    "googleCalendarResourceId": "string"
  },
  {
    "roomId": "guid",
    "roomName": "Meeting Room B",
    "googleCalendarResourceId": "string"
  }
]
```

8.4. Administrator -> IoT Gateway (Device Binding)

Fungsi: Menautkan entitas perangkat keras fisik dengan entitas ruangan logis.

- **Endpoint:** POST /api/devices/bind
- **Host:** IoT Gateway
- **Request Body (JSON):**
- JSON

None

```
{
  "deviceId": "guid",
  "roomId": "guid"
}
```

- **Response:**
    - `200 OK` (Pembaruan disimpan di RDBMS SQLite dan `IMemoryCache` di-*refresh*)
    - `404 Not Found` (Device ID tidak dikenali)

Oke, ini aku tulis ulang PRD kamu secara utuh + sudah diintegrasikan dengan kebutuhan **analytics & grafik**, tanpa merusak filosofi awal (simple, HTTP-only, no broker).

Product Requirements Document (PRD)

Simplified HTTP-Based Smart Meeting Room Occupancy System

---

1. System Architecture Overview

Sistem menggunakan arsitektur terdistribusi berbasis **N-Tier** yang disederhanakan dengan komunikasi **HTTP/REST** secara penuh.

Untuk menjaga kesederhanaan dan kemudahan operasional:

- **Tidak menggunakan message broker** (RabbitMQ, Kafka)
- **Tidak menggunakan distributed cache eksternal** (Redis)

Sistem terdiri dari tiga layer utama:

- **Hardware & Edge Layer**: Sensor fisik + Raspberry Pi untuk akuisisi data okupansi
- **IoT Gateway Backend (.NET)**: Pengelola state real-time berbasis memory
- **Meeting Room Booking Backend (.NET)**: Core business logic + Single Source of Truth (SSOT)

---

2. Hardware & Edge Layer (Raspberry Pi)

Bertanggung jawab untuk membaca kondisi fisik ruangan dan mengirim telemetry secara stateless.

Spesifikasi:

- Sensor: HLK-LD2410B (24GHz FMCW Radar)
- Koneksi: UART (baud rate 256000)

Edge Processing:

- Parsing frame data dari UART

- Debouncing:
    - Delay 30 detik sebelum transisi Occupied → Empty (hindari false-negative)

Komunikasi:

- HTTP POST ke IoT Gateway
- Header: `Authorization: Bearer {AuthToken}`
- Wajib menggunakan **HTTP Keep-Alive** untuk efisiensi koneksi

---

3. IoT Gateway Backend Layer

Berfungsi sebagai **real-time processor (hot path)**.

Teknologi:

- ASP.NET Core Web API
- Clean Architecture

Storage Strategy:

- **Hot Path:**
    - IMemoryCache (Singleton)
    - Menyimpan:
        - State okupansi terakhir
        - Mapping DeviceId → RoomId
    - Kompleksitas akses: O(1)
- **Cold Path:**
    - SQLite via EF Core
    - Untuk:
        - Device registry
        - Binding device ↔ room

Behavior:

- Evaluasi telemetry di memory
- Jika tidak ada perubahan → **drop payload**
- Jika ada perubahan → trigger webhook ke Booking Backend

Resilience:

- HttpClientFactory + Polly:
    - Retry
    - Circuit Breaker

---

4. Meeting Room Booking Backend Layer

Merupakan **core system & SSOT**.

Teknologi:

- ASP.NET Core Web API
- Clean Architecture (Service-Repository Pattern)

Catatan:

- Tidak menggunakan CQRS / MediatR (minimize complexity)

Fungsi utama:

- Mengelola reservasi
- Menerima occupancy event
- Integrasi Google Calendar
- Menyimpan data historis (analytics)

Real-time:

- SignalR untuk update UI

Google Calendar:

- OAuth 2.0 (Delegated Access)
- Push Notification (Webhooks)

---

5. Data Flow & Communication (Pure HTTP)

Fase 1: Edge → IoT Gateway

- Device kirim telemetry kontinu
- Gateway:
    - Evaluasi di RAM
    - Jika tidak berubah → discard
    - Tidak ada write ke DB

Fase 2: IoT Gateway → Booking Backend

- Trigger hanya saat **state berubah**
- Kirim via HTTP Webhook

Fase 3: Metadata Sync

- IoT Gateway fetch room list via HTTP GET (on-demand)

---

6. Core Features & Business Logic

Auto-Release (Anti Ghost Meeting)

- Jika meeting mulai 10:00
- Hingga 10:15 tidak ada okupansi fisik
  → booking dibatalkan otomatis

Ad-Hoc Booking

- Jika ruangan kosong tapi ada okupansi ≥ 5 menit
  → auto-create booking 30 menit

---

7. Analytics & Historical Data Strategy

Untuk mendukung kebutuhan grafik dan reporting, sistem menambahkan layer penyimpanan historis tanpa mengganggu performa real-time.

7.1 Prinsip Utama

- Tidak menyimpan raw telemetry frekuensi tinggi
- Hanya menyimpan **state transition event**
- Booking Backend tetap sebagai **SSOT untuk analytics**

7.2 Storage Architecture

- Menggunakan **RDBMS (PostgreSQL / SQL Server)** di Booking Backend
- IoT Gateway tetap:
  - IMemoryCache (real-time)
  - SQLite (device only)

7.3 Data Model

Event-based storage (mandatory):

```
None
RoomOccupancyEvent
- Id
- RoomId
- IsOccupied
- ChangedAt
- SourceEventId (idempotency)
```

Aggregated storage (optional untuk performa grafik):

```
None
RoomOccupancyAggregate
- Id
- RoomId
- BucketStart
- BucketSize (Minute/Hour/Day)
- OccupiedSeconds
- EmptySeconds
- BookingSeconds
```

7.4 Tujuan (Kenapa ini penting)

Dengan pendekatan ini:

- Bisa generate grafik:
  - Utilization per ruangan
  - Peak hours
  - Ghost meeting rate
- Tetap hemat storage (tidak simpan raw telemetry)
- Query tetap cepat (pakai aggregate)

---

8. Development, CI/CD & Quality Standards

- N-Tier architecture
- DTO separation
- AutoMapper

- FluentValidation

CI/CD:

- Jenkins multi-branch pipeline

Quality Gate:

- JetBrains Qodana
- ReSharper warnings → fail build

---

9. API Contracts

9.1 Edge → IoT Gateway

None
```
POST /api/telemetry
```

JSON
```
{
  "deviceId": "string",
  "timestamp": "2026-05-04T18:50:41Z",
  "isOccupied": true,
  "stationaryTargetDistance": 1.2,
  "movingTargetDistance": 0.0
}
```

9.2 IoT Gateway → Booking Backend

None
```
POST /api/webhooks/occupancy
```

JSON
```
{
  "eventId": "guid",
```

```
  "roomId": "guid",
  "isOccupied": true,
  "changedAt": "2026-05-04T18:50:41Z"
}
```

9.3 Room Sync

None

```
GET /api/rooms/active
```

9.4 Device Binding

None

```
POST /api/devices/bind
```

Summary Perubahan Penting

Yang sebenarnya kamu tambahkan (dan ini krusial):

- Dari sebelumnya **pure real-time tanpa histori**
  → sekarang jadi:
- **Real-time + historical analytics (event-based)**

Dan ini keputusan yang tepat karena:

- Tanpa ini → sistem cuma “monitoring”
- Dengan ini → sistem jadi **decision-making tool (data-driven)**

Kalau mau next step, aku bisa bantu:

- desain query untuk grafik (misal utilization per hari)
- atau bikin contoh dashboard metrics yang “worth tracking”
  biar PRD kamu makin “product-oriented”, bukan cuma system design 👍

Product Requirements Document (PRD)

Simplified HTTP-Based Smart Meeting Room Occupancy System

---

1. Objective

Sistem ini bertujuan untuk mendeteksi okupansi fisik ruang meeting secara real-time menggunakan sensor radar, lalu menghubungkan data okupansi tersebut dengan sistem booking ruangan berbasis Google Calendar.

Sistem harus dapat:

1. Mendeteksi apakah ruangan sedang ditempati atau kosong.
2. Mengirim data occupancy dari Edge Device ke IoT Gateway.
3. Meneruskan perubahan status occupancy dari IoT Gateway ke Meeting Room Booking Backend.
4. Menggunakan occupancy untuk fitur:
    - Auto-release meeting room
    - Ad-hoc booking
    - Real-time room status
    - Historical analytics dan grafik utilization
5. Tetap sederhana secara infrastruktur:
    - Tidak menggunakan RabbitMQ/Kafka
    - Tidak menggunakan Redis
    - Menggunakan HTTP/REST sebagai komunikasi utama

---

2. High-Level Architecture

Sistem terdiri dari tiga komponen utama:

```
None
[Edge Device / Raspberry Pi]
         |
         | HTTP POST + Device Token
```

```
        v
[IoT Gateway Backend]
        |
        | HTTP POST + JWT Gateway Token
        v
[Meeting Room Booking Backend]
        |
        | Google Calendar API
        v
[Google Calendar]
```

2.1 Hardware & Edge Layer

Komponen ini berjalan di Raspberry Pi dan bertugas membaca data dari sensor radar.

Tanggung jawab utama:

- Membaca data sensor HLK-LD2410B
- Melakukan basic filtering/debouncing
- Mengirim telemetry ke IoT Gateway
- Tidak menyimpan data historis
- Tidak berkomunikasi langsung dengan Booking Backend

2.2 IoT Gateway Backend

IoT Gateway adalah layanan perantara antara hardware dan sistem booking.

Tanggung jawab utama:

- Menerima telemetry dari Edge Device
- Memvalidasi Device Token
- Menyimpan state okupansi terakhir di memory
- Melakukan deduplication
- Mendeteksi perubahan status occupancy
- Mengirim event perubahan status ke Booking Backend
- Menyimpan device registry dan room binding di SQLite
- Menyimpan pending webhook event di SQLite Outbox agar event tidak hilang saat Booking Backend down

2.3 Meeting Room Booking Backend

Meeting Room Booking Backend adalah core business system dan **Single Source of Truth (SSOT)**.

Tanggung jawab utama:

- Mengelola data ruangan
- Mengelola reservasi
- Menerima occupancy webhook dari IoT Gateway
- Menghasilkan JWT untuk IoT Gateway
- Menyimpan historical occupancy data
- Menghasilkan analytics/grafik utilization
- Mengintegrasikan Google Calendar
- Mengirim real-time update ke frontend via SignalR

---

3. Authentication & Token Strategy

Sistem menggunakan dua jenis autentikasi berbeda karena kebutuhan Edge Device dan backend service berbeda.

3.1 Edge Device → IoT Gateway

Token Type

Edge Device menggunakan **Opaque Device Token**, bukan JWT.

Alasan:

- Edge Device hanya perlu authenticate ke IoT Gateway.
- Token bisa dibuat sangat sederhana dan aman.
- Tidak perlu expiry/refresh flow yang kompleks di Raspberry Pi.
- Token bisa di-rotate dari dashboard/admin.

Format token:

```
None
dev_{base64url_random_32_bytes}
```

Contoh:

```
None
dev_L2k8sW7Fq2QzA91nR8zMplVx2aT0bcYsVjQeP3rK9dA
```

Header yang digunakan

```
None
Authorization: Bearer {DeviceToken}
```

Contoh:

```
None
Authorization: Bearer
dev_L2k8sW7Fq2QzA91nR8zMplVx2aT0bcYsVjQeP3rK9dA
```

Cara Generate Device Token

Device Token dibuat oleh **IoT Gateway** saat admin melakukan provisioning device.

Flow:

```
None
Admin → IoT Gateway: Create Device
IoT Gateway:
  1. Generate DeviceId
  2. Generate random token 32 bytes
  3. Hash token
  4. Simpan hash ke SQLite
  5. Return plaintext token sekali saja ke admin
Admin:
  1. Copy token ke Raspberry Pi config
  2. Token tidak bisa dilihat ulang setelah provisioning
```

Pseudo logic:

```
C#
var rawBytes = RandomNumberGenerator.GetBytes(32);
var token = "dev_" + Base64UrlEncode(rawBytes);
```

```
var tokenHash = SHA256(token + serverPepper);
```

Yang disimpan di database:

```
None
DeviceTokenHash
```

Yang tidak boleh disimpan:

```
None
Plaintext DeviceToken
```

3.2 IoT Gateway → Meeting Room Booking Backend

Token Type

IoT Gateway menggunakan **JWT Access Token** yang diterbitkan oleh Meeting Room Booking Backend.

Jawaban arsitekturalnya:

> Ya, JWT digunakan untuk POST data dari IoT Gateway ke Meeting Room Booking Backend. JWT tersebut harus di-generate oleh Meeting Room Booking Backend melalui machine-to-machine authentication flow.

IoT Gateway **tidak membuat JWT sendiri**.

IoT Gateway hanya menyimpan:

```
None
GatewayClientId
GatewayClientSecret
```

Lalu IoT Gateway meminta token ke Booking Backend.

JWT Flow

```
None
IoT Gateway → Booking Backend:
POST /api/auth/gateway/token

Booking Backend:
  1. Validasi clientId dan clientSecret
  2. Generate JWT
  3. Return accessToken

IoT Gateway:
  1. Cache JWT di memory
  2. Pakai JWT untuk call API Booking Backend
  3. Refresh token sebelum expired
```

JWT Usage

JWT digunakan untuk endpoint:

```
None
POST /api/webhooks/occupancy
GET  /api/rooms/active
```

Header:

```
None
Authorization: Bearer {GatewayAccessToken}
```

JWT Claims

JWT harus memiliki claims minimal berikut:

```
JSON
{
  "iss": "meeting-room-booking-backend",
```

```
  "aud": "meeting-room-booking-api",
  "sub": "gateway:{gatewayId}",
  "gatewayId": "guid",
  "clientId": "string",
  "scope": "occupancy.write rooms.read",
  "iat": 1777900000,
  "exp": 1777900900,
  "jti": "guid"
}
```

TTL token:

```
None
15 menit
```

Refresh strategy:

```
None
IoT Gateway refresh JWT 60 detik sebelum expired.
```

---

4. Provisioning Flow

4.1 Register IoT Gateway Client di Booking Backend

Sebelum IoT Gateway bisa mengirim occupancy event, gateway harus didaftarkan di Meeting Room Booking Backend.

API Contract

```
None
POST /api/admin/iot-gateways
Host: Booking Backend
```

```
Authorization: Bearer {AdminJwt}
Content-Type: application/json
```

Request:

```JSON
{
  "gatewayName": "Main Office IoT Gateway",
  "locationName": "Jakarta Office",
  "description": "Gateway for all meeting room sensors"
}
```

Response:

```JSON
{
  "gatewayId": "0a9c1c1d-85a1-4ed2-8e97-3f8e4f1b2c11",
  "clientId": "gw_main_office_001",
  "clientSecret": "gws_8aV2n9zQxKf91pLmT0sB7qWeR4yUiOpD",
  "createdAt": "2026-05-04T10:00:00Z"
}
```

Important:

```None
clientSecret hanya ditampilkan sekali.
Booking Backend hanya menyimpan hash dari clientSecret.
```

4.2 Configure IoT Gateway

IoT Gateway harus menyimpan credential berikut di environment variable atau secret manager lokal.

```
None
BOOKING_BACKEND_BASE_URL=https://booking.company.local
GATEWAY_ID=0a9c1c1d-85a1-4ed2-8e97-3f8e4f1b2c11
GATEWAY_CLIENT_ID=gw_main_office_001
GATEWAY_CLIENT_SECRET=gws_8aV2n9zQxKf91pLmT0sB7qWeR4yUiOpD
SQLITE_CONNECTION_STRING=Data Source=iot-gateway.db
```

4.3 Provision Edge Device di IoT Gateway

Admin melakukan provisioning device melalui IoT Gateway.

API Contract

```
None
POST /api/devices
Host: IoT Gateway
Authorization: Bearer {AdminJwt}
Content-Type: application/json
```

Request:

```
JSON
{
  "deviceName": "Radar Sensor Meeting Room A",
  "hardwareModel": "HLK-LD2410B",
  "serialNumber": "LD2410B-ROOM-A-001"
}
```

Response:

```
JSON
{
  "deviceId": "6e15fb67-80ec-4097-8f28-b2e4c5a4a812",
  "deviceToken": "dev_L2k8sW7Fq2QzA91nR8zMplVx2aT0bcYsVjQeP3rK9dA",
  "deviceTokenShownOnce": true,
  "createdAt": "2026-05-04T10:05:00Z"
```

```
}
```

Device Token kemudian dipasang di Raspberry Pi config:

```
None
DEVICE_ID=6e15fb67-80ec-4097-8f28-b2e4c5a4a812
DEVICE_TOKEN=dev_L2k8sW7Fq2QzA91nR8zMplVx2aT0bcYsVjQeP3rK9dA
IOT_GATEWAY_BASE_URL=https://iot-gateway.company.local
TELEMETRY_POST_INTERVAL_MS=1000
```

---

5. Room Synchronization & Device Binding Flow

5.1 IoT Gateway Request JWT dari Booking Backend

Sebelum mengambil room list, IoT Gateway meminta access token.

API Contract

```
None
POST /api/auth/gateway/token
Host: Booking Backend
Content-Type: application/json
```

Request:

```
JSON
{
  "grantType": "client_credentials",
  "clientId": "gw_main_office_001",
  "clientSecret": "gws_8aV2n9zQxKf91pLmT0sB7qWeR4yUiOpD",
```

```
  "scope": "rooms.read occupancy.write"
}
```

Response:

```JSON
{
  "tokenType": "Bearer",
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expiresIn": 900,
  "scope": "rooms.read occupancy.write"
}
```

5.2 IoT Gateway Fetch Active Rooms

IoT Gateway mengambil daftar ruangan aktif dari Booking Backend.

API Contract

```None
GET /api/rooms/active
Host: Booking Backend
Authorization: Bearer {GatewayAccessToken}
```

Response:

```JSON
[
  {
    "roomId": "ad8b76b1-7f5c-4c08-a733-69e9f8f28131",
    "roomName": "Meeting Room A",
    "floor": "12",
    "capacity": 8,
    "googleCalendarResourceId":
"company.com_1234567890@resource.calendar.google.com",
```

```
    "isActive": true
  }
]
```

Required JWT scope:

```
None
rooms.read
```

5.3 Bind Device to Room

Admin memilih device dan room dari dashboard IoT Gateway.

API Contract

```
None
POST /api/devices/{deviceId}/bind
Host: IoT Gateway
Authorization: Bearer {AdminJwt}
Content-Type: application/json
```

Request:

```
JSON
{
  "roomId": "ad8b76b1-7f5c-4c08-a733-69e9f8f28131",
  "roomName": "Meeting Room A",
  "googleCalendarResourceId":
"company.com_1234567890@resource.calendar.google.com"
}
```

Response:

```JSON
{
  "deviceId": "6e15fb67-80ec-4097-8f28-b2e4c5a4a812",
  "roomId": "ad8b76b1-7f5c-4c08-a733-69e9f8f28131",
  "bindingStatus": "Bound",
  "cacheRefreshed": true,
  "updatedAt": "2026-05-04T10:10:00Z"
}
```

Saat binding berhasil:

```None
IoT Gateway:
  1. Simpan RoomDeviceMapping ke SQLite
  2. Refresh IMemoryCache
  3. Mark device sebagai Active
```

---

6. Edge Device Runtime Flow

6.1 Edge Startup Flow

Saat Raspberry Pi menyala:

```None
1. Load config dari environment/local file:
   - DEVICE_ID
   - DEVICE_TOKEN
   - IOT_GATEWAY_BASE_URL
   - TELEMETRY_POST_INTERVAL_MS

2. Initialize UART connection ke HLK-LD2410B
3. Start reading sensor frames
4. Convert sensor data menjadi occupancy state
```

```
5. Start HTTP client dengan Keep-Alive enabled
```

6.2 Sensor Reading Logic

Sensor menghasilkan data seperti:

```
None
stationaryTargetDistance
movingTargetDistance
presenceDetected
```

Edge Device mengubah data tersebut menjadi:

```
None
isOccupied = true / false
```

Rule high-level:

```
None
Jika presenceDetected == true:
  isOccupied = true

Jika presenceDetected == false:
  jangan langsung set empty
  tunggu 30 detik
  jika selama 30 detik tetap tidak ada presence:
    isOccupied = false
```

Tujuan debouncing:

```
None
Mencegah false-negative saat orang diam, sensor drop sesaat, atau
ada noise.
```

6.3 Edge Telemetry POST

Edge mengirim telemetry secara kontinu ke IoT Gateway.

API Contract

```
None
POST /api/telemetry
Host: IoT Gateway
Authorization: Bearer {DeviceToken}
Content-Type: application/json
Connection: keep-alive
```

Request:

```
JSON
{
  "Id": "6e15fb67-80ec-4097-8f28-b2e4c5a4a812",
  "RoomId": "2b40d3dd-c53b-432a-a222-72bc449f0f47",
"Token" : "141o2ihjd0911210147oihoie",
  "timestamp": "2026-05-04T10:15:00Z",
  "isOccupied": true,
  "stationaryTargetDistance": 1.2,
  "movingTargetDistance": 0.0,
}
```

Response jika diterima dan tidak ada perubahan state:

```
JSON
{
  "accepted": true,
  "stateChanged": false,
  "message": "Telemetry accepted. No state transition detected.",
  "serverTime": "2026-05-04T10:15:00Z"
}
```

Response jika diterima dan ada perubahan state:

JSON

```
{
  "accepted": true,
  "stateChanged": true,
  "generatedEventId": "3f4d18ad-ccbb-4ea4-91b6-f0b16b2ec6a4",
  "message": "Telemetry accepted. Occupancy transition event
created.",
  "serverTime": "2026-05-04T10:15:00Z"
}
```

Error response:

None

```
401 Unauthorized
```

JSON

```
{
  "errorCode": "INVALID_DEVICE_TOKEN",
  "message": "Device token is invalid or revoked."
}
```

None

```
404 Not Found
```

JSON

```
{
  "errorCode": "DEVICE_NOT_BOUND",
  "message": "Device is not bound to any room."
}
```

---

7. IoT Gateway Telemetry Processing Flow

Saat IoT Gateway menerima telemetry dari Edge Device:

```
None
1. Validate Authorization header
2. Hash incoming DeviceToken
3. Compare hash dengan token hash di SQLite/cache
4. Validate device status:
   - Active
   - Not revoked
   - Bound to room
5. Validate timestamp:
   - Harus UTC
   - Maksimal skew ±2 menit dari server time
6. Ambil current state dari IMemoryCache
7. Bandingkan incoming isOccupied dengan previous state
8. Jika sama:
   - Return 200 OK
   - Tidak tulis DB
   - Tidak kirim webhook
9. Jika berubah:
   - Generate eventId
   - Update IMemoryCache
   - Simpan event ke SQLite Outbox
   - Dispatch webhook ke Booking Backend
```

7.1 IoT Gateway IMemoryCache Keys

```
None
DeviceState:{deviceId}
RoomState:{roomId}
DeviceRoomMapping:{deviceId}
```

Contoh cached value:

```
JSON
{
```

```
  "deviceId": "6e15fb67-80ec-4097-8f28-b2e4c5a4a812",
  "roomId": "ad8b76b1-7f5c-4c08-a733-69e9f8f28131",
  "isOccupied": true,
  "lastChangedAt": "2026-05-04T10:15:00Z",
  "lastReadingAt": "2026-05-04T10:15:00Z"
}
```

7.2 SQLite Outbox di IoT Gateway

Walaupun tidak menggunakan RabbitMQ/Kafka, IoT Gateway tetap membutuhkan persistent outbox agar event tidak hilang kalau Booking Backend sedang down.

Table: GatewayWebhookOutbox

```
None
GatewayWebhookOutbox
- Id (GUID)
- EventId (GUID, unique)
- DeviceId (GUID)
- RoomId (GUID)
- IsOccupied (boolean)
- ChangedAtUtc (datetime)
- PayloadJson (text)
- Status (Pending, Delivered, Failed)
- AttemptCount (int)
- NextAttemptAtUtc (datetime)
- LastError (text, nullable)
- CreatedAtUtc (datetime)
- DeliveredAtUtc (datetime, nullable)
```

Outbox Behavior

```
None
Saat state berubah:
  1. Insert event ke GatewayWebhookOutbox dengan status Pending
```

```
  2. Coba kirim langsung ke Booking Backend
  3. Jika sukses:
     - Mark Delivered
  4. Jika gagal:
     - Tetap Pending
     - Background worker retry berdasarkan NextAttemptAtUtc
```

Retry policy:

```
None
Immediate retry: 3 kali via Polly
Background retry:
  - 1 menit
  - 5 menit
  - 15 menit
  - 30 menit
  - Maksimum retry tetap berjalan sampai delivered atau manual
intervention
```

---

8. IoT Gateway → Booking Backend Occupancy Webhook

Webhook hanya dikirim saat ada perubahan occupancy definitif.

8.1 API Contract

```
None
POST /api/webhooks/occupancy
Host: Booking Backend
Authorization: Bearer {GatewayAccessToken}
Idempotency-Key: {eventId}
```

```
Content-Type: application/json
```

Request:

```JSON
{
  "eventId": "3f4d18ad-ccbb-4ea4-91b6-f0b16b2ec6a4",
  "gatewayId": "0a9c1c1d-85a1-4ed2-8e97-3f8e4f1b2c11",
  "deviceId": "6e15fb67-80ec-4097-8f28-b2e4c5a4a812",
  "roomId": "ad8b76b1-7f5c-4c08-a733-69e9f8f28131",
  "eventType": "Transition",
  "isOccupied": true,
  "changedAt": "2026-05-04T10:15:00Z",
  "observedAt": "2026-05-04T10:15:00Z"
}
```

eventType

Allowed values:

```None
Transition
Snapshot
```

Transition

Digunakan saat status berubah:

```None
Empty → Occupied
Occupied → Empty
```

Transition event disimpan ke historical table.

Snapshot

Digunakan saat IoT Gateway baru restart atau device baru binding.

Snapshot hanya memperbarui current state jika lebih baru, tetapi tidak wajib dihitung sebagai transition analytics.

8.2 Required JWT Scope

```
None
occupancy.write
```

Jika JWT tidak memiliki scope tersebut:

```
None
403 Forbidden
```

Response:

```
JSON
{
  "errorCode": "INSUFFICIENT_SCOPE",
  "message": "Token does not contain required scope:
occupancy.write"
}
```

8.3 Success Response

```
JSON
{
  "accepted": true,
  "eventId": "3f4d18ad-ccbb-4ea4-91b6-f0b16b2ec6a4",
  "duplicate": false,
  "currentRoomStatus": {
    "roomId": "ad8b76b1-7f5c-4c08-a733-69e9f8f28131",
    "isOccupied": true,
```

```
    "changedAt": "2026-05-04T10:15:00Z"
  }
}
```

Duplicate event harus tetap return `200 OK` agar IoT Gateway bisa mark event sebagai delivered.

Response duplicate:

JSON
```
{
  "accepted": true,
  "eventId": "3f4d18ad-ccbb-4ea4-91b6-f0b16b2ec6a4",
  "duplicate": true,
  "message": "Event already processed."
}
```

---

9. Booking Backend Occupancy Processing Flow

Saat Booking Backend menerima occupancy webhook:

None
```
1. Validate JWT:
   - Signature valid
   - Issuer valid
   - Audience valid
   - Token not expired
   - Scope contains occupancy.write

2. Validate gateway:
   - gatewayId exists
   - gateway active
```

```
   - client not revoked

3. Validate payload:
   - eventId valid GUID
   - roomId exists
   - changedAt UTC
   - eventType valid

4. Idempotency check:
   - Jika SourceEventId/eventId sudah pernah diproses:
     return 200 OK duplicate=true

5. Start DB transaction

6. Upsert RoomCurrentOccupancy

7. Jika eventType == Transition:
   - Insert ke RoomOccupancyEvent
   - Update duration event sebelumnya jika ada
   - Trigger analytics aggregate update

8. Jalankan business logic:
   - Auto-release
   - Ad-hoc booking

9. Publish SignalR event ke frontend

10. Commit transaction

11. Return 200 OK
```

---

10. Historical Data Strategy

Historical data disimpan di Meeting Room Booking Backend, bukan di IoT Gateway.

Prinsip utamanya:

```
Raw telemetry frekuensi tinggi tidak disimpan.
Yang disimpan hanya definitive occupancy transition.
```

Alasan:

- Hemat storage
- Query analytics lebih ringan
- Tidak membebani hot path IoT Gateway
- Tetap cukup untuk grafik utilization

10.1 Booking Backend RDBMS

Booking Backend menggunakan RDBMS utama.

Recommended default:

```
PostgreSQL
```

Alternatif:

```
SQL Server
```

SQLite tidak direkomendasikan untuk Booking Backend analytics karena kebutuhan query historis, agregasi, dan concurrent dashboard access.

10.2 Table: RoomCurrentOccupancy

Menyimpan current state per room.

```
None
RoomCurrentOccupancy
- RoomId (GUID, PK)
- IsOccupied (boolean)
- LastChangedAtUtc (datetime)
- LastObservedAtUtc (datetime)
- LastEventId (GUID)
- GatewayId (GUID)
- DeviceId (GUID)
- UpdatedAtUtc (datetime)
```

Tujuan:

```
None
Digunakan untuk real-time room status, dashboard, dan decision
logic.
```

10.3 Table: RoomOccupancyEvent

Menyimpan event perubahan status.

```
None
RoomOccupancyEvent
- Id (GUID, PK)
- SourceEventId (GUID, unique)
- GatewayId (GUID)
- DeviceId (GUID)
- RoomId (GUID)
- IsOccupied (boolean)
- ChangedAtUtc (datetime)
- ObservedAtUtc (datetime)
- ReceivedAtUtc (datetime)
- DurationSeconds (int, nullable)
- PreviousEventId (GUID, nullable)
- CreatedAtUtc (datetime)
```

Catatan DurationSeconds

Saat event baru masuk, sistem dapat mengisi `DurationSeconds` pada event sebelumnya.

Contoh:

```
None
10:00 Occupied
10:45 Empty
```

Maka event `10:00 Occupied` punya:

```
None
DurationSeconds = 2700
```

Ini memudahkan query grafik karena durasi occupancy sudah tersedia.

10.4 Table: RoomOccupancyAggregate

Digunakan untuk mempercepat grafik.

```
None
RoomOccupancyAggregate
- Id (GUID, PK)
- RoomId (GUID)
- BucketStartUtc (datetime)
- BucketEndUtc (datetime)
- BucketSize (Minute, Hour, Day)
- OccupiedSeconds (int)
- EmptySeconds (int)
- BookingSeconds (int)
- UtilizationPercent (decimal)
- UpdatedAtUtc (datetime)
```

Unique constraint:

```
None
RoomId + BucketStartUtc + BucketSize
```

10.5 Aggregate Calculation

Aggregate dapat dihitung dengan dua pendekatan:

MVP Approach

Hitung aggregate dengan background job di Booking Backend.

```
None
Every 5 minutes:
  1. Ambil occupancy events terbaru
  2. Recalculate bucket untuk 24 jam terakhir
  3. Upsert ke RoomOccupancyAggregate
```

More Real-Time Approach

Saat event baru masuk:

```
None
1. Ambil previous occupancy event
2. Hitung interval antara previous event dan current event
3. Pecah interval ke bucket Minute/Hour/Day
4. Update OccupiedSeconds / EmptySeconds
5. Upsert RoomOccupancyAggregate
```

Untuk MVP, gunakan background job agar implementasi lebih sederhana.

---

11. Analytics API Contracts

11.1 Get Room Occupancy Analytics

Digunakan frontend untuk menampilkan grafik occupancy per room.

API Contract

```
None
GET
/api/analytics/rooms/{roomId}/occupancy?from=2026-05-04T00:00:00Z&to=2026-05-04T23:59:59Z&bucket=Hour
Host: Booking Backend
Authorization: Bearer {UserJwt}
```

Query params:

```
None
from   = UTC datetime
to     = UTC datetime
bucket = Minute | Hour | Day
```

Response:

```JSON
{
  "roomId": "ad8b76b1-7f5c-4c08-a733-69e9f8f28131",
  "roomName": "Meeting Room A",
  "from": "2026-05-04T00:00:00Z",
  "to": "2026-05-04T23:59:59Z",
  "bucket": "Hour",
  "data": [
    {
      "bucketStart": "2026-05-04T09:00:00Z",
      "bucketEnd": "2026-05-04T10:00:00Z",
      "occupiedSeconds": 1800,
      "emptySeconds": 1800,
      "bookingSeconds": 3600,
      "utilizationPercent": 50.0
    },
```

```
    {
      "bucketStart": "2026-05-04T10:00:00Z",
      "bucketEnd": "2026-05-04T11:00:00Z",
      "occupiedSeconds": 2700,
      "emptySeconds": 900,
      "bookingSeconds": 3600,
      "utilizationPercent": 75.0
    }
  ]
}
```

11.2 Get Occupancy Summary

Digunakan untuk dashboard summary.

API Contract

```
None
GET
/api/analytics/occupancy-summary?from=2026-05-01T00:00:00Z&to=2026-
05-04T23:59:59Z
Host: Booking Backend
Authorization: Bearer {UserJwt}
```

Response:

```
JSON
{
  "from": "2026-05-01T00:00:00Z",
  "to": "2026-05-04T23:59:59Z",
  "totalRooms": 12,
  "averageUtilizationPercent": 62.4,
  "mostUsedRooms": [
    {
      "roomId": "ad8b76b1-7f5c-4c08-a733-69e9f8f28131",
```

```
      "roomName": "Meeting Room A",
      "utilizationPercent": 82.1
    }
  ],
  "leastUsedRooms": [
    {
      "roomId": "8a2e46a5-124f-45d8-b0a8-9989f1ff7e12",
      "roomName": "Meeting Room C",
      "utilizationPercent": 21.7
    }
  ]
}
```

11.3 Get Ghost Meeting Report

API Contract

None

```
GET
/api/analytics/ghost-meetings?from=2026-05-01T00:00:00Z&to=2026-05-
04T23:59:59Z
Host: Booking Backend
Authorization: Bearer {UserJwt}
```

Response:

JSON

```
{
  "from": "2026-05-01T00:00:00Z",
  "to": "2026-05-04T23:59:59Z",
  "totalGhostMeetings": 8,
  "autoReleasedMeetings": 6,
  "estimatedReleasedMinutes": 270,
  "items": [
    {
```

```
      "roomId": "ad8b76b1-7f5c-4c08-a733-69e9f8f28131",
      "roomName": "Meeting Room A",
      "bookingId": "9fd01ec7-8cb9-4c99-a601-5c05d44c3f55",
      "scheduledStart": "2026-05-04T10:00:00Z",
      "scheduledEnd": "2026-05-04T11:00:00Z",
      "autoReleasedAt": "2026-05-04T10:15:00Z",
      "reason": "No physical occupancy detected within grace
period."
    }
  ]
}
```

---

12. Core Business Logic

12.1 Auto-Release Logic

Rule:

```
None
Jika meeting dimulai pada T,
dan sampai T + 15 menit RoomCurrentOccupancy.IsOccupied == false,
maka sistem membatalkan booking di Google Calendar.
```

Flow:

```
None
1. Booking dibuat/synced dari Google Calendar
2. Saat booking start, Booking Backend schedule check untuk T + 15
menit
3. Pada T + 15 menit:
   - Load RoomCurrentOccupancy
   - Jika IsOccupied == false:
```

```
    - Cancel Google Calendar event
    - Mark booking sebagai AutoReleased
    - Simpan audit record
    - Emit SignalR update
```

Table audit:

```
None
GhostMeetingAudit
- Id (GUID)
- BookingId (GUID)
- RoomId (GUID)
- ScheduledStartUtc (datetime)
- ScheduledEndUtc (datetime)
- AutoReleasedAtUtc (datetime)
- Reason (text)
- CreatedAtUtc (datetime)
```

12.2 Ad-Hoc Booking Logic

Rule:

```
None
Jika room status Available,
dan RoomCurrentOccupancy.IsOccupied == true selama minimal 5 menit
berturut-turut,
maka sistem membuat booking otomatis selama 30 menit.
```

Flow:

```
None
1. Occupancy event masuk: Empty → Occupied
2. Booking Backend cek current booking untuk room tersebut
3. Jika tidak ada active booking:
```

```
   - Start ad-hoc candidate timer
4. Setelah 5 menit:
   - Cek ulang RoomCurrentOccupancy
   - Cek ulang room availability
5. Jika masih occupied dan available:
   - Create Google Calendar event
   - Create local booking record
   - Emit SignalR update
```

Ad-hoc booking default duration:

```
None
30 menit
```

---

13. Real-Time Client Update

Booking Backend menggunakan SignalR.

13.1 SignalR Hub

```
None
/hubs/rooms
```

Event yang dikirim ke client:

```
None
RoomOccupancyChanged
BookingAutoReleased
AdHocBookingCreated
RoomAnalyticsUpdated
```

Contoh payload `RoomOccupancyChanged`:

JSON

```
{
  "roomId": "ad8b76b1-7f5c-4c08-a733-69e9f8f28131",
  "roomName": "Meeting Room A",
  "isOccupied": true,
  "changedAt": "2026-05-04T10:15:00Z"
}
```

---

14. Database Design Summary

14.1 IoT Gateway SQLite Tables

None

```
Device
- Id (GUID)
- DeviceName
- HardwareModel
- SerialNumber
- IsActive
- CreatedAtUtc
- UpdatedAtUtc

DeviceCredential
- Id (GUID)
- DeviceId (GUID)
- TokenHash
- IsActive
- CreatedAtUtc
- RevokedAtUtc

RoomDeviceMapping
- Id (GUID)
- DeviceId (GUID)
```

```
- RoomId (GUID)
- RoomName
- GoogleCalendarResourceId
- IsActive
- CreatedAtUtc
- UpdatedAtUtc

GatewayWebhookOutbox
- Id (GUID)
- EventId (GUID, unique)
- DeviceId (GUID)
- RoomId (GUID)
- IsOccupied
- ChangedAtUtc
- PayloadJson
- Status
- AttemptCount
- NextAttemptAtUtc
- LastError
- CreatedAtUtc
- DeliveredAtUtc
```

14.2 Booking Backend RDBMS Tables

```
None
Room
- Id (GUID)
- Name
- Floor
- Capacity
- GoogleCalendarResourceId
- IsActive
- CreatedAtUtc
- UpdatedAtUtc
```

```
IoTGatewayClient
- Id (GUID)
- GatewayName
- ClientId
- ClientSecretHash
- AllowedScopes
- IsActive
- CreatedAtUtc
- RevokedAtUtc

RoomCurrentOccupancy
- RoomId (GUID, PK)
- IsOccupied
- LastChangedAtUtc
- LastObservedAtUtc
- LastEventId
- GatewayId
- DeviceId
- UpdatedAtUtc

RoomOccupancyEvent
- Id (GUID)
- SourceEventId (GUID, unique)
- GatewayId
- DeviceId
- RoomId
- IsOccupied
- ChangedAtUtc
- ObservedAtUtc
- ReceivedAtUtc
- DurationSeconds
- PreviousEventId
- CreatedAtUtc

RoomOccupancyAggregate
- Id (GUID)
```

```
- RoomId
- BucketStartUtc
- BucketEndUtc
- BucketSize
- OccupiedSeconds
- EmptySeconds
- BookingSeconds
- UtilizationPercent
- UpdatedAtUtc

Booking
- Id (GUID)
- RoomId
- GoogleCalendarEventId
- StartUtc
- EndUtc
- Status
- CreatedAtUtc
- UpdatedAtUtc

GhostMeetingAudit
- Id (GUID)
- BookingId
- RoomId
- ScheduledStartUtc
- ScheduledEndUtc
- AutoReleasedAtUtc
- Reason
- CreatedAtUtc
```

---

15. Required Indexes & Constraints

15.1 IoT Gateway

```
None
DeviceCredential.TokenHash unique
RoomDeviceMapping.DeviceId unique where IsActive = true
GatewayWebhookOutbox.EventId unique
GatewayWebhookOutbox.Status + NextAttemptAtUtc index
```

15.2 Booking Backend

```
None
IoTGatewayClient.ClientId unique
Room.GoogleCalendarResourceId unique
RoomCurrentOccupancy.RoomId unique
RoomOccupancyEvent.SourceEventId unique
RoomOccupancyEvent.RoomId + ChangedAtUtc index
RoomOccupancyAggregate.RoomId + BucketStartUtc + BucketSize unique
Booking.RoomId + StartUtc + EndUtc index
```

---

16. Error Handling Rules

16.1 Edge → IoT Gateway

```
None
400 Bad Request
- Invalid payload
- Invalid timestamp format

401 Unauthorized
- Missing token
- Invalid token
- Revoked token

404 Not Found
```

```
- Device does not exist
- Device not bound to room

409 Conflict
- Device inactive
```

16.2 IoT Gateway → Booking Backend

```
None
400 Bad Request
- Invalid payload
- Invalid eventType
- changedAt not UTC

401 Unauthorized
- Missing JWT
- Invalid JWT
- Expired JWT

403 Forbidden
- Missing required scope

404 Not Found
- roomId does not exist

200 OK duplicate=true
- eventId already processed
```

Important:

```
None
Duplicate webhook bukan error.
Duplicate webhook harus return 200 OK supaya outbox bisa mark
Delivered.
```

---

17. End-to-End Runtime Flow

17.1 Initial Setup Flow

```
None
1. Admin register IoT Gateway di Booking Backend.
2. Booking Backend generate:
   - gatewayId
   - clientId
   - clientSecret
3. Admin configure clientId/clientSecret di IoT Gateway.
4. Admin create device di IoT Gateway.
5. IoT Gateway generate:
   - deviceId
   - deviceToken
6. Admin configure deviceId/deviceToken di Raspberry Pi.
7. IoT Gateway request JWT ke Booking Backend.
8. IoT Gateway fetch active rooms dari Booking Backend.
9. Admin bind device ke room.
10. Device siap mengirim telemetry.
```

17.2 Normal Telemetry Flow

```
None
1. Raspberry Pi membaca sensor.
2. Raspberry Pi menentukan isOccupied.
3. Raspberry Pi POST /api/telemetry ke IoT Gateway menggunakan
Device Token.
4. IoT Gateway validasi token.
5. IoT Gateway cari RoomDeviceMapping.
6. IoT Gateway bandingkan incoming state dengan cached state.
7. Jika tidak berubah:
   - return 200 OK
   - selesai
```

```
8. Jika berubah:
   - generate eventId
   - update IMemoryCache
   - insert event ke GatewayWebhookOutbox
   - request/reuse JWT dari Booking Backend
   - POST /api/webhooks/occupancy ke Booking Backend
9. Booking Backend validasi JWT.
10. Booking Backend check idempotency.
11. Booking Backend update RoomCurrentOccupancy.
12. Booking Backend insert RoomOccupancyEvent.
13. Booking Backend trigger business logic.
14. Booking Backend emit SignalR update.
15. Booking Backend update/recalculate aggregate.
16. Booking Backend return 200 OK.
17. IoT Gateway mark outbox event sebagai Delivered.
```

17.3 Booking Backend Down Flow

None

```
1. Edge tetap POST telemetry ke IoT Gateway.
2. IoT Gateway tetap memproses state change di memory.
3. Jika ada perubahan state:
   - event disimpan ke SQLite Outbox
4. IoT Gateway mencoba POST ke Booking Backend.
5. Jika gagal:
   - event tetap Pending
   - AttemptCount bertambah
   - NextAttemptAtUtc diset
6. Background worker retry sampai berhasil.
7. Saat Booking Backend kembali online:
   - pending event dikirim ulang
   - Booking Backend process idempotently
   - IoT Gateway mark Delivered
```

---

18. Security Requirements

1. Semua komunikasi harus menggunakan HTTPS.
2. Device Token tidak boleh dilog.
3. Gateway Client Secret tidak boleh dilog.
4. Plaintext token hanya boleh ditampilkan satu kali saat provisioning.
5. Token hash harus dibandingkan menggunakan constant-time comparison.
6. JWT harus short-lived.
7. JWT harus memiliki issuer dan audience validation.
8. Endpoint Booking Backend harus validasi scope.
9. Device Token harus bisa di-rotate.
10. Gateway Client Secret harus bisa di-rotate.
11. Revoked token tidak boleh diterima lagi.
12. Semua timestamp harus UTC.

---

19. Token Rotation API

19.1 Rotate Device Token

```
None
POST /api/devices/{deviceId}/tokens/rotate
Host: IoT Gateway
Authorization: Bearer {AdminJwt}
```

Response:

```
JSON
{
  "deviceId": "6e15fb67-80ec-4097-8f28-b2e4c5a4a812",
  "newDeviceToken": "dev_9kXaL0fQpT8mN2zWcY7eB3rS6vUiD1oP",
  "previousTokenRevoked": true,
  "rotatedAt": "2026-05-04T11:00:00Z"
}
```

19.2 Rotate Gateway Client Secret

```
None
POST /api/admin/iot-gateways/{gatewayId}/client-secret/rotate
Host: Booking Backend
Authorization: Bearer {AdminJwt}
```

Response:

```
JSON
{
  "gatewayId": "0a9c1c1d-85a1-4ed2-8e97-3f8e4f1b2c11",
  "clientId": "gw_main_office_001",
  "newClientSecret": "gws_q1w2e3r4t5y6u7i8o9p0",
  "previousSecretRevoked": true,
  "rotatedAt": "2026-05-04T11:05:00Z"
}
```

---

20. Developer Implementation Checklist

Edge Device

Developer Edge harus implement:

```
None
[ ] UART reader untuk HLK-LD2410B
[ ] Parser sensor frame
[ ] Occupancy detection
[ ] 30-second debounce untuk Occupied → Empty
[ ] HTTP client dengan Keep-Alive
[ ] POST /api/telemetry
[ ] Authorization Bearer DeviceToken
[ ] Retry ringan saat network error
[ ] Config via environment/local file
```

```
[ ] Semua timestamp UTC
```

IoT Gateway

Developer IoT Gateway harus implement:

```
None
[ ] Device provisioning API
[ ] Device token generation
[ ] Device token hashing
[ ] Device token validation
[ ] Device binding API
[ ] Active rooms sync dari Booking Backend
[ ] JWT client_credentials flow ke Booking Backend
[ ] JWT cache + refresh before expiry
[ ] Telemetry ingestion endpoint
[ ] IMemoryCache state management
[ ] State transition detection
[ ] SQLite GatewayWebhookOutbox
[ ] Polly retry
[ ] Background outbox dispatcher
[ ] Webhook dispatch ke Booking Backend
[ ] Token rotation API
```

Meeting Room Booking Backend

Developer Booking Backend harus implement:

```
None
[ ] IoT Gateway registration API
[ ] Gateway client secret generation
[ ] Gateway token endpoint
[ ] JWT generation
```

```
[ ] JWT validation middleware
[ ] Scope-based authorization
[ ] Active room API
[ ] Occupancy webhook endpoint
[ ] Idempotency check by eventId
[ ] RoomCurrentOccupancy table
[ ] RoomOccupancyEvent table
[ ] RoomOccupancyAggregate table
[ ] Aggregate calculation job
[ ] Analytics API
[ ] Auto-release logic
[ ] Ad-hoc booking logic
[ ] Google Calendar integration
[ ] SignalR room updates
[ ] Gateway client secret rotation
```

---

21. Acceptance Criteria

Sistem dianggap siap MVP jika:

```
None
1. Edge Device bisa mengirim telemetry valid ke IoT Gateway.
2. IoT Gateway menolak telemetry dengan token invalid.
3. IoT Gateway tidak mengirim webhook jika occupancy state tidak
berubah.
4. IoT Gateway mengirim webhook jika occupancy berubah.
5. IoT Gateway menyimpan pending webhook ke SQLite Outbox.
6. Booking Backend hanya menerima webhook dengan JWT valid.
7. JWT expired ditolak oleh Booking Backend.
8. IoT Gateway otomatis refresh JWT.
9. Duplicate eventId tidak membuat duplicate RoomOccupancyEvent.
10. RoomCurrentOccupancy selalu mencerminkan state terbaru.
11. RoomOccupancyEvent menyimpan historical transition.
12. Analytics endpoint bisa menghasilkan grafik utilization.
```

```
13. Auto-release berjalan saat meeting tidak ditempati.
14. Ad-hoc booking berjalan saat ruangan available tapi ditempati.
15. SignalR mengirim update real-time ke frontend.
```

---

22. Final Design Decision

Keputusan final untuk token:

```
None
Edge Device → IoT Gateway:
  Gunakan Opaque Device Token.
  Token dibuat oleh IoT Gateway.
  Header: Authorization: Bearer {DeviceToken}

IoT Gateway → Booking Backend:
  Gunakan JWT Access Token.
  JWT dibuat oleh Meeting Room Booking Backend.
  IoT Gateway mendapatkan JWT via /api/auth/gateway/token.
  Header: Authorization: Bearer {GatewayAccessToken}

Admin/User → Backend:
  Gunakan User/Admin JWT sesuai sistem auth aplikasi.
```

Keputusan final untuk historical data:

```
None
IoT Gateway:
  Tidak menyimpan historical analytics.
  Hanya menyimpan device registry, room binding, current memory
state, dan webhook outbox.

Booking Backend:
  Menjadi SSOT untuk current occupancy, historical occupancy event,
aggregate analytics, booking, dan Google Calendar state.
```

Dengan flow ini, developer bisa langsung membagi pekerjaan menjadi tiga track:

```
None
Track 1: Edge Device telemetry sender
Track 2: IoT Gateway ingestion + token + outbox
Track 3: Booking Backend JWT + occupancy + analytics + Google
Calendar
```

Rekomendasi Struktur OpenProject

Work Package Types

Gunakan tipe berikut:

```
None
Epic
User Story
Task
Bug
Spike
Technical Debt
```

Untuk backlog ini:

```
None
Epic       = major capability / module besar
User Story = kebutuhan fungsional atau teknis yang bisa dikerjakan
developer
Task       = pecahan implementasi teknis dari user story
Bug        = defect saat QA/UAT
Spike      = riset teknis terbatas
```

OpenProject juga mendukung konfigurasi tipe work package dan custom field, jadi kamu bisa menambahkan field seperti Component, API Endpoint, Scope, Story Points, dan Team Owner. (OpenProject.org)

Rekomendasi Status Workflow

```
None
New
Ready for Refinement
```

```
Ready for Development
In Progress
Code Review
Ready for QA
QA In Progress
Done
Blocked
Rejected / Won't Do
```

Rekomendasi Custom Fields

None

```
Component:
- Cross Cutting
- Edge Device
- IoT Gateway
- Booking Backend
- Frontend Dashboard
- Google Calendar
- DevOps
- QA

Priority:
- P0 Critical
- P1 High
- P2 Medium
- P3 Low

Release / Version:
- MVP-0 Foundation
- MVP-1 Edge + IoT Gateway
- MVP-2 Booking Backend Occupancy
- MVP-3 Calendar + Business Logic
- MVP-4 Analytics + Dashboard
- MVP-5 Hardening + Release
```

```
Story Points:
- 1, 2, 3, 5, 8, 13

Risk Level:
- Low
- Medium
- High

API Endpoint:
- Text field

Data Store:
- SQLite
- PostgreSQL / SQL Server
- IMemoryCache
- None
```

Karena OpenProject otomatis memberi ID unik untuk setiap work package, penamaan seperti `EPIC-01` atau `US-01.01` sebaiknya dimasukkan ke **Subject** atau custom field, bukan menggantikan ID bawaan OpenProject. (OpenProject.org)

Epic Summary

Epic ID	Epic Subject	Component	Priority	Release
EPIC-01	Architecture, Repository, and Engineering Foundation	Cross-cutting	P0	MVP-0
EPIC-02	Edge Device Sensor Runtime	Edge Device	P0	MVP-1
EPIC-03	IoT Gateway Device Provisioning and Device Token Security	IoT Gateway	P0	MVP-1

Epic ID	Epic Subject	Component	Priority	Release
EPIC-01	Architecture, Repository, and Engineering Foundation	Cross-cutting	P0	MVP-0
EPIC-02	Edge Device Sensor Runtime	Edge Device	P0	MVP-1
EPIC-04	IoT Gateway Room Sync and Device Binding	IoT Gateway	P0	MVP-1
EPIC-05	IoT Gateway Telemetry Ingestion and State Management	IoT Gateway	P0	MVP-1
EPIC-06	IoT Gateway Webhook Outbox, Retry, and JWT Client	IoT Gateway	P0	MVP-2
EPIC-07	Booking Backend Gateway Registration and JWT Auth	Booking Backend	P0	MVP-2
EPIC-08	Booking Backend Room and Booking Domain Foundation	Booking Backend	P0	MVP-2
EPIC-09	Booking Backend Occupancy Webhook and Current State	Booking Backend	P0	MVP-2
EPIC-10	Historical Occupancy Data and Analytics Aggregation	Booking Backend	P0	MVP-4
EPIC-11	Analytics API and Reporting	Booking Backend	P1	MVP-4
EPIC-12	Google Calendar Integration	Booking Backend	P0	MVP-3
EPIC-13	Auto-Release and Ad-Hoc Booking Logic	Booking Backend	P0	MVP-3
EPIC-14	Real-Time Updates and Frontend Dashboard	Frontend + Backend	P1	MVP-4
EPIC-15	Admin Operations, Rotation, and Management UI	Full-stack	P1	MVP-4
EPIC-16	Observability, Security Hardening, QA, and Deployment	Cross-cutting	P0	MVP-5

EPIC-01 — Architecture, Repository, and Engineering Foundation

Type: Epic
Priority: P0
Component: Cross-cutting
Release: MVP-0 Foundation

Goal

Menyiapkan fondasi project agar semua developer bisa bekerja dengan struktur yang konsisten: Clean Architecture, API convention, error handling, environment config, CI/CD, dan quality gate.

US-01.01 — Setup solution structure untuk semua service

As a backend developer,
I want struktur solution `.NET` dibuat sesuai Clean Architecture,
So that setiap service memiliki pemisahan responsibility yang jelas.

Acceptance Criteria:

```
None
Given repository baru dibuat
When developer membuka solution
Then tersedia struktur minimal:
- Api
- Application
- Domain
- Infrastructure
- Tests
```

```
None
Given service IoT Gateway dan Booking Backend berbeda
When solution dibuat
Then masing-masing service memiliki project structure terpisah.
```

```
None
Given developer menambahkan business logic
When code ditulis
Then logic tidak diletakkan langsung di Controller.
```

US-01.02 — Define API response and error response standard

As a backend developer,
I want standar response API dibuat,
So that semua endpoint memiliki format sukses dan error yang konsisten.

Acceptance Criteria:

```
None
Given API mengembalikan response sukses
Then response memiliki format JSON yang predictable.
```

```
None
Given API mengembalikan error
Then response memiliki:
- errorCode
- message
- details optional
- traceId
```

```
None
Given validasi payload gagal
Then API return 400 Bad Request dengan errorCode yang spesifik.
```

US-01.03 — Define global UTC datetime standard

As a developer,
I want semua timestamp menggunakan UTC,
So that event occupancy, booking, dan analytics tidak ambigu.

Acceptance Criteria:

```
None
Given payload API mengandung datetime
Then datetime wajib dalam format ISO-8601 UTC.
```

```
None
Given payload menggunakan local timezone tanpa Z
Then API menolak dengan 400 Bad Request.
```

```
None
Given data disimpan ke database
Then semua field datetime disimpan sebagai UTC.
```

US-01.04 — Setup OpenAPI/Swagger documentation

As a developer,
I want semua API terdokumentasi lewat Swagger/OpenAPI,
So that frontend, QA, dan integrator bisa mencoba endpoint dengan mudah.

Acceptance Criteria:

```
None
Given service berjalan di local
When developer membuka /swagger
Then semua endpoint publik terlihat.
```

```
None
Given endpoint membutuhkan Authorization
Then Swagger menampilkan Bearer Auth configuration.
```

```
None
Given API memiliki request body
Then schema request dan response tersedia.
```

US-01.05 — Setup environment configuration standard

As a DevOps engineer,
I want semua service memakai environment variable yang jelas,
So that deployment antar environment tidak butuh perubahan code.

Acceptance Criteria:

```
None
Given service dijalankan di local/dev/prod
Then konfigurasi dibaca dari environment variable.
```

```
None
Given secret tidak tersedia
Then service gagal start dengan error yang jelas.
```

```
None
Given connection string berubah
Then service bisa menggunakan value baru tanpa rebuild.
```

US-01.06 — Setup CI pipeline baseline

As a engineering lead,
I want Jenkins pipeline berjalan untuk build dan test,
So that perubahan code bisa divalidasi otomatis.

Acceptance Criteria:

```
None
Given developer push branch
```

```
Then Jenkins menjalankan restore, build, test.
```

```
None
Given unit test gagal
Then pipeline gagal.
```

```
None
Given build sukses
Then artifact dapat dibuat.
```

US-01.07 — Setup Qodana / static analysis quality gate

As a engineering lead,
I want static analysis dijalankan di pipeline,
So that code quality issue bisa ditemukan sebelum merge.

Acceptance Criteria:

```
None
Given pipeline berjalan
Then Qodana / static analysis ikut dieksekusi.
```

```
None
Given ada architectural warning critical
Then pipeline gagal.
```

```
None
Given report selesai
Then hasil static analysis bisa dilihat oleh developer.
```

EPIC-02 — Edge Device Sensor Runtime

Type: Epic
Priority: P0
Component: Edge Device
Release: MVP-1 Edge + IoT Gateway

Goal

Membuat runtime di Raspberry Pi yang membaca data HLK-LD2410B, mengubah target/presence state menjadi `isOccupied`, melakukan debounce, lalu mengirim telemetry ke IoT Gateway. Sensor HLK-LD2410B menyediakan target state melalui OUT pin dan juga data hasil deteksi via serial, termasuk target status dan distance auxiliary information. (d.hlktech.net)

US-02.01 — Load Edge Device configuration

As an edge developer,
I want Raspberry Pi membaca konfigurasi dari environment/local config,
So that device bisa dikonfigurasi tanpa mengubah source code.

Acceptance Criteria:

```
None
Given Raspberry Pi runtime start
Then runtime membaca:
- DEVICE_ID
- DEVICE_TOKEN
- IOT_GATEWAY_BASE_URL
- TELEMETRY_POST_INTERVAL_MS
- UART_PORT
- UART_BAUD_RATE
```

```
None
Given konfigurasi wajib kosong
Then runtime gagal start dengan pesan error yang jelas.
```

```
None
Given konfigurasi valid
Then runtime lanjut initialize UART dan HTTP client.
```

US-02.02 — Initialize UART connection to HLK-LD2410B

As an edge developer,
I want runtime membuka koneksi UART ke sensor,
So that data target state bisa dibaca dari HLK-LD2410B.

Acceptance Criteria:

```
None
Given UART_PORT dan UART_BAUD_RATE valid
When runtime start
Then koneksi serial terbuka.
```

```
None
Given UART tidak tersedia
Then runtime mencatat error dan melakukan retry.
```

```
None
Given sensor mengirim frame
Then runtime dapat membaca byte stream tanpa crash.
```

US-02.03 — Parse HLK-LD2410B radar frame

As an edge developer,
I want parser membaca frame serial HLK-LD2410B,
So that target state dan distance dapat digunakan oleh aplikasi.

Acceptance Criteria:

```
None
Given frame serial valid diterima
Then parser menghasilkan object:
- targetState
- movingTargetDistance
- stationaryTargetDistance
- signalStrength optional
```

```
None
Given frame corrupt
Then frame diabaikan dan error dicatat sebagai warning.
```

```
None
Given stream memiliki multiple frame
Then parser dapat memproses frame secara berurutan.
```

US-02.04 — Map sensor target state to isOccupied

As an edge developer,
I want target state sensor diubah menjadi `isOccupied`,
So that IoT Gateway menerima payload occupancy yang sederhana.

Acceptance Criteria:

```
None
Given targetState = NoTarget
Then isOccupied = false.
```

```
None
Given targetState = MovingTarget
Then isOccupied = true.
```

```
None
Given targetState = StationaryTarget
Then isOccupied = true.
```

```
None
Given targetState = MovingAndStationaryTarget
Then isOccupied = true.
```

US-02.05 — Implement Occupied to Empty debounce

As a facility system,
I want transisi Occupied → Empty ditunda selama 30 detik,
So that false-negative sensor tidak langsung mengosongkan ruangan.

Acceptance Criteria:

```
None
Given state sebelumnya Occupied
When sensor membaca NoTarget
Then runtime tidak langsung mengirim isOccupied=false.
```

```
None
Given NoTarget bertahan minimal 30 detik
Then runtime mengubah state menjadi Empty.
```

```
None
Given selama 30 detik sensor kembali mendeteksi target
Then debounce dibatalkan dan state tetap Occupied.
```

US-02.06 — Generate readingId per telemetry reading

As an edge developer,
I want setiap telemetry payload memiliki `readingId`,
So that log dan debugging bisa menelusuri pembacaan sensor.

Acceptance Criteria:

```
None
Given telemetry dibuat
Then readingId berupa GUID.
```

```
None
Given payload dikirim ulang karena network error
Then readingId boleh tetap sama untuk retry payload yang sama.
```

```
None
Given payload baru dibuat
Then readingId berbeda dari payload sebelumnya.
```

US-02.07 — Send telemetry to IoT Gateway via HTTP Keep-Alive

As an edge device,
I want mengirim telemetry via HTTP POST dengan Keep-Alive,
So that pengiriman berkala lebih efisien.

Acceptance Criteria:

```
None
Given telemetry interval 1000 ms
Then runtime mengirim POST /api/telemetry sesuai interval.
```

```
None
Given DeviceToken tersedia
Then request memakai header Authorization: Bearer {DeviceToken}.
```

```
None
Given HTTP client dibuat
Then connection reuse / Keep-Alive aktif.
```

US-02.08 — Implement lightweight network retry

As an edge device,
I want retry ringan saat network error,
So that telemetry tidak langsung gagal saat koneksi transient.

Acceptance Criteria:

```
None
Given request gagal karena timeout
Then runtime mencoba retry dengan batas maksimal.
```

```
None
Given server return 401
Then runtime tidak melakukan retry terus-menerus.
```

```
None
Given server return 404 DEVICE_NOT_BOUND
Then runtime mencatat error dan tetap berjalan.
```

US-02.09 — Add edge runtime logging

As an operator,
I want runtime Edge mencatat event penting,
So that troubleshooting sensor dan koneksi lebih mudah.

Acceptance Criteria:

```
None
Given runtime start
Then log mencatat deviceId, gateway URL, UART port, baud rate.
```

```
None
Given telemetry berhasil dikirim
Then log debug dapat mencatat status response.
```

```
None
Given token atau secret dipakai
Then token tidak boleh muncul di log.
```

EPIC-03 — IoT Gateway Device Provisioning and Device Token Security

Type: Epic
Priority: P0
Component: IoT Gateway
Release: MVP-1 Edge + IoT Gateway

Goal

IoT Gateway dapat membuat device, menghasilkan opaque Device Token, menyimpan token secara hashed, memvalidasi token dari Edge, dan mendukung token rotation.

US-03.01 — Create SQLite schema for device registry

As an IoT Gateway developer,
I want tabel device dibuat di SQLite,
So that perangkat fisik dapat diregistrasi dan dikelola.

Acceptance Criteria:

```
None
Given migration dijalankan
Then tabel Device tersedia dengan field:
- Id
- DeviceName
- HardwareModel
- IsActive
 -RoomId
- CreatedAtUtc
- UpdatedAtUtc
```

```
None
Given Id sudah terdaftar
Then sistem mencegah duplikasi jika aturan unique diterapkan.
```

```
None
Given device dibuat
Then CreatedAtUtc dan UpdatedAtUtc tersimpan UTC.
```

US-03.02 — Create DeviceCredential table

As an IoT Gateway developer,
I want credential device disimpan terpisah dari data device,
So that token management lebih aman.

Acceptance Criteria:

```
None
Given migration dijalankan
Then tabel DeviceCredential tersedia.
```

```
None
Given token dibuat
Then hanya TokenHash yang disimpan.
```

```
None
Given token lama direvoke
Then RevokedAtUtc terisi dan IsActive=false.
```

US-03.03 — Implement POST /api/devices

As an admin,
I want membuat device baru di IoT Gateway,
So that Raspberry Pi bisa diprovision.

Acceptance Criteria:

```
None
Given admin mengirim request valid
When POST /api/devices dipanggil
Then sistem membuat DeviceId dan DeviceToken.
```

```
None
Given device berhasil dibuat
Then response menampilkan plaintext DeviceToken hanya sekali.
```

```
None
Given request invalid
Then API return 400 Bad Request.
```

US-03.04 — Generate secure opaque Device Token

As a security engineer,
I want DeviceToken dibuat dari random bytes yang kuat,
So that token sulit ditebak.

Acceptance Criteria:

```
None
Given device dibuat
Then token berformat dev_{base64url_random_32_bytes}.
```

```
None
Given token dibuat
Then plaintext token tidak disimpan di database.
```

```
None
Given token hash dibuat
Then hash memakai server-side pepper.
```

US-03.05 — Validate Device Token on telemetry request

As an IoT Gateway,
I want memvalidasi DeviceToken di header Authorization,
So that hanya device terdaftar yang bisa mengirim telemetry.

Acceptance Criteria:

```
None
Given Authorization header kosong
Then API return 401 Unauthorized.
```

```
None
Given token invalid
Then API return 401 INVALID_DEVICE_TOKEN.
```

```
None
Given token valid tapi revoked
Then API return 401 REVOKED_DEVICE_TOKEN.
```

US-03.06 — Use constant-time token hash comparison

As a security engineer,
I want perbandingan token hash dilakukan secara constant-time,
So that risiko timing attack berkurang.

Acceptance Criteria:

```
None
Given token diterima
Then sistem menghitung hash incoming token.
```

```
None
Given hash dibandingkan
Then comparison menggunakan constant-time comparison.
```

```
None
Given token tidak cocok
Then API menolak tanpa membocorkan detail.
```

US-03.07 — Implement device token rotation API

As an admin,
I want merotasi DeviceToken,
So that token yang bocor bisa diganti.

Acceptance Criteria:

```
None
Given admin memanggil POST /api/devices/{deviceId}/tokens/rotate
Then sistem membuat token baru.
```

```
None
Given token baru dibuat
Then token lama direvoke.
```

```
None
Given response dikembalikan
Then plaintext token baru hanya tampil sekali.
```

US-03.08 — Implement device active/inactive state

As an admin,
I want mengaktifkan atau menonaktifkan device,
So that device bermasalah bisa dihentikan tanpa menghapus data.

Acceptance Criteria:

```
None
Given device inactive
When device mengirim telemetry
Then API return 409 DEVICE_INACTIVE.
```

```
None
Given device active
Then telemetry dapat diproses jika token valid.
```

```
None
Given status device berubah
Then UpdatedAtUtc ikut diperbarui.
```

EPIC-04 — IoT Gateway Room Sync and Device Binding

Type: Epic
Priority: P0

Component: IoT Gateway
Release: MVP-1 Edge + IoT Gateway

Goal

IoT Gateway dapat mengambil daftar room aktif dari Booking Backend, lalu admin bisa melakukan binding Device → Room.

US-04.01 — Implement Booking Backend configuration in IoT Gateway

As an IoT Gateway developer,
I want menyimpan konfigurasi Booking Backend,
So that IoT Gateway tahu endpoint tujuan untuk token dan room sync.

Acceptance Criteria:

```
None
Given service start
Then IoT Gateway membaca:
- BOOKING_BACKEND_BASE_URL
- GATEWAY_ID
- GATEWAY_CLIENT_ID
- GATEWAY_CLIENT_SECRET
```

```
None
Given credential kosong
Then service gagal start dengan error jelas.
```

```
None
Given secret dipakai
Then secret tidak muncul di log.
```

US-04.02 — Implement GET active rooms client

As an IoT Gateway,
I want mengambil daftar room aktif dari Booking Backend,
So that admin bisa memilih room saat binding device.

Acceptance Criteria:

```
None
Given GatewayAccessToken valid
When IoT Gateway memanggil GET /api/rooms/active
Then daftar room aktif diterima.
```

```
None
Given token expired
Then IoT Gateway refresh token lalu mencoba ulang.
```

```
None
Given Booking Backend down
Then error ditampilkan secara aman di dashboard/log.
```

US-04.03 — Create RoomDeviceMapping table

As an IoT Gateway developer,
I want menyimpan mapping Device → Room di SQLite,
So that telemetry device dapat diterjemahkan ke roomId.

Acceptance Criteria:

```
None
Given migration dijalankan
Then tabel RoomDeviceMapping tersedia.
```

```
None
Given device sudah punya active binding
Then binding aktif kedua tidak boleh dibuat.
```

```
None
Given binding dibuat
Then roomName dan googleCalendarResourceId tersimpan sebagai
metadata cache.
```

US-04.04 — Implement POST /api/devices/{deviceId}/bind

As an admin,
I want bind device ke room,
So that telemetry sensor bisa dipetakan ke ruang meeting.

Acceptance Criteria:

```
None
Given device valid dan room valid
When API bind dipanggil
Then mapping tersimpan di SQLite.
```

```
None
Given device tidak ditemukan
Then API return 404 DEVICE_NOT_FOUND.
```

```
None
Given binding berhasil
Then IMemoryCache di-refresh.
```

US-04.05 — Refresh IMemoryCache after binding

As an IoT Gateway,
I want cache mapping diperbarui setelah binding,
So that telemetry berikutnya langsung memakai roomId yang benar.

Acceptance Criteria:

```
None
Given binding dibuat
Then key DeviceRoomMapping:{deviceId} diperbarui.
```

```
None
Given mapping berubah
Then telemetry memakai mapping terbaru.
```

```
None
Given cache miss
Then IoT Gateway fallback ke SQLite dan repopulate cache.
```

US-04.06 — Validate bound device before telemetry processing

As an IoT Gateway,
I want memastikan device sudah bound ke room,
So that event tanpa roomId tidak dikirim ke Booking Backend.

Acceptance Criteria:

```
None
Given device belum bound
When telemetry diterima
Then API return 404 DEVICE_NOT_BOUND.
```

```
None
Given device bound
Then telemetry diproses ke tahap state detection.
```

```
None
Given mapping inactive
```

```
Then telemetry ditolak.
```

US-04.07 — Support re-binding device to another room

As an admin,
I want memindahkan device ke room lain,
So that perubahan fisik sensor bisa direfleksikan di sistem.

Acceptance Criteria:

```
None
Given device sudah bound ke Room A
When admin bind ke Room B
Then binding lama dibuat inactive.
```

```
None
Given binding baru dibuat
Then cache memakai Room B.
```

```
None
Given re-binding berhasil
Then sistem membuat audit log.
```

EPIC-05 — IoT Gateway Telemetry Ingestion and State Management

Type: Epic
Priority: P0
Component: IoT Gateway
Release: MVP-1 Edge + IoT Gateway

Goal

IoT Gateway menerima telemetry dari Edge, memvalidasi payload, menyimpan current state di IMemoryCache, melakukan deduplication, dan hanya membuat event saat state berubah.

US-05.01 — Implement POST /api/telemetry endpoint

As an Edge Device,
I want mengirim telemetry ke IoT Gateway,
So that sistem dapat mengevaluasi okupansi ruangan.

Acceptance Criteria:

```
None
Given request valid
When POST /api/telemetry dipanggil
Then API return 200 OK.
```

```
None
Given payload missing required field
Then API return 400 Bad Request.
```

```
None
Given token invalid
Then API return 401 Unauthorized.
```

US-05.02 — Validate telemetry payload

As an IoT Gateway,
I want memvalidasi telemetry payload,
So that data invalid tidak masuk ke pipeline.

Acceptance Criteria:

```
None
Given deviceId bukan GUID
Then request ditolak.
```

```
None
Given timestamp bukan UTC
Then request ditolak.
```

```
None
Given timestamp skew lebih dari ±2 menit
Then request ditolak atau diberi warning sesuai konfigurasi.
```

```
None
Given isOccupied null
Then request ditolak.
```

US-05.03 — Store current device and room state in IMemoryCache

As an IoT Gateway,
I want menyimpan current occupancy state di memory,
So that evaluasi hot path cepat dan tidak menulis database untuk setiap telemetry.

Acceptance Criteria:

```
None
Given telemetry valid
Then DeviceState:{deviceId} diperbarui.
```

```
None
Given roomId tersedia
Then RoomState:{roomId} diperbarui.
```

```
None
Given cache kosong
Then initial state dibuat dari telemetry pertama.
```

US-05.04 — Drop telemetry when state is unchanged

As an IoT Gateway,
I want membuang telemetry jika occupancy tidak berubah,
So that sistem tidak membebani Booking Backend dan database.

Acceptance Criteria:

```
None
Given previous state = Occupied
And incoming state = Occupied
Then API return stateChanged=false.
```

```
None
Given state tidak berubah
Then tidak ada insert ke GatewayWebhookOutbox.
```

```
None
Given state tidak berubah
Then tidak ada call webhook ke Booking Backend.
```

US-05.05 — Generate occupancy transition event when state changes

As an IoT Gateway,
I want membuat event saat state berubah,
So that Booking Backend menerima perubahan definitif.

Acceptance Criteria:

```
None
Given previous state = Empty
And incoming state = Occupied
Then eventType = Transition dibuat.
```

```
None
Given previous state = Occupied
And incoming state = Empty
Then eventType = Transition dibuat.
```

```
None
Given event dibuat
Then event memiliki eventId GUID.
```

US-05.06 — Return stateChanged response to Edge

As an Edge Device,
I want mendapat response apakah telemetry mengubah state,
So that debugging Edge lebih mudah.

Acceptance Criteria:

```
None
Given state tidak berubah
Then response accepted=true dan stateChanged=false.
```

```
None
Given state berubah
Then response accepted=true dan stateChanged=true.
```

```
None
Given state berubah
Then response menyertakan generatedEventId.
```

US-05.07 — Create snapshot event after gateway restart

As an IoT Gateway,
I want mengirim snapshot state setelah restart,
So that Booking Backend bisa menyelaraskan current occupancy.

Acceptance Criteria:

```
None
Given gateway baru restart
When telemetry valid pertama diterima untuk device bound
Then eventType Snapshot dapat dibuat.
```

```
None
Given snapshot diterima Booking Backend
Then snapshot tidak dihitung sebagai transition analytics wajib.
```

```
None
Given snapshot lebih lama dari current state
Then Booking Backend boleh mengabaikan.
```

EPIC-06 — IoT Gateway Webhook Outbox, Retry, and JWT Client

Type: Epic
Priority: P0
Component: IoT Gateway
Release: MVP-2 Booking Backend Occupancy

Goal

IoT Gateway dapat mengirim occupancy event ke Booking Backend secara reliable tanpa message broker, menggunakan SQLite Outbox, retry policy, dan JWT client_credentials flow.

US-06.01 — Create GatewayWebhookOutbox table

As an IoT Gateway developer,
I want tabel outbox dibuat di SQLite,
So that event tidak hilang saat Booking Backend down.

Acceptance Criteria:

```
None
Given migration dijalankan
Then tabel GatewayWebhookOutbox tersedia.
```

```
None
Given event baru dibuat
Then EventId unique tersimpan.
```

```
None
Given event pending
Then Status = Pending dan NextAttemptAtUtc terisi.
```

US-06.02 — Insert transition event into outbox before dispatch

As an IoT Gateway,
I want menyimpan event ke outbox sebelum call Booking Backend,
So that event tidak hilang saat network failure.

Acceptance Criteria:

```
None
Given state berubah
Then event disimpan ke outbox sebelum HTTP dispatch.
```

```
None
Given insert outbox gagal
Then webhook tidak dikirim.
```

```
None
Given insert sukses
Then dispatcher boleh mencoba kirim event.
```

US-06.03 — Implement JWT token client for Booking Backend

As an IoT Gateway,
I want meminta JWT dari Booking Backend,
So that gateway dapat mengakses endpoint protected.

Acceptance Criteria:

```
None
Given clientId dan clientSecret valid
When POST /api/auth/gateway/token dipanggil
Then accessToken diterima.
```

```
None
Given token diterima
Then token disimpan di memory cache.
```

```
None
Given expiresIn = 900
Then refresh dijadwalkan sebelum expired.
```

US-06.04 — Refresh JWT 60 seconds before expiration

As an IoT Gateway,
I want refresh JWT sebelum expired,
So that webhook tidak gagal karena token kedaluwarsa.

Acceptance Criteria:

```
None
Given token expired dalam <= 60 detik
Then gateway meminta token baru.
```

```
None
Given token masih valid lebih dari 60 detik
Then gateway reuse token yang ada.
```

```
None
Given refresh gagal
Then dispatcher menunda pengiriman event.
```

US-06.05 — Dispatch occupancy webhook to Booking Backend

As an IoT Gateway,
I want mengirim event ke POST /api/webhooks/occupancy,
So that Booking Backend menerima state transition.

Acceptance Criteria:

```
None
Given outbox event Pending
When dispatcher berjalan
Then request dikirim ke Booking Backend dengan Authorization Bearer
JWT.
```

```
Given request dikirim
Then header Idempotency-Key berisi eventId.
```

```
Given response 200 OK
Then outbox event ditandai Delivered.
```

US-06.06 — Implement Polly immediate retry

As an IoT Gateway,
I want immediate retry untuk transient error,
So that failure singkat tidak masuk background retry.

Acceptance Criteria:

```
Given Booking Backend timeout
Then dispatcher retry maksimal 3 kali.
```

```
Given response 5xx
Then dispatcher retry sesuai policy.
```

```
Given response 4xx non-retryable
Then dispatcher tidak retry immediate.
```

US-06.07 — Implement background outbox dispatcher

As an IoT Gateway,
I want background worker mengirim ulang event pending,
So that event terkirim setelah Booking Backend pulih.

Acceptance Criteria:

```
Given event Status = Pending
And NextAttemptAtUtc <= now
Then worker mencoba kirim event.
```

```
Given pengiriman gagal
Then AttemptCount bertambah.
```

```
Given pengiriman gagal
Then NextAttemptAtUtc dihitung ulang.
```

US-06.08 — Handle duplicate 200 response

As an IoT Gateway,
I want duplicate response dianggap sukses,
So that outbox tidak retry event yang sudah diproses.

Acceptance Criteria:

```
Given Booking Backend return accepted=true duplicate=true
Then outbox event ditandai Delivered.
```

```
Given duplicate event sudah delivered
Then worker tidak mengirim ulang.
```

```
Given response duplicate
```

```
Then tidak dianggap error.
```

EPIC-07 — Booking Backend Gateway Registration and JWT Auth

Type: Epic
Priority: P0
Component: Booking Backend
Release: MVP-2 Booking Backend Occupancy

Goal

Booking Backend dapat meregistrasi IoT Gateway, menghasilkan clientId/clientSecret, menerbitkan JWT, dan memvalidasi scope untuk endpoint gateway.

US-07.01 — Create IoTGatewayClient table

As a Booking Backend developer,
I want menyimpan IoT Gateway client di RDBMS,
So that gateway bisa diautentikasi.

Acceptance Criteria:

```
None
Given migration dijalankan
Then tabel IoTGatewayClient tersedia.
```

```
None
Given clientId dibuat
Then clientId unique.
```

```
None
Given clientSecret dibuat
Then hanya hash yang disimpan.
```

US-07.02 — Implement POST /api/admin/iot-gateways

As an admin,
I want mendaftarkan IoT Gateway di Booking Backend,
So that gateway bisa mendapatkan credential.

Acceptance Criteria:

```
None
Given admin JWT valid
When request valid dikirim
Then gatewayId, clientId, clientSecret dibuat.
```

```
None
Given response dikirim
Then plaintext clientSecret hanya tampil sekali.
```

```
None
Given admin JWT tidak valid
Then API return 401 Unauthorized.
```

US-07.03 — Generate secure gateway client secret

As a security engineer,
I want clientSecret dibuat secara aman,
So that credential gateway sulit ditebak.

Acceptance Criteria:

```
None
Given gateway dibuat
Then clientSecret dibuat dari random bytes yang kuat.
```

```
None
Given secret dibuat
Then secret berformat gws_{random}.
```

```
None
Given secret disimpan
Then hanya hash yang masuk database.
```

US-07.04 — Implement POST /api/auth/gateway/token

As an IoT Gateway,
I want menukar clientId dan clientSecret menjadi JWT,
So that gateway bisa memanggil API Booking Backend.

Acceptance Criteria:

```
None
Given clientId dan clientSecret valid
Then API return tokenType=Bearer dan accessToken.
```

```
None
Given secret invalid
Then API return 401 INVALID_CLIENT.
```

```
None
Given gateway inactive
Then API return 403 GATEWAY_INACTIVE.
```

US-07.05 — Generate JWT with required claims

As a Booking Backend,
I want JWT memiliki claims standar,
So that endpoint dapat memvalidasi issuer, audience, gatewayId, dan scope.

Acceptance Criteria:

```
None
Given token dibuat
Then token memiliki iss, aud, sub, gatewayId, clientId, scope, iat,
exp, jti.
```

```
None
Given token dibuat
Then TTL token = 15 menit.
```

```
None
Given requested scope valid
Then scope masuk ke JWT.
```

US-07.06 — Implement scope-based authorization

As a Booking Backend,
I want endpoint memvalidasi scope JWT,
So that gateway hanya bisa mengakses API sesuai izin.

Acceptance Criteria:

```
None
Given endpoint /api/webhooks/occupancy
Then required scope = occupancy.write.
```

```
None
Given endpoint /api/rooms/active
```

```
Then required scope = rooms.read.
```

```
None
Given token tanpa scope yang dibutuhkan
Then API return 403 INSUFFICIENT_SCOPE.
```

US-07.07 — Implement gateway client secret rotation

As an admin,
I want merotasi clientSecret gateway,
So that credential yang bocor bisa diganti.

Acceptance Criteria:

```
None
Given admin memanggil rotation endpoint
Then clientSecret baru dibuat.
```

```
None
Given secret baru dibuat
Then secret lama direvoke.
```

```
None
Given response dikirim
Then plaintext secret baru hanya tampil sekali.
```

EPIC-08 — Booking Backend Room and Booking Domain Foundation

Type: Epic
Priority: P0
Component: Booking Backend
Release: MVP-2 Booking Backend Occupancy

Goal

Membangun fondasi domain room dan booking agar occupancy event dapat dikaitkan dengan reservasi, Google Calendar, dan analytics.

US-08.01 — Create Room table and repository

As a Booking Backend developer,
I want menyimpan data room,
So that sistem memiliki daftar ruangan sebagai referensi utama.

Acceptance Criteria:

```
None
Given migration dijalankan
Then tabel Room tersedia.
```

```
None
Given room dibuat
Then field Name, Floor, Capacity, GoogleCalendarResourceId,
IsActive tersimpan.
```

```
None
Given GoogleCalendarResourceId sama
Then sistem mencegah duplikasi.
```

US-08.02 — Implement room management service

As an admin,
I want mengelola room,
So that room aktif bisa dipakai untuk booking dan sensor binding.

Acceptance Criteria:

```
None
Given admin membuat room valid
Then room tersimpan.
```

```
None
Given admin menonaktifkan room
Then room tidak muncul di active rooms.
```

```
None
Given room dipakai booking aktif
Then delete fisik tidak diperbolehkan; gunakan deactivate.
```

US-08.03 — Implement GET /api/rooms/active

As an IoT Gateway,
I want mengambil daftar room aktif,
So that device bisa dibind ke room.

Acceptance Criteria:

```
None
Given GatewayAccessToken punya scope rooms.read
Then API return daftar room aktif.
```

```
None
Given token tanpa rooms.read
Then API return 403.
```

```
None
Given room inactive
Then room tidak muncul di response.
```

US-08.04 — Create Booking table

As a Booking Backend developer,
I want menyimpan booking lokal,
So that sistem bisa menjalankan auto-release dan ad-hoc booking.

Acceptance Criteria:

```
None
Given migration dijalankan
Then tabel Booking tersedia.
```

```
None
Given booking dibuat
Then RoomId, GoogleCalendarEventId, StartUtc, EndUtc, Status
tersimpan.
```

```
None
Given booking status berubah
Then UpdatedAtUtc diperbarui.
```

US-08.05 — Implement booking overlap validation

As a booking system,
I want mencegah booking bentrok,
So that satu room tidak memiliki dua booking aktif pada waktu yang sama.

Acceptance Criteria:

```
None
Given booking baru dibuat
And ada booking aktif yang overlap
Then sistem menolak booking.
```

```
None
Given booking cancelled
Then booking tersebut tidak dianggap conflict.
```

```
None
Given booking back-to-back tanpa overlap
Then booking diperbolehkan.
```

US-08.06 — Implement current booking lookup by room and time

As a business logic service,
I want mencari booking aktif berdasarkan room dan waktu,
So that auto-release dan ad-hoc booking bisa berjalan.

Acceptance Criteria:

```
None
Given room memiliki booking aktif pada now
Then service return booking tersebut.
```

```
None
Given room tidak memiliki booking aktif
Then service return null.
```

```
None
Given booking cancelled
```

```
Then booking tidak dianggap aktif.
```

EPIC-09 — Booking Backend Occupancy Webhook and Current State

Type: Epic
Priority: P0
Component: Booking Backend
Release: MVP-2 Booking Backend Occupancy

Goal

Booking Backend menerima occupancy webhook dari IoT Gateway, memvalidasi JWT, melakukan idempotency, memperbarui current occupancy, menyimpan transition event, dan men-trigger business logic.

US-09.01 — Implement POST /api/webhooks/occupancy

As an IoT Gateway,
I want mengirim occupancy event ke Booking Backend,
So that current state room diperbarui.

Acceptance Criteria:

```
None
Given JWT valid dan payload valid
Then API return accepted=true.
```

```
None
Given JWT invalid
Then API return 401 Unauthorized.
```

```
None
Given payload invalid
Then API return 400 Bad Request.
```

US-09.02 — Validate occupancy webhook payload

As a Booking Backend,
I want memvalidasi semua field payload webhook,
So that data occupancy yang masuk bisa dipercaya.

Acceptance Criteria:

```
None
Given eventId bukan GUID
Then API return 400.
```

```
None
Given roomId tidak ditemukan
Then API return 404 ROOM_NOT_FOUND.
```

```
None
Given changedAt bukan UTC
Then API return 400.
```

```
None
Given eventType bukan Transition atau Snapshot
Then API return 400.
```

US-09.03 — Implement idempotency by SourceEventId

As a Booking Backend,
I want mendeteksi duplicate eventId,
So that webhook retry tidak menciptakan data duplikat.

Acceptance Criteria:

```
Given SourceEventId belum pernah diproses
Then event diproses normal.
```

```
Given SourceEventId sudah pernah diproses
Then API return 200 duplicate=true.
```

```
Given duplicate event diterima
Then RoomOccupancyEvent tidak bertambah.
```

US-09.04 — Create RoomCurrentOccupancy table

As a Booking Backend developer,
I want menyimpan current occupancy per room,
So that dashboard dan business logic bisa membaca status terbaru.

Acceptance Criteria:

```
Given migration dijalankan
Then tabel RoomCurrentOccupancy tersedia.
```

```
Given event baru diterima
Then row room terkait dibuat atau diperbarui.
```

```
Given event lebih lama dari LastChangedAtUtc
Then sistem tidak menimpa state yang lebih baru.
```

US-09.05 — Upsert RoomCurrentOccupancy

As a Booking Backend,
I want melakukan upsert current occupancy,
So that state room selalu sinkron dengan event terbaru.

Acceptance Criteria:

```
None
Given room belum punya current state
Then row baru dibuat.
```

```
None
Given room sudah punya current state
Then IsOccupied dan timestamp diperbarui.
```

```
None
Given event Snapshot
Then current state diperbarui jika observedAt lebih baru.
```

US-09.06 — Insert transition into RoomOccupancyEvent

As a Booking Backend,
I want menyimpan transition event,
So that historical analytics dapat dihitung.

Acceptance Criteria:

```
None
Given eventType = Transition
Then row RoomOccupancyEvent dibuat.
```

```
None
Given eventType = Snapshot
Then row transition analytics tidak wajib dibuat.
```

```
None
Given insert berhasil
Then SourceEventId unique tersimpan.
```

US-09.07 — Update previous event duration

As a Booking Backend,
I want mengisi DurationSeconds pada event sebelumnya,
So that query analytics lebih efisien.

Acceptance Criteria:

```
None
Given previous event = Occupied pada 10:00
And current event = Empty pada 10:45
Then previous event DurationSeconds = 2700.
```

```
None
Given tidak ada previous event
Then DurationSeconds tetap null.
```

```
None
Given current event lebih lama dari previous event
Then duration tidak dihitung negatif.
```

US-09.08 — Execute webhook processing inside DB transaction

As a Booking Backend developer,
I want proses occupancy webhook berjalan dalam transaction,
So that current state, event, dan idempotency konsisten.

Acceptance Criteria:

```
None
Given insert event gagal
Then current state tidak ter-update sebagian.
```

```
None
Given semua operasi sukses
Then transaction commit.
```

```
None
Given exception terjadi
Then transaction rollback dan error dicatat.
```

EPIC-10 — Historical Occupancy Data and Analytics Aggregation

Type: Epic
Priority: P0
Component: Booking Backend
Release: MVP-4 Analytics + Dashboard

Goal

Booking Backend menyimpan historical transition event dan menghitung aggregate occupancy agar grafik utilization, occupancy summary, dan ghost meeting report bisa dibuat tanpa menyimpan raw telemetry.

US-10.01 — Create RoomOccupancyEvent table

As a data developer,
I want menyimpan event perubahan occupancy,
So that histori occupancy bisa dianalisis.

Acceptance Criteria:

```
None
Given migration dijalankan
Then tabel RoomOccupancyEvent tersedia.
```

```
None
Given SourceEventId sama
Then insert kedua ditolak oleh unique constraint.
```

```
None
Given query berdasarkan RoomId dan ChangedAtUtc
Then index digunakan.
```

US-10.02 — Create RoomOccupancyAggregate table

As a data developer,
I want menyimpan aggregate occupancy per bucket,
So that dashboard analytics tidak perlu menghitung dari raw event setiap request.

Acceptance Criteria:

```
None
Given migration dijalankan
Then tabel RoomOccupancyAggregate tersedia.
```

```
None
Given aggregate dibuat
Then BucketSize harus Minute, Hour, atau Day.
```

```
None
Given RoomId + BucketStartUtc + BucketSize sama
Then data di-upsert, bukan dibuat duplikat.
```

US-10.03 — Implement utilization formula

As a product owner,
I want utilization dihitung konsisten,
So that laporan penggunaan ruangan tidak ambigu.

Acceptance Criteria:

```
None
Given bucket berdurasi 3600 detik
And occupiedSeconds = 1800
Then utilizationPercent = 50.0.
```

```
None
Given bucketDurationSeconds = 0
Then sistem tidak membagi dengan nol.
```

```
None
Given occupiedSeconds > bucket duration
Then sistem menormalkan atau menolak data invalid.
```

Formula:

```
None
utilizationPercent = occupiedSeconds / bucketDurationSeconds * 100
```

US-10.04 — Implement background aggregate calculation job

As a Booking Backend,
I want menghitung aggregate setiap 5 menit,
So that analytics tetap cukup fresh untuk MVP.

Acceptance Criteria:

```
None
Given job berjalan setiap 5 menit
Then job mengambil occupancy events terbaru.
```

```
None
Given event dalam 24 jam terakhir berubah
Then bucket terkait dihitung ulang.
```

```
None
Given aggregate sudah ada
Then job melakukan upsert.
```

US-10.05 — Calculate occupied and empty seconds per bucket

As a data developer,
I want menghitung durasi occupied dan empty,
So that grafik bisa menampilkan penggunaan ruangan.

Acceptance Criteria:

```
None
Given room Occupied dari 10:00 sampai 10:30
Then bucket 10:00-11:00 memiliki occupiedSeconds=1800.
```

```
None
Given room Empty dari 10:30 sampai 11:00
Then bucket 10:00-11:00 memiliki emptySeconds=1800.
```

```
None
Given event melewati batas bucket
Then durasi dipecah ke masing-masing bucket.
```

US-10.06 — Calculate bookingSeconds per bucket

As a product owner,
I want membandingkan okupansi fisik dengan booking,
So that ghost meeting dan utilization gap bisa terlihat.

Acceptance Criteria:

```
None
Given booking dari 10:00 sampai 11:00
Then bookingSeconds pada bucket 10:00 = 3600.
```

```
None
Given booking dari 10:30 sampai 11:30
Then bookingSeconds dibagi ke bucket 10:00 dan 11:00.
```

```
None
Given booking cancelled
Then booking tidak dihitung sebagai bookingSeconds aktif.
```

US-10.07 — Recalculate aggregate safely

As a data developer,
I want aggregate recalculation idempotent,
So that job bisa diulang tanpa merusak data.

Acceptance Criteria:

```
None
Given job dijalankan dua kali untuk rentang yang sama
Then hasil aggregate tetap sama.
```

```
None
Given event terlambat masuk dari outbox retry
Then bucket historis terkait dihitung ulang.
```

```
None
Given job gagal di tengah
Then run berikutnya bisa memperbaiki aggregate.
```

US-10.08 — Add analytics data retention policy

As an operator,
I want aturan retention data analytics,
So that database tidak tumbuh tanpa batas.

Acceptance Criteria:

```
None
Given RoomOccupancyEvent berumur lebih dari retention period
Then data dapat diarsipkan sesuai konfigurasi.
```

```
None
Given aggregate harian sudah tersedia
Then data event lama bisa dipertimbangkan untuk archive.
```

```
None
Given retention belum dikonfigurasi
Then sistem tidak menghapus data otomatis.
```

EPIC-11 — Analytics API and Reporting

Type: Epic
Priority: P1
Component: Booking Backend
Release: MVP-4 Analytics + Dashboard

Goal

Menyediakan API untuk grafik occupancy per room, summary utilization, dan ghost meeting report.

US-11.01 — Implement GET /api/analytics/rooms/{roomId}/occupancy

As a frontend dashboard,
I want mengambil data occupancy per room,
So that user bisa melihat grafik penggunaan ruangan.

Acceptance Criteria:

```
None
Given UserJwt valid
When API dipanggil dengan from, to, bucket
Then response berisi data bucket occupancy.
```

```
None
Given bucket invalid
Then API return 400.
```

```
None
Given roomId tidak ditemukan
Then API return 404.
```

US-11.02 — Validate analytics date range

As a Booking Backend,
I want membatasi rentang query analytics,
So that query tidak terlalu berat.

Acceptance Criteria:

```
None
Given from lebih besar dari to
Then API return 400.
```

```
None
Given range terlalu panjang untuk bucket Minute
Then API return 400 atau meminta bucket lebih besar.
```

```
None
Given datetime bukan UTC
Then API return 400.
```

US-11.03 — Implement GET /api/analytics/occupancy-summary

As a facility manager,
I want melihat summary utilization semua room,
So that saya tahu ruangan mana paling sering dan paling jarang dipakai.

Acceptance Criteria:

```
None
Given request valid
Then response berisi totalRooms dan averageUtilizationPercent.
```

```
None
Given data tersedia
Then response berisi mostUsedRooms dan leastUsedRooms.
```

```
None
Given tidak ada data
Then response tetap valid dengan list kosong.
```

US-11.04 — Implement GET /api/analytics/ghost-meetings

As a facility manager,
I want melihat laporan ghost meeting,
So that saya bisa mengevaluasi efektivitas auto-release.

Acceptance Criteria:

```
None
Given ghost meeting audit tersedia
Then API mengembalikan daftar ghost meeting.
```

```
None
Given autoReleasedMeetings tersedia
Then total dan estimatedReleasedMinutes dihitung.
```

```
None
Given tidak ada ghost meeting
Then totalGhostMeetings = 0.
```

US-11.05 — Add room utilization ranking

As a facility manager,
I want ranking room berdasarkan utilization,
So that saya bisa mengambil keputusan kapasitas ruang.

Acceptance Criteria:

```
None
Given utilization data tersedia
Then rooms dapat diurutkan descending.
```

```
None
Given utilization sama
Then sorting sekunder berdasarkan roomName.
```

```
None
Given room inactive
Then bisa dikecualikan sesuai query parameter.
```

US-11.06 — Add analytics authorization

As a security engineer,
I want analytics API hanya bisa diakses user authorized,
So that data operasional tidak terbuka untuk user tanpa izin.

Acceptance Criteria:

```
None
Given UserJwt kosong
Then API return 401.
```

```
None
Given user tidak punya permission analytics.read
Then API return 403.
```

```
None
Given user punya permission
Then API return data.
```

EPIC-12 — Google Calendar Integration

Type: Epic
Priority: P0
Component: Booking Backend
Release: MVP-3 Calendar + Business Logic

Goal

Booking Backend terhubung dengan Google Calendar menggunakan OAuth delegated access, melakukan sync booking, membuat ad-hoc booking, dan membatalkan booking saat auto-release.

US-12.01 — Configure Google OAuth delegated access

As an admin,
I want Booking Backend menggunakan OAuth delegated access,
So that sistem dapat mengelola calendar resource sesuai arsitektur PRD.

Acceptance Criteria:

```
None
Given OAuth config valid
Then Booking Backend dapat memperoleh access token Google.
```

```
None
Given OAuth token expired
Then sistem melakukan refresh token.
```

```
None
Given OAuth gagal
Then error dicatat tanpa membocorkan secret.
```

US-12.02 — Store Google Calendar resource ID per room

As an admin,
I want setiap room punya googleCalendarResourceId,
So that booking dapat disinkronkan dengan calendar resource.

Acceptance Criteria:

```
None
Given room dibuat
Then googleCalendarResourceId wajib jika room digunakan untuk
booking.
```

```
None
Given resource ID duplikat
Then sistem menolak.
```

```
None
Given room inactive
Then sync calendar untuk room tersebut bisa dihentikan.
```

US-12.03 — Implement Google Calendar push notification receiver

As a Booking Backend,
I want menerima Google Calendar push notification,
So that perubahan booking dari Google bisa disinkronkan.

Acceptance Criteria:

```
None
Given Google mengirim notification
Then endpoint menerima dan memvalidasi request.
```

```
None
Given notification valid
Then sistem menjadwalkan sync event terkait.
```

```
None
Given notification invalid
Then request ditolak.
```

US-12.04 — Sync Google Calendar events to Booking table

As a Booking Backend,
I want menyinkronkan event Google Calendar ke tabel Booking,
So that business logic lokal punya data reservasi.

Acceptance Criteria:

```
None
Given Google Calendar event baru
Then Booking lokal dibuat.
```

```
None
Given Google Calendar event berubah
Then Booking lokal diperbarui.
```

```
None
Given Google Calendar event dibatalkan
Then Booking lokal menjadi Cancelled.
```

US-12.05 — Create Google Calendar event for ad-hoc booking

As a Booking Backend,
I want membuat event Google Calendar untuk ad-hoc booking,
So that penggunaan spontan tetap tercatat di calendar.

Acceptance Criteria:

```
None
Given ad-hoc booking valid
Then Google Calendar event dibuat selama 30 menit.
```

```
None
Given Google Calendar create gagal
Then Booking lokal tidak boleh dianggap confirmed.
```

```
None
Given event berhasil dibuat
Then GoogleCalendarEventId tersimpan.
```

US-12.06 — Cancel Google Calendar event for auto-release

As a Booking Backend,
I want membatalkan calendar event saat auto-release,
So that room kembali available.

Acceptance Criteria:

```
None
Given booking memenuhi aturan auto-release
Then Google Calendar event dibatalkan.
```

```
None
Given Google Calendar cancel sukses
Then Booking status menjadi AutoReleased.
```

```
None
Given Google Calendar cancel gagal
Then retry atau error handling dijalankan.
```

EPIC-13 — Auto-Release and Ad-Hoc Booking Logic

Type: Epic
Priority: P0
Component: Booking Backend
Release: MVP-3 Calendar + Business Logic

Goal

Menggunakan data occupancy untuk otomatis membatalkan ghost meeting dan membuat booking spontan saat ruangan available tapi ditempati.

US-13.01 — Schedule auto-release check at T + 15 minutes

As a Booking Backend,
I want menjadwalkan pengecekan 15 menit setelah meeting mulai,
So that ghost meeting bisa dibatalkan otomatis.

Acceptance Criteria:

```
None
Given booking dimulai pada T
Then sistem menjadwalkan check pada T + 15 menit.
```

```
None
Given booking sudah cancelled sebelum T+15
Then check tidak membatalkan ulang.
```

```
None
Given service restart
Then scheduled check tetap bisa dipulihkan.
```

US-13.02 — Evaluate room occupancy for auto-release

As a Booking Backend,
I want memeriksa RoomCurrentOccupancy saat T+15,
So that sistem tahu apakah meeting benar-benar digunakan.

Acceptance Criteria:

```
None
Given RoomCurrentOccupancy.IsOccupied = false pada T+15
Then booking eligible untuk auto-release.
```

```
None
Given IsOccupied = true
Then booking tidak dibatalkan.
```

```
None
Given occupancy state unknown
Then sistem mengikuti rule fallback yang dikonfigurasi.
```

US-13.03 — Mark booking as AutoReleased

As a Booking Backend,
I want mengubah status booking menjadi AutoReleased,
So that riwayat booking mencatat pembatalan otomatis.

Acceptance Criteria:

```
Given Google Calendar cancel sukses
Then Booking.Status = AutoReleased.
```

```
Given Booking sudah AutoReleased
Then proses tidak dijalankan ulang.
```

```
Given update berhasil
Then UpdatedAtUtc diperbarui.
```

US-13.04 — Create GhostMeetingAudit record

As a facility manager,
I want setiap auto-release tercatat,
So that laporan ghost meeting dapat dibuat.

Acceptance Criteria:

```
Given booking auto-released
Then GhostMeetingAudit dibuat.
```

```
Given audit dibuat
Then reason berisi "No physical occupancy detected within grace
period."
```

```
Given duplicate auto-release dicegah
```

```
Then audit tidak terduplikasi.
```

US-13.05 — Detect ad-hoc booking candidate

As a Booking Backend,
I want mendeteksi room available yang occupied,
So that penggunaan spontan bisa dicatat otomatis.

Acceptance Criteria:

```
None
Given event Empty → Occupied diterima
And tidak ada active booking
Then sistem membuat ad-hoc candidate.
```

```
None
Given ada active booking
Then candidate tidak dibuat.
```

```
None
Given room inactive
Then candidate tidak dibuat.
```

US-13.06 — Confirm ad-hoc booking after 5 minutes

As a Booking Backend,
I want menunggu occupancy 5 menit berturut-turut,
So that sistem tidak membuat booking karena orang hanya lewat.

Acceptance Criteria:

```
None
Given room occupied selama minimal 5 menit
```

```
And room tetap available
Then ad-hoc booking dibuat.
```

```
None
Given room menjadi empty sebelum 5 menit
Then candidate dibatalkan.
```

```
None
Given booking lain masuk sebelum 5 menit
Then candidate dibatalkan.
```

US-13.07 — Create 30-minute ad-hoc booking

As a Booking Backend,
I want membuat booking 30 menit,
So that penggunaan spontan tercatat di Google Calendar.

Acceptance Criteria:

```
None
Given candidate valid
Then Booking dibuat dengan durasi 30 menit.
```

```
None
Given Google Calendar event berhasil dibuat
Then Booking status menjadi Confirmed.
```

```
None
Given Google Calendar gagal
Then error dicatat dan booking tidak dianggap confirmed.
```

EPIC-14 — Real-Time Updates and Frontend Dashboard

Type: Epic
Priority: P1
Component: Frontend + Booking Backend
Release: MVP-4 Analytics + Dashboard

Goal

Frontend dapat menampilkan status room real-time, grafik analytics, ghost meeting report, dan update langsung via SignalR.

US-14.01 — Implement SignalR Rooms Hub

As a frontend client,
I want subscribe ke `/hubs/rooms`,
So that status room berubah secara real-time tanpa refresh manual.

Acceptance Criteria:

```
None
Given client authenticated
Then client bisa connect ke /hubs/rooms.
```

```
None
Given occupancy berubah
Then event RoomOccupancyChanged dikirim.
```

```
None
Given client disconnect
Then reconnect dapat dilakukan oleh frontend.
```

US-14.02 — Publish RoomOccupancyChanged event

As a Booking Backend,
I want mengirim event saat current occupancy berubah,
So that UI selalu menampilkan status terbaru.

Acceptance Criteria:

```
None
Given occupancy webhook berhasil diproses
Then SignalR event RoomOccupancyChanged dikirim.
```

```
None
Given duplicate webhook
Then event tidak perlu dikirim ulang.
```

```
None
Given payload event dikirim
Then payload berisi roomId, roomName, isOccupied, changedAt.
```

US-14.03 — Build real-time room status dashboard

As a user,
I want melihat status Available/Occupied per room,
So that saya tahu room mana yang bisa dipakai.

Acceptance Criteria:

```
None
Given dashboard dibuka
Then daftar room aktif tampil.
```

```
None
Given SignalR event diterima
Then status room diperbarui tanpa reload.
```

```
None
Given room occupied
Then UI menampilkan state occupied dengan timestamp.
```

US-14.04 — Build occupancy analytics chart

As a facility manager,
I want melihat grafik occupancy per room,
So that saya bisa menganalisis penggunaan ruangan.

Acceptance Criteria:

```
None
Given user memilih room dan date range
Then frontend memanggil analytics occupancy API.
```

```
None
Given data bucket diterima
Then grafik menampilkan utilizationPercent per bucket.
```

```
None
Given data kosong
Then UI menampilkan empty state.
```

US-14.05 — Build occupancy summary dashboard

As a facility manager,
I want melihat summary most used dan least used rooms,
So that saya bisa mengambil keputusan fasilitas.

Acceptance Criteria:

```
None
Given summary API berhasil
Then totalRooms dan averageUtilization tampil.
```

```
None
Given mostUsedRooms tersedia
Then UI menampilkan ranking.
```

```
None
Given leastUsedRooms tersedia
Then UI menampilkan ranking.
```

US-14.06 — Build ghost meeting report page

As a facility manager,
I want melihat daftar ghost meeting,
So that saya bisa mengevaluasi meeting yang tidak digunakan.

Acceptance Criteria:

```
None
Given ghost meeting API berhasil
Then totalGhostMeetings tampil.
```

```
None
Given items tersedia
Then tabel menampilkan roomName, scheduledStart, scheduledEnd,
autoReleasedAt, reason.
```

```
None
Given tidak ada data
Then UI menampilkan empty state.
```

US-14.07 — Show auto-release and ad-hoc booking notifications

As a user,
I want mendapat update saat booking auto-released atau ad-hoc dibuat,
So that saya tahu perubahan room availability.

Acceptance Criteria:

```
None
Given BookingAutoReleased event diterima
Then UI menampilkan notifikasi.
```

```
None
Given AdHocBookingCreated event diterima
Then UI memperbarui booking list.
```

```
None
Given event invalid diterima
Then frontend tidak crash.
```

EPIC-15 — Admin Operations, Rotation, and Management UI

Type: Epic
Priority: P1
Component: Full-stack
Release: MVP-4 Analytics + Dashboard

Goal

Admin dapat mengelola IoT Gateway, device, binding, token rotation, dan status operasional melalui UI atau API.

US-15.01 — Build IoT Gateway registration admin UI

As an admin,
I want mendaftarkan IoT Gateway dari UI,
So that saya tidak perlu memanggil API manual.

Acceptance Criteria:

```
None
Given admin membuka page gateway management
Then form create gateway tersedia.
```

```
None
Given submit sukses
Then clientId dan clientSecret ditampilkan sekali.
```

```
None
Given admin meninggalkan page
Then clientSecret tidak bisa dilihat ulang.
```

US-15.02 — Build device provisioning UI

As an admin,
I want membuat device dari dashboard IoT Gateway,
So that Raspberry Pi dapat diprovision dengan mudah.

Acceptance Criteria:

```
None
Given admin mengisi deviceName, hardwareModel, serialNumber
Then device dibuat.
```

```
None
Given device dibuat
Then DeviceToken ditampilkan sekali.
```

```
None
Given request gagal
Then UI menampilkan error yang jelas.
```

US-15.03 — Build device binding UI

As an admin,
I want memilih device dan room lalu melakukan binding,
So that telemetry device masuk ke room yang benar.

Acceptance Criteria:

```
None
Given active rooms berhasil di-fetch
Then daftar room tampil.
```

```
None
Given device belum bound
Then admin dapat memilih room.
```

```
None
Given binding berhasil
Then status device menjadi Bound.
```

US-15.04 — Build device token rotation UI

As an admin,
I want merotasi DeviceToken dari UI,
So that token device bisa diganti saat dibutuhkan.

Acceptance Criteria:

```
None
Given admin klik rotate token
Then sistem meminta konfirmasi.
```

```
None
Given rotasi sukses
Then token baru ditampilkan sekali.
```

```
None
Given token lama dipakai lagi
Then telemetry ditolak.
```

US-15.05 — Build gateway secret rotation UI

As an admin,
I want merotasi gateway client secret,
So that credential gateway tetap aman.

Acceptance Criteria:

```
None
Given admin klik rotate client secret
Then secret baru dibuat.
```

```
None
Given secret baru tampil
Then UI memberi instruksi update environment IoT Gateway.
```

```
None
Given secret lama dipakai
Then token endpoint menolak.
```

US-15.06 — Build device operational status page

As an operator,
I want melihat status device terakhir,
So that saya tahu sensor mana yang offline atau bermasalah.

Acceptance Criteria:

```
None
Given telemetry terakhir tersedia
Then UI menampilkan lastReadingAt.
```

```
None
Given device tidak mengirim data melewati threshold
Then UI menandai device sebagai potentially offline.
```

```
None
Given device unbound
Then UI menampilkan warning.
```

US-15.07 — Build outbox monitoring page

As an operator,
I want melihat pending webhook events,
So that saya tahu apakah ada masalah komunikasi ke Booking Backend.

Acceptance Criteria:

```
None
Given outbox memiliki pending event
Then UI menampilkan jumlah pending.
```

```
None
Given event gagal berulang
Then LastError dan AttemptCount tampil.
```

```
None
Given event delivered
Then status delivered tampil.
```

EPIC-16 — Observability, Security Hardening, QA, and Deployment

Type: Epic
Priority: P0
Component: Cross-cutting
Release: MVP-5 Hardening + Release

Goal

Memastikan sistem aman, terukur, observable, dapat diuji, dan siap deploy sebagai MVP.

US-16.01 — Add structured logging for all services

As an operator,
I want log terstruktur,
So that troubleshooting antar Edge, Gateway, dan Booking Backend lebih mudah.

Acceptance Criteria:

```
None
Given API request diterima
Then log mencatat traceId dan endpoint.
```

```
None
Given occupancy event diproses
Then log mencatat eventId, gatewayId, roomId.
```

```
None
Given token/secret ada di request
Then token/secret tidak muncul di log.
```

US-16.02 — Add health check endpoints

As a DevOps engineer,
I want health check endpoint,
So that service readiness dapat dipantau.

Acceptance Criteria:

```
None
Given service running
Then /health/live return healthy.
```

```
None
Given database tidak bisa diakses
Then /health/ready return unhealthy.
```

```
None
Given Booking Backend dependency down
Then IoT Gateway readiness menunjukkan degraded sesuai konfigurasi.
```

US-16.03 — Add metrics for telemetry and webhook

As an operator,
I want metrics telemetry dan webhook,
So that bottleneck dan failure rate bisa dipantau.

Acceptance Criteria:

```
None
Given telemetry diterima
```

```
Then counter telemetry_received bertambah.
```

```
None
Given state berubah
Then counter occupancy_transition_created bertambah.
```

```
None
Given webhook gagal
Then counter webhook_dispatch_failed bertambah.
```

US-16.04 — Implement security checklist validation

As a security engineer,
I want validasi keamanan diterapkan,
So that token dan endpoint tidak mudah disalahgunakan.

Acceptance Criteria:

```
None
Given HTTPS tidak aktif di production
Then deployment ditolak atau warning critical muncul.
```

```
None
Given JWT issuer/audience invalid
Then request ditolak.
```

```
None
Given revoked credential digunakan
Then request ditolak.
```

US-16.05 — Create unit tests for critical business logic

As a QA engineer,
I want unit test untuk logic penting,
So that regresi dapat dicegah.

Acceptance Criteria:

```
None
Given debounce logic diuji
Then Occupied → Empty hanya terjadi setelah 30 detik.
```

```
None
Given idempotency diuji
Then duplicate event tidak membuat row baru.
```

```
None
Given utilization formula diuji
Then hasil percent sesuai expected.
```

US-16.06 — Create integration tests for Edge → Gateway → Booking flow

As a QA engineer,
I want integration test end-to-end,
So that flow utama MVP tervalidasi.

Acceptance Criteria:

```
None
Given telemetry valid dikirim ke IoT Gateway
Then webhook dikirim ke Booking Backend.
```

```
None
Given Booking Backend menerima webhook
Then RoomCurrentOccupancy diperbarui.
```

```
None
Given duplicate webhook dikirim
Then response duplicate=true.
```

US-16.07 — Create failure scenario tests for outbox retry

As a QA engineer,
I want menguji saat Booking Backend down,
So that event tidak hilang.

Acceptance Criteria:

```
None
Given Booking Backend down
When state transition terjadi
Then event masuk GatewayWebhookOutbox.
```

```
None
Given Booking Backend kembali online
Then background worker mengirim event pending.
```

```
None
Given event sukses terkirim
Then status menjadi Delivered.
```

US-16.08 — Setup deployment scripts

As a DevOps engineer,
I want deployment script untuk setiap service,
So that MVP dapat dijalankan di environment target.

Acceptance Criteria:

```
None
Given deployment dijalankan
Then IoT Gateway service start dengan SQLite.
```

```
None
Given Booking Backend deploy
Then migration RDBMS berjalan.
```

```
None
Given environment variable lengkap
Then service berjalan tanpa manual config tambahan.
```

US-16.09 — Prepare MVP release checklist

As a product owner,
I want checklist rilis MVP,
So that go-live dilakukan dengan kontrol yang jelas.

Acceptance Criteria:

```
None
Given semua P0 story Done
Then MVP release candidate dapat dibuat.
```

```
None
Given acceptance criteria PRD terpenuhi
Then release siap UAT.
```

```
None
Given issue critical ditemukan
Then release diblokir sampai fixed.
```

Dependency Map

Gunakan relasi **Blocked by / Follows** di OpenProject untuk menghubungkan work package yang saling bergantung. OpenProject memang mendukung relasi antar work package seperti follows, blocked by, dan hierarchy parent-child. (OpenProject.org)

```
None
EPIC-01 harus dimulai dulu sebelum semua epic lain.

EPIC-02 bergantung pada:
- EPIC-01

EPIC-03 bergantung pada:
- EPIC-01

EPIC-04 bergantung pada:
- EPIC-03
- EPIC-07
- EPIC-08

EPIC-05 bergantung pada:
- EPIC-02
- EPIC-03
- EPIC-04

EPIC-06 bergantung pada:
- EPIC-05
- EPIC-07
- EPIC-09

EPIC-07 bergantung pada:
```

```
- EPIC-01

EPIC-08 bergantung pada:
- EPIC-01

EPIC-09 bergantung pada:
- EPIC-07
- EPIC-08

EPIC-10 bergantung pada:
- EPIC-09
- EPIC-08

EPIC-11 bergantung pada:
- EPIC-10

EPIC-12 bergantung pada:
- EPIC-08

EPIC-13 bergantung pada:
- EPIC-09
- EPIC-12

EPIC-14 bergantung pada:
- EPIC-09
- EPIC-11
- EPIC-13

EPIC-15 bergantung pada:
- EPIC-03
- EPIC-04
- EPIC-07

EPIC-16 berjalan paralel, tapi final release bergantung pada semua
P0.
```

Suggested MVP Cut

Agar implementasi tidak terlalu melebar, gunakan prioritas berikut.

MVP Must-Have / P0

```
None
EPIC-01 Architecture, Repository, and Engineering Foundation
EPIC-02 Edge Device Sensor Runtime
EPIC-03 IoT Gateway Device Provisioning and Device Token Security
EPIC-04 IoT Gateway Room Sync and Device Binding
EPIC-05 IoT Gateway Telemetry Ingestion and State Management
EPIC-06 IoT Gateway Webhook Outbox, Retry, and JWT Client
EPIC-07 Booking Backend Gateway Registration and JWT Auth
EPIC-08 Booking Backend Room and Booking Domain Foundation
EPIC-09 Booking Backend Occupancy Webhook and Current State
EPIC-10 Historical Occupancy Data and Analytics Aggregation
EPIC-12 Google Calendar Integration
EPIC-13 Auto-Release and Ad-Hoc Booking Logic
EPIC-16 Observability, Security Hardening, QA, and Deployment
```

MVP Should-Have / P1

```
None
EPIC-11 Analytics API and Reporting
EPIC-14 Real-Time Updates and Frontend Dashboard
EPIC-15 Admin Operations, Rotation, and Management UI
```

Post-MVP / P2

```
None
Advanced analytics
Data retention automation
Advanced outbox monitoring
Advanced sensor calibration UI
Multi-gateway load balancing
```

```
Advanced alerting
```

Definition of Done untuk Semua User Story

Tempelkan ini sebagai standard DoD di OpenProject project wiki atau template work package.

```
None
A user story dianggap Done jika:

1. Acceptance criteria terpenuhi.
2. Unit test dibuat untuk logic penting.
3. Integration test dibuat untuk endpoint/API yang kritikal.
4. Swagger/OpenAPI diperbarui jika ada perubahan API.
5. Migration database dibuat jika ada perubahan schema.
6. Error response mengikuti standar errorCode/message/traceId.
7. Logging tidak membocorkan token, secret, atau credential.
8. Semua timestamp menggunakan UTC.
9. Code sudah melewati review.
10. Pipeline build dan test berhasil.
11. Qodana/static analysis tidak memiliki critical issue.
12. Dokumentasi konfigurasi environment diperbarui.
```

OpenProject Import Naming Convention

Gunakan subject seperti ini supaya mudah dicari:

```
None
[EPIC-01] Architecture, Repository, and Engineering Foundation
[US-01.01] Setup solution structure untuk semua service
[US-01.02] Define API response and error response standard
```

Untuk setiap User Story:

```
Type: User Story
Parent: [EPIC-xx] Nama Epic
Priority: P0/P1/P2
Component: Edge Device / IoT Gateway / Booking Backend / Frontend / DevOps
Version: MVP-x
Story Points: 1/2/3/5/8/13
```

Struktur ini sudah cukup untuk developer mulai eksekusi dari OpenProject: setiap Epic punya scope jelas, setiap User Story punya acceptance criteria, dan dependency antar modul sudah terlihat.

Urutan Besar Pengerjaan

```
None
PHASE 0  – Project Setup & OpenProject Setup
PHASE 1  – Engineering Foundation
PHASE 2  – Booking Backend Foundation: Gateway Auth + Room +
Booking
PHASE 3  – IoT Gateway Foundation: Device + Token + SQLite
PHASE 4  – IoT Gateway Room Sync + Device Binding
PHASE 5  – Edge Device Runtime
PHASE 6  – IoT Gateway Telemetry Ingestion
PHASE 7  – Booking Backend Occupancy Webhook + Current State
PHASE 8  – IoT Gateway Outbox + Webhook Dispatch
PHASE 9  – End-to-End Occupancy MVP Integration
PHASE 10 – Historical Analytics Core
PHASE 11 – Analytics API
PHASE 12 – Google Calendar Integration
PHASE 13 – Auto-Release + Ad-Hoc Booking
PHASE 14 – SignalR + Frontend Dashboard
PHASE 15 – Admin UI + Operations UI
PHASE 16 – Observability, QA, Security, Deployment
```

Critical Path

Ini jalur yang **jangan dibalik**:

```
None
1. EPIC-01 Foundation
2. EPIC-07 Booking Backend Gateway Auth
3. EPIC-08 Booking Backend Room Domain
4. EPIC-03 IoT Gateway Device Provisioning
5. EPIC-04 IoT Gateway Room Sync + Binding
6. EPIC-05 IoT Gateway Telemetry Ingestion
```

```
7. EPIC-09 Booking Backend Occupancy Webhook
8. EPIC-06 IoT Gateway Outbox + Dispatch
9. End-to-End Edge → Gateway → Booking Backend
10. EPIC-10 Historical Aggregation
11. EPIC-11 Analytics API
12. EPIC-12 Google Calendar
13. EPIC-13 Auto-Release + Ad-Hoc Booking
14. EPIC-14 Dashboard
15. EPIC-15 Admin UI
16. EPIC-16 QA + Release
```

PHASE 0 — OpenProject Setup

Tujuan: setup project management dulu supaya semua task bisa dimasukkan dan di-track dengan rapi.

Order	Task	Parent	Output
0.1	Buat project OpenProject untuk Smart Meeting Room Occupancy System	Project Setup	Project tersedia
0.2	Buat Work Package Type: Epic, User Story, Task, Bug, Spike, Technical Debt	Project Setup	Type tersedia
0.3	Buat workflow status: New, Ready for Refinement, Ready for Development, In Progress, Code Review, Ready for QA, QA In Progress, Done, Blocked	Project Setup	Workflow siap
0.4	Buat custom field: Component, Priority, Release/Version, Story Points, Risk Level, API Endpoint, Data Store	Project Setup	Field tersedia
0.5	Input semua Epic dari backlog	All Epics	Epic masuk OpenProject
0.6	Input semua User Story sebagai child dari Epic	All User Stories	Story masuk OpenProject

0.7	Tambahkan relasi dependency/blocker antar Epic	Dependency Map	Dependency jelas
0.8	Buat wiki “Definition of Done”	Project Setup	DoD tersedia
0.9	Buat milestone/version: MVP-0 sampai MVP-5	Project Setup	Release plan tersedia

---

PHASE 1 — Engineering Foundation

Parent Epic: EPIC-01
Wajib selesai sebelum implementasi fitur besar.

1.1 Repository & Solution Structure

Order	Task	Parent User Story	Component	Depends On
1.1.1	Buat repository utama atau mono-repo sesuai keputusan tim	US-01.01	Cross-cutting	0.9
1.1.2	Buat solution untuk Booking Backend	US-01.01	Booking Backend	1.1.1
1.1.3	Buat solution untuk IoT Gateway Backend	US-01.01	IoT Gateway	1.1.1
1.1.4	Buat folder/project: Api, Application, Domain, Infrastructure, Tests untuk Booking Backend	US-01.01	Booking Backend	1.1.2
1.1.5	Buat folder/project: Api, Application, Domain, Infrastructure, Tests untuk IoT Gateway	US-01.01	IoT Gateway	1.1.3
1.1.6	Buat struktur project Edge Device runtime	US-02.01	Edge Device	1.1.1
1.1.7	Tambahkan README awal untuk setup local	US-01.05	Cross-cutting	1.1.2

1.2 Common API Standard

Order	Task	Parent User Story	Component	Depends On
1.2.1	Definisikan standard success response	US-01.02	Cross-cutting	1.1
1.2.2	Definisikan standard error response: errorCode, message, details, traceId	US-01.02	Cross-cutting	1.2.1
1.2.3	Implement global exception middleware di Booking Backend	US-01.02	Booking Backend	1.2.2
1.2.4	Implement global exception middleware di IoT Gateway	US-01.02	IoT Gateway	1.2.2
1.2.5	Buat error code enum/list untuk Booking Backend	US-01.02	Booking Backend	1.2.3
1.2.6	Buat error code enum/list untuk IoT Gateway	US-01.02	IoT Gateway	1.2.4

1.3 UTC & Validation Standard

Order	Task	Parent User Story	Component	Depends On
1.3.1	Tentukan semua datetime API wajib ISO-8601 UTC	US-01.03	Cross-cutting	1.2
1.3.2	Implement UTC datetime validator di Booking Backend	US-01.03	Booking Backend	1.3.1
1.3.3	Implement UTC datetime validator di IoT Gateway	US-01.03	IoT Gateway	1.3.1
1.3.4	Buat unit test untuk valid/invalid UTC datetime	US-01.03	QA	1.3.2

1.4 Swagger/OpenAPI

Order	Task	Parent User Story	Component	Depends On
1.4.1	Setup Swagger di Booking Backend	US-01.04	Booking Backend	1.2

1.4.2	Setup Swagger di IoT Gateway	US-01.04	IoT Gateway	1.2
1.4.3	Tambahkan Bearer auth config di Swagger Booking Backend	US-01.04	Booking Backend	1.4.1
1.4.4	Tambahkan Bearer auth config di Swagger IoT Gateway	US-01.04	IoT Gateway	1.4.2

1.5 Environment Config

Order	Task	Parent User Story	Component	Depends On
1.5.1	Definisikan env variable Booking Backend	US-01.05	Booking Backend	1.1
1.5.2	Definisikan env variable IoT Gateway	US-01.05	IoT Gateway	1.1
1.5.3	Definisikan env/local config Edge Device	US-02.01	Edge Device	1.1
1.5.4	Implement config validation saat service startup	US-01.05	Cross-cutting	1.5.1
1.5.5	Pastikan secret tidak muncul di log startup	US-01.05	Cross-cutting	1.5.4

1.6 CI/CD & Static Analysis

Order	Task	Parent User Story	Component	Depends On
1.6.1	Setup Jenkins pipeline restore/build/test untuk Booking Backend	US-01.06	DevOps	1.1
1.6.2	Setup Jenkins pipeline restore/build/test untuk IoT Gateway	US-01.06	DevOps	1.1
1.6.3	Setup test runner untuk Edge Device	US-01.06	DevOps	1.1
1.6.4	Tambahkan Qodana/static analysis untuk Booking Backend	US-01.07	DevOps	1.6.1

1.6.5	Tambahkan Qodana/static analysis untuk IoT Gateway	US-01.07	DevOps	1.6.2
1.6.6	Set pipeline fail kalau test/static analysis critical gagal	US-01.07	DevOps	1.6.4

Gate Phase 1 selesai kalau:

```
None
- Booking Backend bisa run local
- IoT Gateway bisa run local
- Swagger aktif di dua backend
- Error format sudah standar
- UTC validator tersedia
- CI pipeline minimal jalan
```

PHASE 2 — Booking Backend Foundation: Gateway Auth + Room + Booking

Parent Epic: EPIC-07 dan EPIC-08
Kenapa dikerjakan dulu: IoT Gateway butuh `/api/auth/gateway/token` dan `/api/rooms/active` sebelum room sync/binding bisa berjalan.

2.1 Booking Backend Database Foundation

Order	Task	Parent User Story	Component	Depends On
2.1.1	Setup RDBMS connection PostgreSQL/SQL Server	US-08.01	Booking Backend	1.5
2.1.2	Setup EF Core migration infrastructure	US-08.01	Booking Backend	2.1.1
2.1.3	Buat base entity fields: Id, CreatedAtUtc, UpdatedAtUtc	US-08.01	Booking Backend	2.1.2

2.1.4	Buat repository/service pattern baseline	US-08.01	Booking Backend	2.1.2

2.2 IoT Gateway Client Auth

Order	Task	Parent User Story	Component	Depends On
2.2.1	Buat entity `IoTGatewayClient`	US-07.01	Booking Backend	2.1
2.2.2	Buat migration tabel `IoTGatewayClient`	US-07.01	Booking Backend	2.2.1
2.2.3	Tambahkan unique constraint `ClientId`	US-07.01	Booking Backend	2.2.2
2.2.4	Buat service generate `clientId` dan `clientSecret`	US-07.03	Booking Backend	2.2.1
2.2.5	Buat service hash clientSecret	US-07.03	Booking Backend	2.2.4
2.2.6	Implement `POST /api/admin/iot-gateways`	US-07.02	Booking Backend	2.2.5
2.2.7	Pastikan plaintext clientSecret hanya tampil sekali	US-07.02	Booking Backend	2.2.6
2.2.8	Buat unit test client secret generation dan hash	US-07.03	QA	2.2.5

2.3 Gateway JWT Token Endpoint

Order	Task	Parent User Story	Component	Depends On
2.3.1	Implement request DTO untuk `/api/auth/gateway/token`	US-07.04	Booking Backend	2.2
2.3.2	Implement client credential validation	US-07.04	Booking Backend	2.2

2.3.3	Implement JWT generation service	US-07.05	Booking Backend	2.3.2
2.3.4	Tambahkan claims: iss, aud, sub, gatewayId, clientId, scope, iat, exp, jti	US-07.05	Booking Backend	2.3.3
2.3.5	Set TTL JWT 15 menit	US-07.05	Booking Backend	2.3.3
2.3.6	Implement `POST /api/auth/gateway/token`	US-07.04	Booking Backend	2.3.5
2.3.7	Implement JWT validation middleware	US-07.06	Booking Backend	2.3.6
2.3.8	Implement scope-based authorization policy	US-07.06	Booking Backend	2.3.7
2.3.9	Tambahkan test invalid client, revoked gateway, invalid scope	US-07.04/07.06	QA	2.3.8

2.4 Room Domain

Order	Task	Parent User Story	Component	Depends On
2.4.1	Buat entity `Room`	US-08.01	Booking Backend	2.1
2.4.2	Buat migration tabel `Room`	US-08.01	Booking Backend	2.4.1
2.4.3	Tambahkan unique constraint `GoogleCalendarResourceId`	US-08.01	Booking Backend	2.4.2
2.4.4	Implement RoomRepository	US-08.01	Booking Backend	2.4.2
2.4.5	Implement RoomService create/update/deactivate	US-08.02	Booking Backend	2.4.4
2.4.6	Implement seed/manual create room untuk dev environment	US-08.02	Booking Backend	2.4.5
2.4.7	Implement `GET /api/rooms/active`	US-08.03	Booking Backend	2.3.8 + 2.4.5

2.4.8	Protect `GET /api/rooms/active` dengan scope `rooms.read`	US-08.03	Booking Backend	2.4.7
2.4.9	Tambahkan Swagger dan test untuk active rooms	US-08.03	QA	2.4.8

2.5 Booking Domain Foundation

Order	Task	Parent User Story	Component	Depends On
2.5.1	Buat entity `Booking`	US-08.04	Booking Backend	2.4
2.5.2	Buat migration tabel `Booking`	US-08.04	Booking Backend	2.5.1
2.5.3	Buat index `RoomId + StartUtc + EndUtc`	US-08.04	Booking Backend	2.5.2
2.5.4	Implement BookingRepository	US-08.04	Booking Backend	2.5.2
2.5.5	Implement booking overlap validation	US-08.05	Booking Backend	2.5.4
2.5.6	Implement current booking lookup by room/time	US-08.06	Booking Backend	2.5.4
2.5.7	Unit test overlap dan current booking lookup	US-08.05/08.06	QA	2.5.6

Gate Phase 2 selesai kalau:

```
None
- Admin bisa register IoT Gateway
- Booking Backend bisa generate JWT untuk IoT Gateway
- Scope rooms.read dan occupancy.write sudah bisa divalidasi
- Room aktif bisa diambil lewat GET /api/rooms/active
- Booking table dan lookup dasar sudah tersedia
```

PHASE 3 — IoT Gateway Foundation: Device + Token + SQLite

Parent Epic: EPIC-03

3.1 IoT Gateway SQLite Foundation

Order	Task	Parent User Story	Component	Depends On
3.1.1	Setup SQLite connection di IoT Gateway	US-03.01	IoT Gateway	1.5
3.1.2	Setup EF Core migration untuk SQLite	US-03.01	IoT Gateway	3.1.1
3.1.3	Buat entity `Device`	US-03.01	IoT Gateway	3.1.2
3.1.4	Buat migration tabel `Device`	US-03.01	IoT Gateway	3.1.3
3.1.5	Buat entity `DeviceCredential`	US-03.02	IoT Gateway	3.1.3
3.1.6	Buat migration tabel `DeviceCredential`	US-03.02	IoT Gateway	3.1.5
3.1.7	Tambahkan unique constraint `DeviceCredential.TokenHash`	US-03.02	IoT Gateway	3.1.6

3.2 Device Token Service

Order	Task	Parent User Story	Component	Depends On
3.2.1	Implement random token generator format `dev_{base64url_random_32_bytes}`	US-03.04	IoT Gateway	3.1
3.2.2	Implement token hash dengan server-side pepper	US-03.04	IoT Gateway	3.2.1

3.2.3	Implement constant-time hash comparison	US-03.06	IoT Gateway	3.2.2
3.2.4	Unit test token format, hash, dan comparison	US-03.04/03.06	QA	3.2.3

3.3 Device Provisioning API

Order	Task	Parent User Story	Component	Depends On
3.3.1	Implement DTO request/response `POST /api/devices`	US-03.03	IoT Gateway	3.2
3.3.2	Implement DeviceService create device	US-03.03	IoT Gateway	3.2
3.3.3	Simpan Device dan DeviceCredential hash ke SQLite	US-03.03	IoT Gateway	3.3.2
3.3.4	Return plaintext DeviceToken sekali saja	US-03.03	IoT Gateway	3.3.3
3.3.5	Tambahkan validasi request dan error 400	US-03.03	IoT Gateway	3.3.4
3.3.6	Tambahkan Swagger untuk `POST /api/devices`	US-03.03	IoT Gateway	3.3.5
3.3.7	Integration test create device	US-03.03	QA	3.3.6

3.4 Device Token Validation

Order	Task	Parent User Story	Component	Depends On
3.4.1	Implement Authorization header parser	US-03.05	IoT Gateway	3.2
3.4.2	Implement DeviceToken validation middleware/service	US-03.05	IoT Gateway	3.4.1
3.4.3	Return 401 untuk missing/invalid token	US-03.05	IoT Gateway	3.4.2

3.4.4	Return 401 untuk revoked token	US-03.05	IoT Gateway	3.4.2
3.4.5	Implement device active/inactive status check	US-03.08	IoT Gateway	3.4.2
3.4.6	Return 409 untuk inactive device	US-03.08	IoT Gateway	3.4.5
3.4.7	Test valid token, invalid token, revoked token, inactive device	US-03.05/03.08	QA	3.4.6

3.5 Device Token Rotation

Order	Task	Parent User Story	Component	Depends On
3.5.1	Implement `POST /api/devices/{deviceId}/tokens/rotate`	US-03.07	IoT Gateway	3.4
3.5.2	Revoke token lama saat token baru dibuat	US-03.07	IoT Gateway	3.5.1
3.5.3	Return plaintext token baru sekali saja	US-03.07	IoT Gateway	3.5.2
3.5.4	Test token lama ditolak setelah rotation	US-03.07	QA	3.5.3

Gate Phase 3 selesai kalau:

```
None
- Device bisa dibuat
- DeviceToken secure dihasilkan
- Token hash tersimpan
- Plaintext token tidak disimpan
- Token invalid/revoked/inactive ditolak
- Token rotation berjalan
```

PHASE 4 — IoT Gateway Room Sync + Device Binding

Parent Epic: EPIC-04
Depends on: Phase 2 dan Phase 3

4.1 Booking Backend Client Config di IoT Gateway

Order	Task	Parent User Story	Component	Depends On
4.1.1	Tambahkan env config `BOOKING_BACKEND_BASE_URL`	US-04.01	IoT Gateway	1.5
4.1.2	Tambahkan env config `GATEWAY_ID`, `GATEWAY_CLIENT_ID`, `GATEWAY_CLIENT_SECRET`	US-04.01	IoT Gateway	4.1.1
4.1.3	Implement config validation untuk credential Booking Backend	US-04.01	IoT Gateway	4.1.2
4.1.4	Pastikan client secret tidak dilog	US-04.01	IoT Gateway	4.1.3

4.2 JWT Client dari IoT Gateway ke Booking Backend

Order	Task	Parent User Story	Component	Depends On
4.2.1	Buat client untuk `POST /api/auth/gateway/token`	US-06.03	IoT Gateway	2.3 + 4.1
4.2.2	Simpan JWT di IMemoryCache/local memory	US-06.03	IoT Gateway	4.2.1
4.2.3	Implement refresh token 60 detik sebelum expired	US-06.04	IoT Gateway	4.2.2
4.2.4	Handle invalid client/expired/retry token request	US-06.03/06.04	IoT Gateway	4.2.3

4.2.5	Integration test IoT Gateway request JWT ke Booking Backend	US-06.03	QA	4.2.4

4.3 Active Rooms Client

Order	Task	Parent User Story	Component	Depends On
4.3.1	Buat client untuk `GET /api/rooms/active`	US-04.02	IoT Gateway	4.2
4.3.2	Tambahkan Authorization Bearer GatewayAccessToken	US-04.02	IoT Gateway	4.3.1
4.3.3	Handle token expired dengan refresh lalu retry	US-04.02	IoT Gateway	4.3.2
4.3.4	Return daftar room aktif ke layer admin/dashboard IoT Gateway	US-04.02	IoT Gateway	4.3.3
4.3.5	Integration test fetch active rooms	US-04.02	QA	4.3.4

4.4 RoomDeviceMapping

Order	Task	Parent User Story	Component	Depends On
4.4.1	Buat entity `RoomDeviceMapping`	US-04.03	IoT Gateway	3.1
4.4.2	Buat migration tabel `RoomDeviceMapping`	US-04.03	IoT Gateway	4.4.1
4.4.3	Tambahkan unique active binding per device	US-04.03	IoT Gateway	4.4.2
4.4.4	Implement repository mapping	US-04.03	IoT Gateway	4.4.2

4.5 Device Binding API

Order	Task	Parent User Story	Component	Depends On

4.5.1	Implement DTO `POST /api/devices/{deviceId}/bind`	US-04.04	IoT Gateway	4.4
4.5.2	Validasi device exists dan active	US-04.04	IoT Gateway	3.4 + 4.5.1
4.5.3	Simpan mapping Device → Room ke SQLite	US-04.04	IoT Gateway	4.5.2
4.5.4	Refresh `DeviceRoomMapping:{deviceId}` di IMemoryCache	US-04.05	IoT Gateway	4.5.3
4.5.5	Implement fallback cache miss ke SQLite	US-04.05	IoT Gateway	4.5.4
4.5.6	Implement validation device bound sebelum telemetry	US-04.06	IoT Gateway	4.5.5
4.5.7	Return 404 `DEVICE_NOT_BOUND` kalau belum bind	US-04.06	IoT Gateway	4.5.6
4.5.8	Implement re-binding device ke room lain	US-04.07	IoT Gateway	4.5.5
4.5.9	Audit log sederhana untuk re-binding	US-04.07	IoT Gateway	4.5.8
4.5.10	Integration test create device → fetch rooms → bind device	US-04.04	QA	4.5.9

Gate Phase 4 selesai kalau:

```
None
- IoT Gateway bisa request JWT ke Booking Backend
- IoT Gateway bisa fetch active rooms
- Device bisa dibind ke room
- Cache mapping aktif
- Telemetry dari unbound device ditolak
```

PHASE 5 — Edge Device Runtime

Parent Epic: EPIC-02
Bisa mulai paralel setelah API contract `/api/telemetry` disepakati, tapi final integration menunggu Phase 6.

5.1 Edge Config & Startup

Order	Task	Parent User Story	Component	Depends On
5.1.1	Buat config loader untuk `DEVICE_ID`	US-02.01	Edge Device	1.5
5.1.2	Buat config loader untuk `DEVICE_TOKEN`	US-02.01	Edge Device	5.1.1
5.1.3	Buat config loader untuk `IOT_GATEWAY_BASE_URL`	US-02.01	Edge Device	5.1.2
5.1.4	Buat config loader untuk `TELEMETRY_POST_INTERVAL_MS`	US-02.01	Edge Device	5.1.3
5.1.5	Buat config loader untuk `UART_PORT` dan `UART_BAUD_RATE`	US-02.01	Edge Device	5.1.4
5.1.6	Implement startup validation	US-02.01	Edge Device	5.1.5

5.2 UART & Sensor Parser

Order	Task	Parent User Story	Component	Depends On
5.2.1	Initialize UART connection ke HLK-LD2410B	US-02.02	Edge Device	5.1
5.2.2	Implement retry kalau UART belum tersedia	US-02.02	Edge Device	5.2.1
5.2.3	Implement byte stream reader	US-02.02	Edge Device	5.2.1
5.2.4	Implement parser frame serial HLK-LD2410B	US-02.03	Edge Device	5.2.3

5.2.5	Extract `targetState` dari frame	US-02.03	Edge Device	5.2.4
5.2.6	Extract `movingTargetDistance` dan `stationaryTargetDistance`	US-02.03	Edge Device	5.2.4
5.2.7	Handle corrupt frame tanpa crash	US-02.03	Edge Device	5.2.4
5.2.8	Unit test parser dengan sample frame	US-02.03	QA	5.2.7

5.3 Occupancy Logic di Edge

Order	Task	Parent User Story	Component	Depends On
5.3.1	Map `NoTarget` → `isOccupied=false`	US-02.04	Edge Device	5.2
5.3.2	Map `MovingTarget` → `isOccupied=true`	US-02.04	Edge Device	5.3.1
5.3.3	Map `StationaryTarget` → `isOccupied=true`	US-02.04	Edge Device	5.3.2
5.3.4	Map `MovingAndStationaryTarget` → `isOccupied=true`	US-02.04	Edge Device	5.3.3
5.3.5	Implement debounce 30 detik untuk Occupied → Empty	US-02.05	Edge Device	5.3.4
5.3.6	Cancel debounce kalau target muncul lagi	US-02.05	Edge Device	5.3.5
5.3.7	Unit test mapping dan debounce	US-02.04/02.05	QA	5.3.6

5.4 Telemetry Sender

Order	Task	Parent User Story	Component	Depends On
5.4.1	Generate `readingId` GUID untuk setiap reading	US-02.06	Edge Device	5.3

5.4.2	Build telemetry payload sesuai contract	US-02.07	Edge Device	5.4.1
5.4.3	Implement HTTP client dengan Keep-Alive	US-02.07	Edge Device	5.4.2
5.4.4	Tambahkan header `Authorization: Bearer {DeviceToken}`	US-02.07	Edge Device	5.4.3
5.4.5	Implement POST `/api/telemetry`	US-02.07	Edge Device	5.4.4
5.4.6	Implement lightweight retry untuk timeout/network error	US-02.08	Edge Device	5.4.5
5.4.7	Jangan retry terus untuk 401/404	US-02.08	Edge Device	5.4.6
5.4.8	Add logging tanpa token	US-02.09	Edge Device	5.4.7
5.4.9	Buat simulator mode untuk development tanpa sensor fisik	Technical Task	Edge Device	5.4.8

Gate Phase 5 selesai kalau:

```
None
- Edge bisa membaca config
- Edge bisa parsing sensor/simulator
- Edge bisa menghasilkan isOccupied
- Debounce bekerja
- Edge bisa kirim telemetry dengan DeviceToken
```

PHASE 6 — IoT Gateway Telemetry Ingestion

Parent Epic: EPIC-05
Depends on: Phase 3 dan Phase 4

6.1 Telemetry Endpoint

Order	Task	Parent User Story	Component	Depends On
6.1.1	Implement request DTO `/api/telemetry`	US-05.01	IoT Gateway	4.5
6.1.2	Implement `POST /api/telemetry` controller	US-05.01	IoT Gateway	6.1.1
6.1.3	Integrasikan DeviceToken validation ke endpoint	US-05.01	IoT Gateway	3.4
6.1.4	Return 401 kalau token invalid	US-05.01	IoT Gateway	6.1.3

6.2 Payload Validation

Order	Task	Parent User Story	Component	Depends On
6.2.1	Validasi `deviceId` GUID	US-05.02	IoT Gateway	6.1
6.2.2	Validasi `readingId` GUID	US-05.02	IoT Gateway	6.2.1
6.2.3	Validasi `timestamp` UTC	US-05.02	IoT Gateway	6.2.2
6.2.4	Validasi timestamp skew ±2 menit	US-05.02	IoT Gateway	6.2.3
6.2.5	Validasi `isOccupied` wajib boolean	US-05.02	IoT Gateway	6.2.4
6.2.6	Return error 400 sesuai standard	US-05.02	IoT Gateway	6.2.5

6.3 Bound Device Validation

Order	Task	Parent User Story	Component	Depends On
6.3.1	Ambil mapping dari `DeviceRoomMapping:{deviceId}`	US-04.06	IoT Gateway	4.5

6.3.2	Fallback ke SQLite kalau cache miss	US-04.06	IoT Gateway	6.3.1
6.3.3	Return 404 `DEVICE_NOT_BOUND` kalau tidak ada mapping aktif	US-04.06	IoT Gateway	6.3.2

6.4 IMemoryCache State Management

Order	Task	Parent User Story	Component	Depends On
6.4.1	Definisikan cache key `DeviceState:{deviceId}`	US-05.03	IoT Gateway	6.3
6.4.2	Definisikan cache key `RoomState:{roomId}`	US-05.03	IoT Gateway	6.4.1
6.4.3	Implement initial state kalau cache kosong	US-05.03	IoT Gateway	6.4.2
6.4.4	Update `lastReadingAt` setiap telemetry valid	US-05.03	IoT Gateway	6.4.3
6.4.5	Update `lastChangedAt` hanya saat state berubah	US-05.03	IoT Gateway	6.4.4

6.5 State Change Detection

Order	Task	Parent User Story	Component	Depends On
6.5.1	Bandingkan incoming `isOccupied` dengan previous state	US-05.04	IoT Gateway	6.4
6.5.2	Jika state sama, return `stateChanged=false`	US-05.04/05.06	IoT Gateway	6.5.1
6.5.3	Pastikan state sama tidak tulis DB dan tidak call webhook	US-05.04	IoT Gateway	6.5.2
6.5.4	Jika state berubah, generate `eventId` GUID	US-05.05	IoT Gateway	6.5.1

6.5.5	Build occupancy event object `Transition`	US-05.05	IoT Gateway	6.5.4
6.5.6	Return `stateChanged=true` dan `generatedEventId`	US-05.06	IoT Gateway	6.5.5
6.5.7	Implement `Snapshot` event setelah gateway restart/cache kosong	US-05.07	IoT Gateway	6.4

Gate Phase 6 selesai kalau:

```
None
- Edge/simulator bisa POST telemetry ke IoT Gateway
- Token valid diterima, token invalid ditolak
- Unbound device ditolak
- Telemetry unchanged tidak menulis DB
- Telemetry changed menghasilkan eventId
```

PHASE 7 — Booking Backend Occupancy Webhook + Current State

Parent Epic: EPIC-09
Depends on: Phase 2

7.1 Occupancy Tables

Order	Task	Parent User Story	Component	Depends On
7.1.1	Buat entity `RoomCurrentOccupancy`	US-09.04	Booking Backend	2.4
7.1.2	Buat migration tabel `RoomCurrentOccupancy`	US-09.04	Booking Backend	7.1.1
7.1.3	Tambahkan unique constraint `RoomId`	US-09.04	Booking Backend	7.1.2

7.1.4	Buat entity `RoomOccupancyEvent`	US-09.06 / US-10.01	Booking Backend	7.1.2
7.1.5	Buat migration tabel `RoomOccupancyEvent`	US-09.06 / US-10.01	Booking Backend	7.1.4
7.1.6	Tambahkan unique constraint `SourceEventId`	US-09.03	Booking Backend	7.1.5
7.1.7	Tambahkan index `RoomId + ChangedAtUtc`	US-10.01	Booking Backend	7.1.5

7.2 Webhook Endpoint

Order	Task	Parent User Story	Component	Depends On
7.2.1	Implement DTO request `POST /api/webhooks/occupancy`	US-09.01	Booking Backend	7.1
7.2.2	Implement endpoint controller	US-09.01	Booking Backend	7.2.1
7.2.3	Protect endpoint dengan JWT auth	US-09.01	Booking Backend	2.3
7.2.4	Require scope `occupancy.write`	US-07.06 / US-09.01	Booking Backend	7.2.3
7.2.5	Validate `Idempotency-Key` header optional/required sesuai policy	US-09.03	Booking Backend	7.2.2

7.3 Payload Validation

Order	Task	Parent User Story	Component	Depends On
7.3.1	Validasi `eventId` GUID	US-09.02	Booking Backend	7.2
7.3.2	Validasi `gatewayId` exists dan active	US-09.02	Booking Backend	7.3.1

7.3.3	Validasi `roomId` exists	US-09.02	Booking Backend	7.3.2
7.3.4	Validasi `changedAt` UTC	US-09.02	Booking Backend	7.3.3
7.3.5	Validasi `eventType` Transition/Snapshot	US-09.02	Booking Backend	7.3.4
7.3.6	Return 400/401/403/404 sesuai error rule	US-09.02	Booking Backend	7.3.5

7.4 Idempotency & Transaction

Order	Task	Parent User Story	Component	Depends On
7.4.1	Implement check duplicate by `SourceEventId`	US-09.03	Booking Backend	7.1
7.4.2	Return 200 `duplicate=true` untuk duplicate event	US-09.03	Booking Backend	7.4.1
7.4.3	Implement transaction boundary untuk webhook processing	US-09.08	Booking Backend	7.4.1
7.4.4	Rollback kalau insert/update gagal	US-09.08	Booking Backend	7.4.3

7.5 Current State + Historical Event

Order	Task	Parent User Story	Component	Depends On
7.5.1	Implement upsert `RoomCurrentOccupancy`	US-09.05	Booking Backend	7.4
7.5.2	Jangan overwrite kalau event lebih lama dari current state	US-09.05	Booking Backend	7.5.1
7.5.3	Insert `RoomOccupancyEvent` kalau `eventType=Transition`	US-09.06	Booking Backend	7.5.1

7.5.4	Handle `Snapshot` hanya untuk current state	US-09.05/09.06	Booking Backend	7.5.3
7.5.5	Cari previous event per room	US-09.07	Booking Backend	7.5.3
7.5.6	Update `DurationSeconds` previous event	US-09.07	Booking Backend	7.5.5
7.5.7	Integration test webhook normal, duplicate, older event	US-09.01-09.08	QA	7.5.6

Gate Phase 7 selesai kalau:

```
None
- Booking Backend menerima occupancy webhook dengan JWT valid
- Duplicate event return 200 duplicate=true
- RoomCurrentOccupancy ter-update
- RoomOccupancyEvent tersimpan untuk Transition
- Previous event duration terisi
```

PHASE 8 — IoT Gateway Outbox + Webhook Dispatch

Parent Epic: EPIC-06
Depends on: Phase 6 dan Phase 7

8.1 Outbox Table

Order	Task	Parent User Story	Component	Depends On
8.1.1	Buat entity `GatewayWebhookOutbox`	US-06.01	IoT Gateway	6.5
8.1.2	Buat migration tabel `GatewayWebhookOutbox`	US-06.01	IoT Gateway	8.1.1

8.1.3	Tambahkan unique constraint `EventId`	US-06.01	IoT Gateway	8.1.2
8.1.4	Tambahkan index `Status + NextAttemptAtUtc`	US-06.01	IoT Gateway	8.1.2

8.2 Insert Outbox Saat State Change

Order	Task	Parent User Story	Component	Depends On
8.2.1	Build webhook payload dari transition event	US-06.02	IoT Gateway	6.5
8.2.2	Insert event ke outbox sebelum dispatch	US-06.02	IoT Gateway	8.1
8.2.3	Set status `Pending`, AttemptCount=0	US-06.02	IoT Gateway	8.2.2
8.2.4	Kalau insert outbox gagal, jangan dispatch webhook	US-06.02	IoT Gateway	8.2.2

8.3 Webhook Dispatcher

Order	Task	Parent User Story	Component	Depends On
8.3.1	Implement HTTP client Booking Backend	US-06.05	IoT Gateway	4.2
8.3.2	Tambahkan Authorization Bearer GatewayAccessToken	US-06.05	IoT Gateway	8.3.1
8.3.3	Tambahkan header `Idempotency-Key: {eventId}`	US-06.05	IoT Gateway	8.3.2
8.3.4	Implement POST `/api/webhooks/occupancy`	US-06.05	IoT Gateway	8.3.3
8.3.5	Mark outbox `Delivered` saat response 200	US-06.05	IoT Gateway	8.3.4

8.3.6	Treat `duplicate=true` sebagai success	US-06.08	IoT Gateway	8.3.5

8.4 Retry Policy

Order	Task	Parent User Story	Component	Depends On
8.4.1	Implement Polly retry untuk timeout	US-06.06	IoT Gateway	8.3
8.4.2	Implement Polly retry untuk 5xx	US-06.06	IoT Gateway	8.4.1
8.4.3	Jangan retry immediate untuk 4xx non-retryable	US-06.06	IoT Gateway	8.4.2
8.4.4	Implement background worker pending outbox	US-06.07	IoT Gateway	8.4.3
8.4.5	Hitung NextAttemptAtUtc: 1m, 5m, 15m, 30m	US-06.07	IoT Gateway	8.4.4
8.4.6	Increment AttemptCount dan simpan LastError saat gagal	US-06.07	IoT Gateway	8.4.5
8.4.7	Integration test Booking Backend down → pending → retry → delivered	US-06.07	QA	8.4.6

Gate Phase 8 selesai kalau:

```
None
- Transition event masuk outbox
- Event dikirim ke Booking Backend
- Delivered jika sukses
- Duplicate dianggap sukses
- Kalau Booking Backend down, event tetap Pending dan retry otomatis
```

PHASE 9 — End-to-End Occupancy MVP Integration

Ini adalah milestone terpenting sebelum lanjut analytics/calendar.

Order	Task	Component	Depends On	Output
9.1	Register IoT Gateway di Booking Backend	Booking Backend	Phase 2	gatewayId/clientId/clientSecret
9.2	Configure IoT Gateway dengan credential tersebut	IoT Gateway	9.1	env siap
9.3	Buat room di Booking Backend	Booking Backend	Phase 2	active room
9.4	Buat device di IoT Gateway	IoT Gateway	Phase 3	deviceId/deviceToken
9.5	Configure Edge simulator/device dengan deviceId/deviceToken	Edge Device	Phase 5	Edge siap
9.6	IoT Gateway fetch active rooms	IoT Gateway	Phase 4	room list muncul
9.7	Bind device ke room	IoT Gateway	Phase 4	mapping aktif
9.8	Edge kirim telemetry `isOccupied=false`	Edge + IoT	Phase 6	initial state
9.9	Edge kirim telemetry `isOccupied=true`	Edge + IoT	Phase 8	transition event
9.10	Pastikan outbox Delivered	IoT Gateway	9.9	outbox success
9.11	Pastikan Booking Backend `RoomCurrentOccupancy=true`	Booking Backend	9.10	current state update

9.12	Pastikan `RoomOccupancyEvent` tersimpan	Booking Backend	9.10	historical event
9.13	Kirim duplicate event manual	QA	9.12	duplicate=true
9.14	Matikan Booking Backend lalu kirim transition baru	QA	Phase 8	outbox Pending
9.15	Nyalakan Booking Backend lalu verify retry Delivered	QA	9.14	reliability verified

Gate Phase 9 selesai kalau:

```
Edge → IoT Gateway → Booking Backend berhasil secara end-to-end.
Ini adalah MVP occupancy pipeline.
```

PHASE 10 — Historical Analytics Core

Parent Epic: EPIC-10
Depends on: Phase 7 dan Phase 9

10.1 Aggregate Schema

Order	Task	Parent User Story	Component	Depends On
10.1.1	Buat entity `RoomOccupancyAggregate`	US-10.02	Booking Backend	Phase 7
10.1.2	Buat migration tabel `RoomOccupancyAggregate`	US-10.02	Booking Backend	10.1.1
10.1.3	Tambahkan unique constraint `RoomId + BucketStartUtc + BucketSize`	US-10.02	Booking Backend	10.1.2
10.1.4	Tambahkan enum `BucketSize`: Minute, Hour, Day	US-10.02	Booking Backend	10.1.2

10.2 Calculation Engine

Order	Task	Parent User Story	Component	Depends On
10.2.1	Implement bucket duration helper	US-10.03	Booking Backend	10.1
10.2.2	Implement utilization formula	US-10.03	Booking Backend	10.2.1
10.2.3	Implement interval splitter kalau event melewati bucket boundary	US-10.05	Booking Backend	10.2.1
10.2.4	Hitung occupiedSeconds per bucket	US-10.05	Booking Backend	10.2.3
10.2.5	Hitung emptySeconds per bucket	US-10.05	Booking Backend	10.2.4
10.2.6	Hitung bookingSeconds per bucket	US-10.06	Booking Backend	2.5 + 10.2.3
10.2.7	Unit test formula, interval split, occupied/empty/booking seconds	US-10.03-10.06	QA	10.2.6

10.3 Background Aggregation Job

Order	Task	Parent User Story	Component	Depends On
10.3.1	Implement background job scheduler setiap 5 menit	US-10.04	Booking Backend	10.2
10.3.2	Ambil events terbaru / 24 jam terakhir	US-10.04	Booking Backend	10.3.1
10.3.3	Recalculate bucket terkait	US-10.04	Booking Backend	10.3.2
10.3.4	Upsert aggregate	US-10.04	Booking Backend	10.3.3
10.3.5	Pastikan job idempotent	US-10.07	Booking Backend	10.3.4

10.3.6	Handle late event dari outbox retry dengan recalculation	US-10.07	Booking Backend	10.3.5
10.3.7	Tambahkan retention config, default disabled	US-10.08	Booking Backend	10.3.6

Gate Phase 10 selesai kalau:

```
None
- Event occupancy bisa dihitung menjadi aggregate
- UtilizationPercent benar
- Aggregate bisa di-upsert
- Job aman dijalankan berulang
```

PHASE 11 — Analytics API

Parent Epic: EPIC-11
Depends on: Phase 10

Order	Task	Parent User Story	Component	Depends On
11.1	Implement analytics authorization `analytics.read`	US-11.06	Booking Backend	2.3
11.2	Implement date range validator	US-11.02	Booking Backend	Phase 10
11.3	Implement bucket validator	US-11.02	Booking Backend	11.2
11.4	Implement `GET /api/analytics/rooms/{roomId}/occupancy`	US-11.01	Booking Backend	11.3
11.5	Query `RoomOccupancyAggregate` untuk occupancy chart	US-11.01	Booking Backend	11.4

11.6	Implement `GET /api/analytics/occupancy-summary`	US-11.03	Booking Backend	11.5
11.7	Implement most used / least used ranking	US-11.05	Booking Backend	11.6
11.8	Implement `GET /api/analytics/ghost-meetings`	US-11.04	Booking Backend	Phase 13 nanti, bisa stub dulu
11.9	Update Swagger analytics APIs	US-11.01 -11.04	Booking Backend	11.8
11.10	Integration test analytics APIs	US-11.01 -11.06	QA	11.9

Catatan: `ghost-meetings` akan lengkap setelah `GhostMeetingAudit` dari Phase 13 selesai. Bisa buat endpoint dulu dengan response kosong/stub, lalu finalize di Phase 13.

PHASE 12 — Google Calendar Integration

Parent Epic: EPIC-12
Depends on: Phase 2. Bisa paralel dengan Phase 10/11 setelah Booking domain siap.

Order	Task	Parent User Story	Component	Depends On
12.1	Setup Google OAuth config	US-12.01	Booking Backend	1.5
12.2	Implement OAuth token acquisition	US-12.01	Booking Backend	12.1
12.3	Implement OAuth refresh token handling	US-12.01	Booking Backend	12.2
12.4	Pastikan Google secret tidak dilog	US-12.01	Booking Backend	12.3

12.5	Validasi `googleCalendarResourceId` di Room	US-12.02	Booking Backend	2.4
12.6	Implement Google Calendar push notification receiver	US-12.03	Booking Backend	12.3
12.7	Implement Google event sync worker/service	US-12.04	Booking Backend	12.6
12.8	Sync new Google Calendar event → Booking local	US-12.04	Booking Backend	12.7
12.9	Sync updated Google Calendar event → update Booking local	US-12.04	Booking Backend	12.8
12.10	Sync cancelled Google Calendar event → Booking Cancelled	US-12.04	Booking Backend	12.9
12.11	Implement create Google Calendar event for ad-hoc	US-12.05	Booking Backend	12.10
12.12	Implement cancel Google Calendar event for auto-release	US-12.06	Booking Backend	12.10
12.13	Integration test Google Calendar create/cancel/sync	US-12.01-12.06	QA	12.12

Gate Phase 12 selesai kalau:

```
None
- Booking Backend bisa sync Google Calendar ke Booking table
- Bisa create calendar event untuk ad-hoc
- Bisa cancel calendar event untuk auto-release
```

PHASE 13 — Auto-Release + Ad-Hoc Booking

Parent Epic: EPIC-13
Depends on: Phase 7 dan Phase 12

13.1 Auto-Release

Order	Task	Parent User Story	Component	Depends On
13.1.1	Buat scheduler/checker untuk booking start + 15 menit	US-13.01	Booking Backend	2.5
13.1.2	Pastikan scheduled check survive service restart	US-13.01	Booking Backend	13.1.1
13.1.3	Load `RoomCurrentOccupancy` saat T+15	US-13.02	Booking Backend	Phase 7
13.1.4	Implement rule jika `IsOccupied=false` maka eligible auto-release	US-13.02	Booking Backend	13.1.3
13.1.5	Implement fallback kalau occupancy unknown	US-13.02	Booking Backend	13.1.4
13.1.6	Cancel Google Calendar event	US-12.06 / US-13.03	Booking Backend	Phase 12
13.1.7	Mark Booking status `AutoReleased`	US-13.03	Booking Backend	13.1.6
13.1.8	Buat entity dan migration `GhostMeetingAudit`	US-13.04	Booking Backend	13.1.7
13.1.9	Insert audit record saat auto-release	US-13.04	Booking Backend	13.1.8
13.1.10	Finalize `GET /api/analytics/ghost-meetings`	US-11.04	Booking Backend	13.1.9

13.2 Ad-Hoc Booking

Order	Task	Parent User Story	Component	Depends On
13.2.1	Detect event Empty → Occupied dari webhook processing	US-13.05	Booking Backend	Phase 7
13.2.2	Cek current booking untuk room tersebut	US-13.05	Booking Backend	2.5

13.2.3	Jika no active booking, buat ad-hoc candidate	US-13.05	Booking Backend	13.2.2
13.2.4	Buat timer/check 5 menit	US-13.06	Booking Backend	13.2.3
13.2.5	Setelah 5 menit, cek room masih occupied	US-13.06	Booking Backend	13.2.4
13.2.6	Setelah 5 menit, cek room masih available	US-13.06	Booking Backend	13.2.5
13.2.7	Jika valid, create Google Calendar event 30 menit	US-13.07 / US-12.05	Booking Backend	Phase 12
13.2.8	Buat Booking local dengan status Confirmed	US-13.07	Booking Backend	13.2.7
13.2.9	Cancel candidate jika room empty sebelum 5 menit	US-13.06	Booking Backend	13.2.4
13.2.10	Cancel candidate jika booking lain masuk sebelum 5 menit	US-13.06	Booking Backend	13.2.4
13.2.11	Unit/integration test auto-release dan ad-hoc	US-13.01-13.07	QA	13.2.10

Gate Phase 13 selesai kalau:

```
None
- Meeting kosong sampai T+15 bisa auto-release
- GhostMeetingAudit tersimpan
- Room available yang occupied 5 menit bisa auto-create booking 30 menit
```

PHASE 14 — SignalR + Frontend Dashboard

Parent Epic: EPIC-14
Depends on: Phase 7, Phase 11, Phase 13

14.1 SignalR Backend

Order	Task	Parent User Story	Component	Depends On
14.1	Implement `/hubs/rooms` SignalR hub	US-14.01	Booking Backend	Phase 7
14.2	Publish `RoomOccupancyChanged` setelah webhook sukses	US-14.02	Booking Backend	14.1
14.3	Jangan publish ulang untuk duplicate webhook	US-14.02	Booking Backend	14.2
14.4	Publish `BookingAutoReleased`	US-14.07	Booking Backend	Phase 13
14.5	Publish `AdHocBookingCreated`	US-14.07	Booking Backend	Phase 13
14.6	Publish `RoomAnalyticsUpdated` setelah aggregate job	US-14.01	Booking Backend	Phase 10

14.2 Frontend Dashboard

Order	Task	Parent User Story	Component	Depends On
14.7	Build frontend auth/session baseline	Technical Task	Frontend	1.5
14.8	Build real-time room status page	US-14.03	Frontend	14.1
14.9	Connect frontend ke SignalR hub	US-14.01	Frontend	14.8
14.10	Update room status tanpa reload saat event masuk	US-14.03	Frontend	14.9
14.11	Build occupancy analytics chart	US-14.04	Frontend	Phase 11
14.12	Build occupancy summary dashboard	US-14.05	Frontend	Phase 11
14.13	Build ghost meeting report page	US-14.06	Frontend	Phase 13
14.14	Show auto-release notification	US-14.07	Frontend	14.4
14.15	Show ad-hoc booking notification	US-14.07	Frontend	14.5

14.16	Frontend integration test dashboard flow	US-14.01-14.07	QA	14.15

PHASE 15 — Admin UI + Operations UI

Parent Epic: EPIC-15
Depends on: Phase 2, 3, 4, 8

Order	Task	Parent User Story	Component	Depends On
15.1	Build IoT Gateway registration admin UI	US-15.01	Frontend/Admin	2.2
15.2	Display clientId/clientSecret once	US-15.01	Frontend/Admin	15.1
15.3	Build device provisioning UI	US-15.02	IoT Gateway UI	3.3
15.4	Display DeviceToken once	US-15.02	IoT Gateway UI	15.3
15.5	Build device binding UI	US-15.03	IoT Gateway UI	4.5
15.6	Integrate active rooms dropdown	US-15.03	IoT Gateway UI	4.3
15.7	Build device token rotation UI	US-15.04	IoT Gateway UI	3.5
15.8	Build gateway secret rotation endpoint/UI	US-15.05	Booking Backend + UI	2.3
15.9	Build device operational status page	US-15.06	IoT Gateway UI	Phase 6
15.10	Track `lastReadingAt` per device	US-15.06	IoT Gateway	Phase 6
15.11	Mark device potentially offline by threshold	US-15.06	IoT Gateway UI	15.10
15.12	Build outbox monitoring page	US-15.07	IoT Gateway UI	Phase 8
15.13	Display pending/delivered/failed outbox status	US-15.07	IoT Gateway UI	15.12
15.14	Admin UI E2E test	EPIC-15	QA	15.13

PHASE 16 — Observability, QA, Security, Deployment

Parent Epic: EPIC-16
Sebagian dikerjakan paralel dari awal, tapi final gate dikerjakan setelah feature utama selesai.

16.1 Observability

Order	Task	Parent User Story	Component	Depends On
16.1.1	Add structured logging Booking Backend	US-16.01	Booking Backend	Phase 1
16.1.2	Add structured logging IoT Gateway	US-16.01	IoT Gateway	Phase 1
16.1.3	Add structured logging Edge Device	US-16.01	Edge Device	Phase 5
16.1.4	Add traceId/correlationId antar request	US-16.01	Cross-cutting	16.1.1
16.1.5	Mask token/secret di semua log	US-16.01	Cross-cutting	16.1.4
16.1.6	Add `/health/live` dan `/health/ready` Booking Backend	US-16.02	Booking Backend	Phase 2
16.1.7	Add `/health/live` dan `/health/ready` IoT Gateway	US-16.02	IoT Gateway	Phase 3
16.1.8	Add telemetry/webhook metrics counters	US-16.03	Cross-cutting	Phase 8

16.2 Security Hardening

Order	Task	Parent User Story	Component	Depends On
16.2.1	Enforce HTTPS di production config	US-16.04	DevOps	Phase 1
16.2.2	Validate JWT issuer/audience/signature/expiry	US-16.04	Booking Backend	Phase 2
16.2.3	Validate scope di endpoint protected	US-16.04	Booking Backend	Phase 2

16.2.4	Validate revoked device token	US-16.04	IoT Gateway	Phase 3
16.2.5	Validate revoked gateway secret	US-16.04	Booking Backend	Phase 2
16.2.6	Security test token leakage in logs	US-16.04	QA	16.2.5

16.3 Testing

Order	Task	Parent User Story	Component	Depends On
16.3.1	Unit test Edge debounce	US-16.05	QA	Phase 5
16.3.2	Unit test Booking Backend idempotency	US-16.05	QA	Phase 7
16.3.3	Unit test utilization formula	US-16.05	QA	Phase 10
16.3.4	Integration test Edge → Gateway telemetry	US-16.06	QA	Phase 6
16.3.5	Integration test Gateway → Booking webhook	US-16.06	QA	Phase 8
16.3.6	Integration test duplicate webhook	US-16.06	QA	Phase 7
16.3.7	Failure test Booking Backend down → outbox pending	US-16.07	QA	Phase 8
16.3.8	Failure test Booking Backend up → retry delivered	US-16.07	QA	Phase 8
16.3.9	E2E test auto-release	US-16.06	QA	Phase 13
16.3.10	E2E test ad-hoc booking	US-16.06	QA	Phase 13
16.3.11	E2E dashboard test	US-16.06	QA	Phase 14

16.4 Deployment

Order	Task	Parent User Story	Component	Depends On
16.4.1	Buat deployment script Booking Backend	US-16.08	DevOps	Phase 2

16.4.2	Buat migration run script Booking Backend	US-16.08	DevOps	16.4.1
16.4.3	Buat deployment script IoT Gateway	US-16.08	DevOps	Phase 3
16.4.4	Buat migration run script IoT Gateway SQLite	US-16.08	DevOps	16.4.3
16.4.5	Buat deployment guide Edge Device Raspberry Pi	US-16.08	DevOps/Edge	Phase 5
16.4.6	Buat environment variable checklist	US-16.08	DevOps	16.4.5
16.4.7	Dry-run deployment ke dev/staging	US-16.08	DevOps	16.4.6
16.4.8	Prepare MVP release checklist	US-16.09	Product/QA	all P0
16.4.9	UAT sign-off	US-16.09	Product/QA	16.4.8
16.4.10	Production release	US-16.09	DevOps	16.4.9

Urutan Sprint yang Disarankan

Kalau dibuat sprint 2 mingguan, susun seperti ini.

Sprint 0 — Planning & Foundation

```
None
- PHASE 0
- PHASE 1
```

Deliverable:

```
None
Project OpenProject siap
Repo siap
Solution siap
Swagger baseline
CI baseline
Error response standard
```

```
UTC standard
```

Sprint 1 — Booking Backend Auth + Room Foundation

```
None
- PHASE 2.1
- PHASE 2.2
- PHASE 2.3
- PHASE 2.4
```

Deliverable:

```
None
Booking Backend bisa:
- register IoT Gateway
- generate JWT
- validate scope
- expose active rooms
```

Sprint 2 — IoT Gateway Device + Binding

```
None
- PHASE 3
- PHASE 4
```

Deliverable:

```
None
IoT Gateway bisa:
- create device
- generate DeviceToken
- validate DeviceToken
```

```
- fetch active rooms
- bind device ke room
```

Sprint 3 — Edge + Telemetry Ingestion

```
None
- PHASE 5
- PHASE 6
```

Deliverable:

```
None
Edge/simulator bisa:
- parse/mapping occupancy
- debounce
- POST telemetry

IoT Gateway bisa:
- receive telemetry
- validate token
- detect state change
```

Sprint 4 — Booking Occupancy + Outbox Reliability

```
None
- PHASE 7
- PHASE 8
- PHASE 9
```

Deliverable:

```
None
End-to-end occupancy pipeline selesai:
Edge → IoT Gateway → Booking Backend

Termasuk:
- current occupancy
- historical event
- idempotency
- outbox retry
```

Sprint 5 — Historical Analytics

```
None
- PHASE 10
- PHASE 11
```

Deliverable:

```
None
Analytics backend selesai:
- occupancy aggregate
- utilization formula
- occupancy chart API
- occupancy summary API
```

Sprint 6 — Google Calendar + Business Logic

```
None
- PHASE 12
- PHASE 13
```

Deliverable:

```
None
Business automation selesai:
- Google Calendar sync
- auto-release
- ad-hoc booking
- ghost meeting audit
```

Sprint 7 — Dashboard + Admin UI

```
None
- PHASE 14
- PHASE 15
```

Deliverable:

```
None
Frontend/admin selesai:
- real-time room status
- analytics chart
- ghost meeting report
- device provisioning UI
- binding UI
- token rotation UI
- outbox monitoring
```

Sprint 8 — QA, Hardening, Release

```
None
- PHASE 16
```

Deliverable:

```
None
MVP ready:
- integration tests
- failure tests
- security checks
- deployment scripts
- release checklist
```

Pekerjaan yang Bisa Paralel

Agar tim tidak idle, ini pembagian paralel yang aman.

Setelah Phase 1 selesai

```
None
Backend Booking Team:
- Phase 2

IoT Gateway Team:
- Phase 3

Edge Team:
- Phase 5.1 sampai 5.3

DevOps:
- CI/CD, env, deployment skeleton
```

Setelah Booking JWT dan Room API siap

```
None
IoT Gateway Team:
- Phase 4
- Phase 6
```

```
Booking Team:
- Phase 7

Edge Team:
- Phase 5.4 telemetry sender
```

Setelah occupancy end-to-end selesai

```
None
Data/Backend Team:
- Phase 10 + 11

Calendar/Backend Team:
- Phase 12 + 13

Frontend Team:
- Phase 14 + 15

QA:
- Phase 16 integration/failure tests
```

Jangan Dikerjakan Terlalu Awal

Ini penting supaya tidak buang waktu.

```
None
Jangan mulai dashboard analytics sebelum Analytics API siap.
Jangan mulai auto-release sebelum Google Calendar cancel event
siap.
Jangan mulai ad-hoc booking sebelum Google Calendar create event
siap.
Jangan mulai outbox dispatch penuh sebelum Booking Backend webhook
endpoint siap.
```

```
Jangan mulai device binding sebelum Booking Backend active rooms API siap.
Jangan mulai frontend admin binding sebelum IoT Gateway binding API siap.
```

MVP Completion Gate

MVP dianggap selesai kalau urutan ini sudah hijau:

```
None
1. Admin register IoT Gateway di Booking Backend.
2. Booking Backend generate clientId/clientSecret.
3. IoT Gateway bisa request JWT.
4. Booking Backend validate JWT dan scope.
5. Admin create room.
6. IoT Gateway fetch active rooms.
7. Admin create device di IoT Gateway.
8. IoT Gateway generate DeviceToken.
9. Edge memakai DeviceToken.
10. Admin bind device ke room.
11. Edge kirim telemetry.
12. IoT Gateway validate token.
13. IoT Gateway detect state transition.
14. IoT Gateway insert outbox.
15. IoT Gateway dispatch webhook.
16. Booking Backend process webhook idempotently.
17. RoomCurrentOccupancy ter-update.
18. RoomOccupancyEvent tersimpan.
19. RoomOccupancyAggregate terhitung.
20. Analytics API return data grafik.
21. Google Calendar sync berjalan.
22. Auto-release berjalan.
23. Ad-hoc booking berjalan.
24. SignalR update terkirim.
```

```
25. Dashboard menampilkan status dan grafik.
26. Outbox retry berhasil saat Booking Backend down.
27. Semua P0 test pass.
28. Security checklist pass.
29. Deployment script siap.
30. UAT sign-off.
```

Rekomendasi Praktis untuk OpenProject

Buat **Task** di bawah masing-masing User Story menggunakan prefix seperti ini:

```
None
[TASK-2.3.4] Add JWT claims: iss, aud, sub, gatewayId, clientId, scope
[TASK-4.5.4] Refresh DeviceRoomMapping cache after bind
[TASK-8.4.7] Test Booking Backend down → outbox pending → retry delivered
```

Lalu isi field:

```
None
Type: Task
Parent: User Story terkait
Priority: ikut parent story
Component: Booking Backend / IoT Gateway / Edge / Frontend / QA / DevOps
Version: MVP-x
Depends on: task sebelumnya
```

Dengan urutan di atas, developer bisa mulai dari Phase 0 lalu lanjut phase berikutnya tanpa bingung dependency mana yang harus dipenuhi dulu.